Aplicación de análisis de imágenes térmicas

Asistente de escritorio para radiometría, segmentación de ROI, análisis espectral de Fourier 2D y exportación DICOM

Documento técnico-académico de referencia

Descripción detallada del proyecto, fundamentación físico-matematica de cada algoritmo, desarrollo explicito de las fórmulas implementadas, justificación técnica de cada librería empleada, análisis crítico cuantitativo del comportamiento real de los méTodos y bibliografía enlazada y verificada.

Tabla 0.1. Ficha técnica del proyecto

Característica	Descripción
Proyecto	Herramienta Térmica — Asistente de análisis de imágenes térmicas
Lenguaje y versión	Python 3 (desarrollado y probado con CPython 3.14.7)
Interfaz gráfica	Tkinter + CustomTkinter (modo oscuro, escalado DPI de Windows)
Nucleo numérico	NumPy (matrices y FFT 2D) + OpenCV (operaciones de imagen)
Interoperabilidad	pydicom (DICOM) + ExifTool (metadatos radiométricos FLIR)
Cámaras soportadas	FLIR E6 (calibración lineal) y FLIR A3 (calibración de Planck)
Salida médica	DICOM Secondary Capture, MONOCHROME2, int16, RescaleSlope 0.01
Base analizada	Repositorio local, commit 1558801
Uso previsto	Tesis / trabajo académico: herramienta de apoyo al análisis termográfico

Como leer este documento

Los capítulos 2 y 4 contienen el nucleo teórico y algorítmico que un tribunal suele evaluar: la física de la radiometría infrarroja (2.1), el modelo de calibración de la cámara (4.2 y 4.3), la transformada de Fourier 2D y la descomposición en anillos (4.9) y el mecanismo de codificación DICOM (4.10). El capítulo 5 justifica cada librería empleada. El capítulo 6 presenta un análisis crítico **cuantitativo y reproducible** del comportamiento real de los algoritmos, con hallazgos que sostienen la sección de trabajo futuro. El capítulo 11 recoge todos los enlaces de referencia, verificados uno a uno.

Contenido

1. Introducción . . . 7

1.1 Motivación y problema . . . 7

1.2 Objetivos . . . 7

Objetivo general . . . 7

Objetivos específicos . . . 7

1.3 Alcance y consideraciones de uso . . . 8

1.4 Contribuciones del trabajo . . . 8

2. Marco teórico y fundamentos científicos . . . 9

2.1 Termografía infrarroja: principios físicos . . . 9

2.1.1 Radiación electromagnética y cuerpo negro . . . 9

2.1.2 Potencia total y ley de Stefan-Boltzmann . . . 9

2.1.3 Emisividad, reflexion y transmision . . . 9

2.1.4 Modelo radiométrico de la cámara: ecuación de calibración . . . 10

2.1.5 Corrección por distancia y reflectancia . . . 10

2.1.6 Métricas de calidad que condicionan el análisis . . . 10

2.2 El formato de archivo FLIR . . . 11

2.3 Segmentación de la region de interes . . . 11

2.3.1 Encuadre del problema . . . 11

2.3.2 Umbralización binaria . . . 11

2.3.3 Dibujado interactivo del contorno . . . 12

2.3.4 Alternativas de estado del arte y por que no se usan aquí . . . 12

2.4 Estadística descriptiva e histograma térmico . . . 12

2.4.1 Estadísticos de la ROI . . . 12

2.4.2 El histograma como descriptor . . . 12

2.5 Análisis de frecuencias espaciales: Fourier bidimensional . . . 13

2.5.1 Que cambia realmente la transformada . . . 13

2.5.2 La transformada discreta 2D . . . 13

2.5.3 El algoritmo de Cooley-Tukey . . . 14

2.5.4 Zero-padding . . . 14

2.5.5 Centrado del espectro: fftshift . . . 14

2.5.6 De amplitud a potencia, y por que se logaritma . . . 14

2.5.7 De pixeles a frecuencias físicas: la clave de la interpretación . . . 15

2.5.8 Fuga espectral y funciones de ventana . . . 16

2.5.9 El indicador de textura térmica . . . 16
2.6 Interoperabilidad: el estándar DICOM . . . 16
2.6.1 Que es DICOM . . . 16
2.6.2 Anatomía de un archivo DICOM . . . 16
2.6.3 Representación de valores: los VR . . . 17
2.6.4 Clase SOP: Secondary Capture . . . 17
2.6.5 Calibración lineal del valor de pixel . . . 17
2.6.6 Ventana de contraste: Window Center y Window Width . . . 18
2.6.7 Tipos de atributo y conformidad . . . 18

3. Descripción del sistema . . . 19
3.1 Arquitectura por capas . . . 19
3.2 Estructura del proyecto . . . 19
3.3 Modulos y sus contratos públicos . . . 20
3.4 Flujo de trabajo del usuario . . . 21
3.5 Modelo de datos en memoria . . . 21
3.6 Validación y manejo de errores . . . 22

4. Implementación de los algoritmos . . . 22
4.1 Extracción del dato radiométrico . . . 23
4.1.1 Planteamiento . . . 23
4.1.2 Resolución de la ruta de ExifTool . . . 23
4.1.3 Dos invocaciones distintas . . . 24
4.1.4 De los bytes a la matriz numérica . . . 25
4.2 Calibración radiométrica: del valor crudo a grados Celsius . . . 25
4.2.1 Selección del modelo . . . 25
4.2.2 Caso A: calibración lineal (FLIR E6) . . . 26
4.3 Caso B: ecuación de Planck (FLIR A3) . . . 27
4.3.1 Estructura del cálculo y su justificación numérica . . . 27
4.4 Aplicación de la region de interes . . . 28
4.4.1 Ajuste dimensional y umbral de binarización . . . 28
4.4.2 Estadísticos descriptivos . . . 29
4.4.3 Histograma de frecuencias térmicas . . . 29
4.5 Segmentación de la region de interes . . . 30
4.5.1 Vía manual: dibujo interactivo del contorno . . . 30
4.5.2 Vía automática: umbralización térmica . . . 32
4.6 Análisis espectral de Fourier 2D . . . 33

4.6.1 Encuadre . . . 33
4.6.2 Zero-padding a tamano óptimo . . . 34
4.6.3 Cálculo de la transformada . . . 34
4.6.4 Partición en tres anillos concéntricos . . . 35
4.6.5 Traducción de los radios a frecuencias físicas . . . 36
4.6.6 Interpretación y firma de la función . . . 37
4.6.7 Punto de entrada alternativo desde ficheros CSV . . . 37
4.7 Codificación DICOM . . . 38
4.7.1 El problema de la precision decimal . . . 38
4.7.2 Construcción del Dataset . . . 39
4.7.3 Descripción de la imagen . . . 40
4.7.4 Metadatos de calibración en la descripción del dispositivo . . . 40
4.7.5 Resultados de Fourier y datos del paciente . . . 41
4.7.6 Identificadores y modality . . . 42

5. Librerías empleadas y justificación técnica . . . 43
5.1 Mapa de dependencias . . . 43
5.2 NumPy: el nucleo numérico . . . 44
5.2.1 Por que y no listas de Python . . . 44
5.2.2 Funciones concretas del proyecto . . . 44
5.3 OpenCV: operaciones de imagen . . . 45
5.3.1 La elección de INTER_NEAREST para la máscara . . . 45
5.4 Matplotlib: visualización del histograma . . . 46
5.5 Pillow: decodificación de la imagen cruda . . . 46
5.6 CustomTkinter: interfaz gráfica . . . 46
5.7 pydicom: interoperabilidad médica . . . 47
5.7.1 Que aporta frente a escribir el archivo a mano . . . 47
5.7.2 Verificación de la round-trip . . . 47
5.8 ExifTool: lectura de metadatos propietarios . . . 48
5.9 Síntesis: cobertura y solapamientos . . . 48

6. Análisis crítico y verificación experimental . . . 49
6.1 Metodología de verificación . . . 49
6.1.1 Preguntas a responder . . . 49
6.1.2 Construcción de la imagen de prueba . . . 49
6.2 P1: ¿el reparto en anillos es correcto? . . . 50
6.3 P2: ¿el algoritmo responde a la textura? . . . 50

6.4 Causa raiz: la fuga espectral del borde de la máscara . . . 51
6.4.1 El mecanismo . . . 51
6.4.2 Cuantificación del efecto . . . 52
6.5 P3: dependencia de los radios . . . 52
6.6 P4: sensibilidad a la posición del objeto . . . 53
6.7 Propuesta de corrección . . . 53
6.7.1 Tres alternativas evaluadas . . . 53
6.7.2 La corrección recomendada . . . 54
6.8 Síntesis de hallazgos . . . 55

7. Verificación y pruebas . . . 55
7.1 Estrategia de pruebas . . . 55
7.2 N2: verificación de la lógica de Fourier . . . 56
7.3 N1: verificación de la interoperabilidad DICOM . . . 56
7.3.1 Prueba de ida y vuelta . . . 56
7.3.2 Atributos presentes y verificados . . . 57
7.3.3 Brechas de conformidad: atributos que faltan . . . 58
7.3.4 Defecto detectado: longitud de la descripción de equipo . . . 59
7.4 N4: verificación de la calibración radiométrica (pendiente) . . . 59
7.5 Resumen de la campana de pruebas . . . 60

8. Discusion, generalización y trabajo futuro . . . 60
8.1 Lo que el trabajo demuestra . . . 60
8.2 Limitaciones del estudio . . . 61
8.3 Trabajo futuro inmediato . . . 61
8.3.1 Correcciones de prioridad alta . . . 61
8.3.2 Mejoras de medio plazo . . . 62
8.3.3 Mejoras de largo plazo . . . 62
8.4 Consideraciones éticas y de uso . . . 62

9. Conclusiones . . . 63
Sobre los objetivos . . . 63
Sobre el hallazgo principal . . . 63
Sobre las lecciones metodológicas . . . 63
Sobre el alcance de las afirmaciones . . . 63

10. Trabajo futuro . . . 65

10.1 Correcciones inmediatas . . . 65
10.2 Mejoras del algoritmo de análisis . . . 65
10.3 Mejoras de la interfaz . . . 65
10.4 Mejoras de la interoperabilidad y la gestion clinica . . . 66
10.5 Plan de validación clinica . . . 66

11. Referencias y enlaces verificados . . . 67

11.1 Estándar DICOM . . . 67
11.2 pydicom . . . 67
11.3 ExifTool y formato FLIR . . . 67
11.4 NumPy: transformada de Fourier y arrays . . . 67
11.5 OpenCV . . . 68
11.6 Matplotlib, Pillow y CustomTkinter . . . 68
11.7 Teoría y contexto . . . 68
11.8 SOFTWARE Y ENTORNO DE DESARROLLO . . . 69
11.9 Nota sobre la verificacion de los enlaces . . . 69

A. Anexo A: pseudocodigo de los algoritmos . . . 70

A.1 [A1] Localizacion de ExifTool . . . 70
A.2 [A2] Extraccion del dato radiometrico . . . 70
A.3 [A3] Calibracion lineal (FLIR E6) . . . 70
A.4 [A4] Calibracion por ecuacion de Planck (FLIR A3) . . . 71
A.5 [A5] Estadisticos e histograma de la ROI . . . 71
A.6 [A6] Segmentacion interactiva . . . 72
A.7 [A7] Segmentacion por umbral termico . . . 72
A.8 [A8] Analisis de Fourier 2D . . . 73
A.9 [A9] Exportacion DICOM . . . 74

B. Anexo B: guiones de verificacion . . . 75

B.1 Bucle comun de calculo del reparto en bandas . . . 75
B.2 P1: clasificacion de frecuencias de verdad conocida . . . 75
B.3 P2: sensibilidad a la textura termica . . . 76
B.4 N1: prueba de ida y vuelta DICOM . . . 76
B.5 Entorno de ejecución . . . 77

1. Introducción

1.1 Motivación y problema

La termografía infrarroja en el rango de onda larga (LW, 8–14 µm) mide la radiación térmica emitida por la superficie de los tejidos y la convierte en un mapa de temperaturas. Frente a la radiografía o la resonancia magnética ofrece una ventaja decisiva en oncología: es una técnica no ionizante, indolora, en tiempo real, portable y de coste reducido. Su limitación fundamental es que una imagen térmica **no es diagnóstica por si sola**: la temperatura superficial es un indicador indirecto de la actividad metabólica y de la perfusion, y es precisamente esa cadena de inferencia la que obliga a un post-procesado riguroso y, sobre todo, a una extracción correcta de los datos radiométricos.

El problema técnico que este proyecto resuelve es triple:

Tabla 1.1. Los tres problemas técnicos que resuelve el proyecto

(a) El dato radiométrico esta escondido. Una imagen FLIR es un JPEG cuyo contenido visible es una paleta falsa de colores. La temperatura de cada pixel no esta en los pixeles: esta codificada en un *MakerNote* propietario (constantes de Planck, emisividad, temperatura reflejada) y en un flujo binario aparte, la etiqueta `RawThermalImage`. Sin decodificar ese bloque, una «termografía» es solo una fotografía de colores sin valor médico.
(b) La region de interes hay que aislarla. La imagen completa contiene fondo, ropa, aire y el tejido de interes. Cualquier estadístico calculado sobre el marco completo carece de sentido: la temperatura media de la escena no es la temperatura media de la lesion.
(c) El resultado debe ser interoperable. Un valor en grados Celsius aislado en una pantalla no puede compararse, auditarse ni archivarse. Para que el dato entre en la historia clinica del paciente debe expresarse en un formato que los visores médicos ya saben leer: DICOM.

1.2 Objetivos

Objetivo general

Desarrollar una aplicación de escritorio en Python que realice el ciclo completo de análisis termográfico radiométrico sobre imágenes FLIR — desde la extracción del dato crudo hasta la exportación en formato médico interoperable — codificando un resultado cuantitativo, trazable y reproducible, y documentando explicitamente el fundamento físico y matematico de cada transformación que aplica.

Objetivos específicos

- **O1.** Extraer de forma fiable la matriz radiométrica cruda y las constantes de calibración de la fabrica mediante ExifTool, resolviendo la ruta del ejecutable de forma independiente del directorio de trabajo.
- **O2.** Convertir el valor crudo del sensor a temperatura en grados Celsius mediante el modelo de calibración correspondiente al modelo de cámara: lineal para FLIR E6, ecuación de Planck para FLIR A3.
- **O3.** Proporcionar dos vías de segmentación de la region de interes: manual interactiva (lazo libre con el raton) y automática (umbral térmico), más la carga de máscaras preexistentes.
- **O4.** Calcular estadísticos descriptivos robustos de la ROI y un histograma de frecuencias térmico con anotación de porcentajes por intervalo.
- **O5.** Implementar el análisis de textura por espectro de Fourier 2D, con descomposición de la energía en tres bandas concéntricas: bajas, medias y altas frecuencias espaciales.
- **O6.** Exportar el resultado a un archivo DICOM conforme al estándar, preservando la precision decimal y transportando los metadatos de calibración de la cámara.

- **O7.** Justificar tecnicamente cada dependencia de software elegida y evaluar criticamente, con simulación reproducible, el comportamiento real de los algoritmos implementados.

1.3 Alcance y consideraciones de uso

La aplicación es una **herramienta de análisis de apoyo**, no un sistema de diagnóstico. No sustituye a la exploración física, la mamografía, la ecografía ni la biopsia; sus resultados son cuantitativos y no constituyen por si mismos un diagnóstico. El ámbito de la herramienta es el de la termografía clinica de superficie y el de la investigación en análisis de textura sobre imagen térmica. Cualquier uso diagnóstico requiere validación clinica previa (capítulo 8) y cumplimiento de la normativa de protección de datos, dado que los archivos DICOM generados contienen datos identificativos del paciente.

1.4 Contribuciones del trabajo

- Una **cadena radiométrica completa y auditable**: JPEG → ExifTool → matriz cruda → modelo de calibración → grados Celsius, con validación explicita del dominio numérico en cada paso y de la unidad de las constantes.
- Una **descomposición de la energía espectral en tres anillos concéntricos**, junto con la traducción de los radios en pixeles a frecuencias espaciales físicas (ciclos por milimetro), que es lo que hace comparables los resultados entre cámaras y entre ROIs de distinto tamano.
- Una **especificación de codificación DICOM** que preserva dos decimales de temperatura en un contenedor entero de 16 bits mediante el mecanismo estándar RescaleSlope.
- Un **análisis crítico cuantitativo** (capítulo 6) que identifica por que la versión actual del indicador de energía de banda es poco discriminante, lo reproduce con numeros y propone una variante corregida y validable.

2. Marco teórico y fundamentos científicos

2.1 Termografía infrarroja: principios físicos

2.1.1 Radiación electromagnética y cuerpo negro

Todo cuerpo con temperatura absoluta T > 0 K emite radiación electromagnética. Esa radiación se modela con el **cuerpo negro**, emisor ideal que absorbe toda la radiación incidente. La **ley de radiación de Planck** describe la exitancia espectral, es decir la potencia emitida por unidad de área y por unidad de longitud de onda:

$$E(\lambda, T) = \sqrt{(\pi)} \cdot c_1 \cdot \lambda^5 \cdot [\exp(h \cdot c / (\lambda \cdot k_B \cdot T)) - 1]^{-1}$$

$E(\lambda, T)$ en $W \cdot m^{-2} \cdot m^{-1}$; $c_1 = 2 \cdot c^2 = 1{,}191 \times 10^{-16}$ $W \cdot m^2$. Las tres constantes físicas son c (velocidad de la luz), h (constante de Planck) y k_B (constante de Boltzmann). La única variable del modelo es la temperatura T.

El máximo de esta curva se desplaza con la temperatura según la **ley de desplazamiento de Wien**:

$$\lambda_{max} \cdot T = b = 2{,}898 \times 10^{-3} \text{ m} \cdot \text{K}$$

A la temperatura de la piel humana (310 K = 37 °C) el máximo de emision cae en $\lambda_{max} \approx 9{,}35$ µm, dentro de la banda térmica LW. Esta es la razon física de que las cámaras clinicas operen en la ventana atmosférica de 8 a 14 µm: ahi la radiación propia del cuerpo es intensa y la absorción atmosférica es pequena, lo que se conoce como **ventana atmosférica infrarroja**.

2.1.2 Potencia total y ley de Stefan-Boltzmann

Integrando la ley de Planck en todas las longitudes de onda se obtiene la exitancia total:

$$M = \sigma \cdot T^4$$

$\sigma = 5{,}670 \times 10^{-8}$ $W \cdot m^{-2} \cdot K^{-4}$. La dependencia con la cuarta potencia explica por que una variación de 1 K a 310 K implica un cambio relativo de 4/310 ≈ 1,3 % en la potencia emitida, y por que la sensibilidad térmica de las cámaras (NETD del orden de 0,03 a 0,05 °C) es un parámetro crítico de diseno del sistema completo.

2.1.3 Emisividad, reflexion y transmision

Un cuerpo real no es un cuerpo negro. Su comportamiento se describe por la **emisividad** ε, definida como la fracción de la radiación de cuerpo negro que el material emite a su misma temperatura. Por la ley de Kirchhoff, la radiación que no se emite se refleja o se transmite. La radiometría que alcanza el detector se descompone en dos terminos:

$$L_\lambda = \tau_\lambda \cdot \varepsilon_\lambda \cdot M_\lambda(T_{obj}) + (1 - \varepsilon_\lambda) \cdot L_{\lambda amb}$$

Radiación propia del objeto que atraviesa la atmosfera, más la radiación ambiental reflejada por el objeto. El segundo termino explica por que un ajuste erroneo de emisividad desplaza la temperatura medida: es el error sistemático más frecuente en termografía clinica.

- **Piel humana:** ε ≈ 0,98 en el rango LW; es prácticamente un cuerpo negro. Los protectores transparentes y los preparados médicos reducen la emisividad efectiva y sesgan la lectura.
- **Temperatura reflejada aparente** (etiqueta `ReflectedApparentTemperature`): es la radiación ambiental que la superficie refleja. Si el operador no la fija, la cámara la estima y comete error en superficies reflectantes.
- **Transmision atmosférica** $\tau_\lambda < 1$: FLIR la modela con los parámetros `AtmosphericTransAlpha1/2`, `AtmosphericTransBeta1/2` y `AtmosphericTransX`, que la cámara resuelve internamente antes de escribir el

archivo.

2.1.4 Modelo radiométrico de la cámara: ecuación de calibración

El detector no mide temperatura: mide un numero entero proporcional a un voltaje, que se almacena como *valor crudo* (RAW). La relación entre el valor crudo y la temperatura absoluta del objeto esta incarnada en la **ecuación de calibración** que el fabricante graba en la propia imagen, en forma de constantes. La que Teledyne FLIR utiliza en sus microbolométricos, y que implementa el algoritmo A3 de esta herramienta, es:

$$RAW = R_1 / \{ R_2 \cdot [\exp(B / T) - F] \} - O$$

Con T en kelvin y R_1, R_2, B, F y O como constantes de fabricación. Despejando T se obtiene la forma exacta que usa el código (algoritmo A3, sección 4.3).

Las constantes se leen del MakerNote con los nombres de etiqueta `PlanckR1`, `PlanckR2`, `PlanckB`, `PlanckF`, `Planck0`. Para las cámaras de la serie E, más antiguas, el fabricante emplea en su lugar una aproximación **lineal** entre el rango de valores crudos y el rango de temperatura (algoritmo A2, sección 4.2).

2.1.5 Corrección por distancia y reflectancia

La cámara se calibra opticamente suponiendo que el objeto esta en su distancia focal nominal. Si el objeto esta más lejos, la radiación que llega al detector proviene de un cono más estrecho y la temperatura aparente se desvia. FLIR corrige este efecto con la relación:

$$\Delta T_{corregida} = (\varepsilon_{obj}(1-\varepsilon_{atm}) - \varepsilon_{refl}(1-\alpha_{refl})) \cdot \Delta T_{aparente} \cdot (D / \sqrt{(D^2 + H^2)})^k$$

D = distancia al objeto, H = altura de la cámara sobre el objeto, k ≈ 0,99 para óptica de 14 µm; ΔT es la diferencia de temperatura medida.

> **Consecuencia metodológica**
>
> Que la cámara resuelva la corrección de distancia significa que el metadato que la aplicación lee y guarda corresponde a una imagen **ya corregida por la cámara**. Por tanto, las constantes de Planck presentes en el archivo son precisamente las que la cámara utilizo para esa corrección, y la cadena radiométrica reconstruida en la sección 4.3 resulta coherente pixel a pixel siempre que se utilicen las constantes leidas del propio archivo. Este es el argumento que sustenta la decision de leer las constantes del archivo en lugar de fijarlas en el código (capítulo 5.2).

2.1.6 Métricas de calidad que condicionan el análisis

Tabla 2.1. Parámetros físicos radiométricos de la termografía infrarroja

Parámetro	Definición	Por que importa en este trabajo
NETD *Noise Equivalent Temperature Difference*	Diferencia mínima de temperatura que el sensor puede distinguir por encima del ruido.	Fija el *piso* de las diferencias que el análisis de textura puede descubrir. Con un NETD de 0,05 °C no es posible discriminar texturas cuya amplitud este por debajo de ese valor.
MTF / resolución espacial *Modulation Transfer Function*	Capacidad del conjunto óptica + detector para reproducir un detalle de tamano dado.	Determina la **frecuencia de Nyquist** útil y, con ella, hasta que frecuencia espacial es fisicamente medible (sección 2.5.7).
`Pixel pitch` *paso de pixel*	Distancia física entre los centros de dos pixeles consecutivos del detector.	Es la constante que traduce los radios de los anillos (en pixeles) a **frecuencias físicas** (sección 4.9.4). Imprescindible para comparar estudios.
Rango dinámico y bits útiles	Numero de niveles distintos que el detector codifica en el valor crudo.	Determina la cuantización del dato y, con ella, el error máximo de temperatura por cuantización (sección 4.3.2).

2.2 El formato de archivo FLIR

Una radiometría FLIR se guarda como un JPEG convencional cuyo segmento de imagen es una paleta falsa de colores (ironbow, ironbow2, rainbow, gris, etc.). Los datos útiles viven en otros lugares del archivo:

- **Segmento APP1 con identificador FLIR** (o el *MakerNote* del fabricante): contiene la estructura `CameraInfo` con las constantes de calibración, la emisividad, la distancia al objeto, la temperatura reflejada y las condiciones atmosféricas.
- **Flujo binario** `RawThermalImage`: los valores enteros del sensor, uno por pixel, ordenados por filas. No es una imagen: es un bloque de bytes sin comprimir que hay que decodificar como imagen monocroma de 16 bits (o de 8 según el modelo).
- **Rango del sensor**: las etiquetas `RawValueRangeMin` y `RawValueRangeMax` delimitan los extremos usados por la calibración lineal.
- **Constantes de calibración**: `PlanckR1`, `PlanckR2`, `PlanckB`, `PlanckF`, `PlanckO`, las cinco magnitudes de la ecuación de Planck.

Esta estructura es **propietaria y no forma parte de EXIF estándar**. Es la razon técnica determinante de la elección de ExifTool como lector (capítulo 5.2).

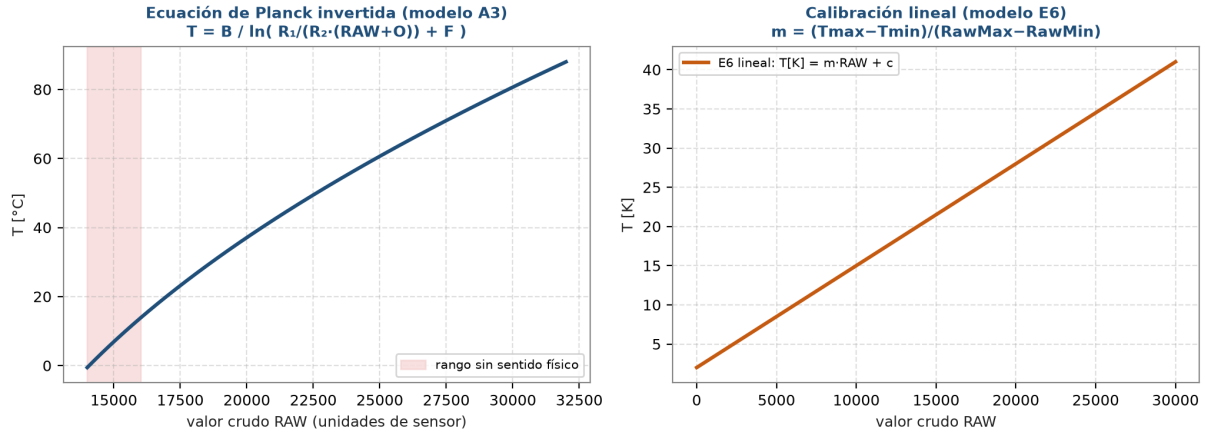


Figura 1. *Curvas de calibración RAW → T de los dos modelos soportados. (a) Modelo de Planck (A3): relación no lineal; la zona sombreada senala el rango de valores crudos que produce un argumento logarítmico no positivo y, por tanto, una temperatura sin sentido físico. (b) Modelo lineal (E6): relación recta entre el valor crudo y la temperatura absoluta.*

2.3 Segmentación de la region de interes

2.3.1 Encuadre del problema

La segmentación en este dominio es un problema de **extracción de una region de interes** (ROI, *Region of Interest*). Su dificultad es característica: en imagen térmica no hay bordes de contraste de intensidad como en imágenes naturales, porque la paleta de color mapea la temperatura de forma monotona pero no lineal, y porque el contraste térmico entre tejido sano y tejido tumoral es frecuentemente inferior a 2 °C.

2.3.2 Umbralización binaria

La forma más simple y la única realmente trazable en tiempo real es la umbralización con un umbral térmico τ fijado por el especialista:

$$M(x, y) = 255 \text{ si } T(x, y) \geq \tau; \quad 0 \text{ si no}$$

La función `np.where(T >= umbral, 255, 0)` *de NumPy es exactamente este if vectorizado sobre toda la matriz. El resultado es una imagen binaria de un byte por pixel.*

La ventaja decisiva de esta técnica en termografía es que el criterio es **explicito y auditable**: el umbral de 36,5 °C queda registrado y cualquier lector puede reproducir exactamente la misma máscara. Su limitación es la sensibilidad a la heterogeneidad: si el umbral cae sobre una pendiente térmica suave, la ROI se trunca de forma abrupta y se incluye o se excluye tejido de forma sistémica.

2.3.3 Dibujado interactivo del contorno

La alternativa manual evita por completo el problema del umbral. El operador traza un contorno de forma libre con el raton; ese contorno se convierte en un **poligono** que se rellena usando la regla par-impar: un punto pertenece al interior si un rayo que sale de el cruza un numero impar de aristas del poligono. En OpenCV esto es una sola llamada, `cv2.drawContours(mascara, [contorno], -1, 255, cv2.FILLED)`.

2.3.4 Alternativas de estado del arte y por que no se usan aquí

Tabla 2.2. Métodos de segmentación de la región de interés y criterio de exclusión

Método	Principio	Por que no se incluyo en esta versión
Umbral de Otsu	Maximiza la varianza inter-clase entre fondo y objeto; el umbral se deduce solo de la imagen.	Exige un histograma bimodal. El histograma térmico de una lesion es tipicamente monomodal o trimodal, de modo que el criterio de maximización no es valido. Además el umbral debe ser reproducible y comparable entre estudios, cosa que Otsu no garantiza.
k-medias sobre intensidades	Agrupa pixeles en k clases minimizando la inercia intra-cluster.	El parámetro k es arbitrario y el resultado sensible a la inicialización; no garantiza la reproducibilidad que exige un protocolo clinico.
Crecimiento de regiones	Crece desde una semilla comparando cada pixel con sus vecinos.	Sensible a la heterogeneidad térmica interna: se filtra por los pixeles más frios de la propia lesion y produce ROI fragmentadas.
Redes convolucionales (U-Net, DeepLab)	Aprenden la segmentación a partir de ejemplos anotados.	Requieren un conjunto de datos etiquetados de la institución. Es la vía correcta para la versión 2.0 y se propone como trabajo futuro (capítulo 10).
Modelos deformables	Ajustan un contorno suave minimizando una energía de borde.	Requieren inicialización y criterios de parada; no aportan sobre el poligono libre para este caso de uso concreto.

> **Decision de ingeniería**
>
> Se implementaron las dos vías clásicas más robustas (umbral y poligono) más la carga de una máscara existente. Son deterministas, auditables y no requieren entrenamiento. Es una elección consciente: para un protocolo clinico la reproducibilidad importa más que la sofisticación del segmentador. La segmentación automática con modelos aprendidos es la línea natural de continuidad del trabajo (capítulo 10).

2.4 Estadística descriptiva e histograma térmico

2.4.1 Estadísticos de la ROI

Sobre el conjunto de N pixeles de la ROI se calculan los tres estadísticos que resumen la distribución:

$$T_{min} = \min_i T_i \qquad T_{max} = \max_i T_i \qquad \bar{T} = (1/N) \cdot \Sigma_{i=1..N} T_i$$

Con $N = \Sigma_{y,x} M(x,y)$, es decir el numero total de pixeles con $M = 1$.

El uso de las funciones `nanmin / nanmax / nanmean` de NumPy, en lugar de `min / max / mean`, es una decision de robustez: si algun pixel llegara como *NaN* (por ejemplo, tras un logaritmo invalido), esas funciones lo ignoran en lugar de propagar el error a toda la cadena de resultados. Esta es la razon de la validación previa del dominio logarítmico en el algoritmo A3.

2.4.2 El histograma como descriptor

El histograma de frecuencias reune la distribución de temperaturas de la ROI en intervalos (*bins*) de anchura constante. Con K = 25 intervalos y extremos t_{min} y t_{max}:

$$\Delta = (t_{max} - t_{min}) / K \qquad n_k = \#\{ i : t_{min} + (k-1)\Delta \le T_i < t_{min} + k\Delta \}$$

y el porcentaje que se rotula sobre cada barra del gráfico es la frecuencia relativa:

$$p_k = (n_k / N) \cdot 100 \ \%$$

La etiqueta impresa sobre cada barra combina tres datos: el conteo absoluto de pixeles, el porcentaje y la temperatura de inicio del intervalo. De este modo la gráfica es autosuficiente para una lectura clinica sin recurrir a la tabla de datos.

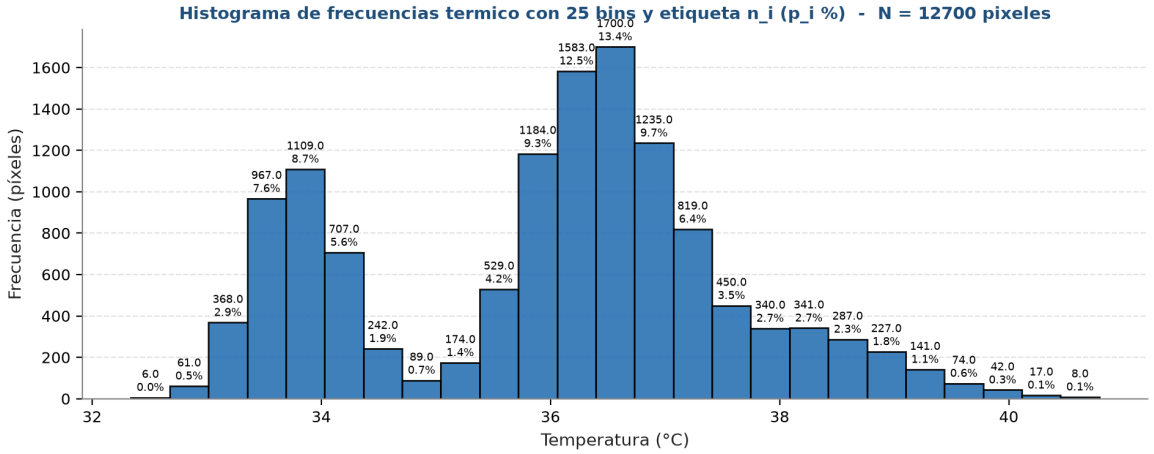


Figura 2. *Histograma térmico de ejemplo con K = 25 intervalos. Sobre cada barra se anade la etiqueta n_k (conteo de pixeles) y p_k (porcentaje sobre N). La distribución bimodal típica de una ROI con tejido sano y foco caliente queda inmediatamente visible.*

El histograma cumple una función concreta: la **asimetría**. En tejido sano la distribución es unimodal y aproximadamente simétrica; ante un foco caliente con hiperemia se produce un hombro o un segundo modo hacia temperaturas altas, y esa asimetría positiva es un indicador de hipertermia focal. El histograma es por tanto un descriptor cualitativo y cuantitativo de primer orden, coherente con la cuantificación espectral que desarrolla el algoritmo A9.

2.5 Análisis de frecuencias espaciales: Fourier bidimensional

2.5.1 Que cambia realmente la transformada

La transformada de Fourier sustituye la coordenada espacial por la coordenada de frecuencia. En el dominio espacial, T(x, y) dice cuanto calor hay en cada punto. En el dominio de Fourier, F(u, v) dice que escala de detalle contiene la imagen. La descomposición es exacta y completa: la imagen es una suma de cosenos y senos de todas las frecuencias posibles, y el espectro indica cuanto peso tiene cada una.

2.5.2 La transformada discreta 2D

Para una imagen de M filas y N columnas, la transformada de Fourier discreta bidimensional se define como:

$$F(u, v) \ = \ \Sigma_{x=0..M-1} \Sigma_{y=0..N-1} f(x, y) \cdot \exp[-2\pi \cdot i \cdot (u \cdot x/M + v \cdot y/N)]$$

$F(u, v)$ es un numero **complejo**: $F(u, v) = |F(u, v)| \cdot e^{i\varphi}$. *El* **modulo** $|F(u, v)| = \sqrt{(Re(F)^2 + Im(F)^2)}$ *es la amplitud del componente; el* **argumento** φ *es la fase. Esta es exactamente la convención de signo que implementa NumPy.*

La propiedad de **separabilidad** es lo que hace viable el cálculo:

$$F(u, v) \ = \ F_x\{ F_y\{ f \} \} \qquad O(M \cdot N \cdot M \cdot N) \ \rightarrow \ O(M \cdot N \cdot (M + N))$$

La transformada 2D se descompone en dos transformadas 1D sucesivas, una por filas y otra por columnas. Multiplicando la complejidad de ambas, el costo cae de $O(M^2N^2)$ a $O(M{\cdot}N{\cdot}(M + N))$.

2.5.3 El algoritmo de Cooley-Tukey

Sobre esa base, el algoritmo de Cooley y Tukey (1965) reduce el cálculo a $O(N{\cdot}\log_2 N)$ descomponiendo recursivamente la transformada de longitud N en dos transformadas de longitud N/2, más una combinación en O(N) conocida como la *mariposa*. Para una ROI de 120×160 pixeles la diferencia frente a la DFT ingenua es de tres ordenes de magnitud. NumPy implementa esta variante en `np.fft.fft2` y `np.fft.fftshift`.

2.5.4 Zero-padding

La FFT alcanza su velocidad máxima cuando la longitud de la transformada es un producto de potencias de los factores primos 2, 3, 5 y 7. Una ROI de 137×151 pixeles, por ejemplo, obliga a un algoritmo mixto mucho más lento que una dimension de 140 o de 150. La práctica estándar es rellenar con ceros hasta el tamano óptimo. OpenCV expone esa cuenta en `cv2.getOptimalDFTSize(n)`, que devuelve el menor valor factible mayor o igual que n, y en `cv2.copyMakeBorder`, que realiza el relleno. Comprobado empiricamente en este proyecto: `getOptimalDFTSize(49) = 50` y `getOptimalDFTSize(63) = 64`, lo que confirma que el factor 7 forma parte de la optimización.

> **Importante: el zero-padding no crea información**
>
> Rellenar con ceros **no anade ni quita resolución espacial**. Solo alinea la longitud con un tamano computacionalmente eficiente. La resolución en ciclos por milimetro la fijan el numero de pixeles útiles de la ROI y el paso de pixel del detector, no el tamano del array tras el relleno. Confundir ambas magnitudes es un error frecuente al interpretar estos espectros.

2.5.5 Centrado del espectro: fftshift

La DFT coloca la frecuencia cero (la componente continua o DC) en la esquina superior izquierda. Para que la máscara concéntrica del algoritmo A9 pueda referirse a un único centro, `np.fft.fftshift` reordena el array trasladando esa esquina al centro, de modo que las frecuencias crecen linealmente hacia fuera:

$$F_s(u, v) = F(\,(u - u_0) \bmod M\,,\,(v - v_0) \bmod N\,) \quad \text{con} \quad u_0 = \lfloor M/2 \rfloor,\ \ v_0 = \lfloor N/2 \rfloor$$

A partir de aquí, la distancia al centro del array es una medida directa de la frecuencia espacial: el eje de coordenadas del array coincide con el eje de frecuencias, y la distancia de cada celda al centro se convierte en un radio en el dominio de la frecuencia.

2.5.6 De amplitud a potencia, y por que se logaritma

El espectro de amplitud |F| describe la intensidad de cada componente; el **espectro de potencia** $|F|^2$ describe la energía y cumple la identidad de Parseval. La aplicación usa |F| porque es la magnitud del vector complejo y la lectura más directa, pero conviene ser consciente de la diferencia: para energía por banda, $|F|^2$ es la elección estadisticamente correcta, y |F| tiende a subestimar el peso relativo de las altas frecuencias.

$$\text{Parseval:}\quad \Sigma_{u,v}\,|F(u,v)|^2 = M{\cdot}N \cdot \Sigma_{x,y}\,|f(x,y)|^2$$

La energía de la imagen es la misma en los dos dominios, salvo el factor M·N que corresponde al zero-padding. De aquí se deduce la normalización canónica de un espectro de potencia: $|F(u,v)|^2 / (M{\cdot}N)$. El código actual no normaliza, lo cual no altera los porcentajes relativos entre bandas, porque el factor se cancela en el cociente, pero si impide comparar energía absoluta entre imágenes de distinto tamano.

Además, la diferencia dinámica entre el centro y la periferia del espectro es de varios ordenes de magnitud: la componente DC vale T·M·N mientras que un coeficiente de alta frecuencia puede valer del orden de 10^{-3}.

Mostrar |F| en escala lineal daría una imagen con un único pixel blanco. Por eso se aplica la compresion logarítmica:

$$I_{vis}(u, v) = \log (1 + |F(u, v)|)$$

Transformación monotónica que **conserva el orden** *(no altera que frecuencia domina) pero comprime el rango dinámico para hacerlo visible. Es una operación de visualización, no de análisis: los porcentajes de energía se calculan sobre |F|, no sobre I_{vis}. En el código es la línea* `np.log(1 + magnitud)` *seguida de* `cv2.normalize(..., 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)`.

2.5.7 De pixeles a frecuencias físicas: la clave de la interpretación

Este es el punto más importante para la interpretación clinica, y también el más fácil de entender mal. El radio r que el algoritmo A9 mide **no es una magnitud física**: es una distancia en el array de frecuencias, es decir un numero de *bins*. Para convertirlo a una frecuencia espacial hay que conocer dos cosas más: el paso de pixel p del detector y la anchura N del array tras el zero-padding.

La relación se obtiene del muestreo. Un patron de periodo Λ milimetros se extiende a lo largo de N·p milimetros de la escena y por tanto completa exactamente N·p/Λ ciclos; la transformada discreta coloca su energía en el bin r = N·p/Λ. Despejando:

$$r = N \cdot p / \Lambda \quad \square \quad \Lambda = N \cdot p / r \quad y \quad f = 1/\Lambda = r / (N \cdot p) \quad [\text{ciclos/mm}]$$

p es el paso de pixel en milimetros, r el radio en bins y N la anchura del array de la transformada. En dos dimensiones la relación exacta es elipse: $f_x = r_x/N$ y $f_y = r_y/M$ en ciclos por pixel, de modo que el radio circular del código corresponde a $f = \sqrt{((r_x/N)^2 + (r_y/M)^2)}$. Para transformaciones casi cuadradas la aproximación f = r/N es suficiente.

> **Consecuencia: el mismo radio significa cosas distintas en imágenes distintas**
>
> Como r depende de N, **un radio fijo no corresponde a una frecuencia física fija**. Recortar una ROI más pequena reduce N y desplaza hacia dentro la frecuencia que ese bin representa; comparar el mismo par (15, 50) entre dos ROIs de distinto tamano **no** compara la misma banda. Por eso el código guarda los radios usados junto al resultado, y por eso el capítulo 7 incluye una verificación con senos de periodo conocido: es la única forma de fijar la correspondencia entre bins y escalas anatómicas.

La tabla siguiente aplica la relación a N = 320 bins, que es el valor que produce `getOptimalDFTSize` para una ROI de 320 pixeles de ancho, la medida habitual en microbolométricos de 320×240. Se incluye también el limite de Nyquist para comparar:

Tabla 2.3. Relación entre el paso del píxel y el periodo espacial de la rejilla de anillos

Paso de pixel p	r_1 = 15 bins → periodo Λ (f)	r_2 = 50 bins → periodo Λ (f)	Limite de Nyquist 0,5 ciclos/pixel	Escala anatómica
17 µm (0,017 mm)	0,363 mm (2,76 mm^{-1})	0,109 mm (9,19 mm^{-1})	0,068 mm (29,4 mm^{-1})	Detalle superficial muy fino
25 µm (0,025 mm)	0,533 mm (1,88 mm^{-1})	0,160 mm (6,25 mm^{-1})	0,050 mm (20,0 mm^{-1})	Arteriolas y detalle cutaneo
34 µm (0,034 mm)	0,725 mm (1,38 mm^{-1})	0,218 mm (4,60 mm^{-1})	0,068 mm (14,7 mm^{-1})	**Red vascular superficial: neovascularización**
41 µm (0,041 mm)	0,875 mm (1,14 mm^{-1})	0,262 mm (3,81 mm^{-1})	0,082 mm (12,2 mm^{-1})	Vasos de mayor calibre y bordes de lesion

Con un paso de pixel de 34 µm y N = 320, los radios por defecto del algoritmo A9 cubren respectivamente periodos de **0,725 mm y 0,218 mm**, es decir 1,38 y 4,60 ciclos por milimetro. La primera banda cae en la escala de la **red vascular superficial**, que es la firma termográfica de la **neovascularización**: la formación de vasos de nuevo caliber anomal, más densos, asociada a la angiogenesis que sostiene un tumor en crecimiento. Esta

traducción es lo que convierte un parámetro expresado en bins en una magnitud con significado fisiológico, y constituye el nexo entre el algoritmo de Fourier y la hipótesis clinica.

Obsérvese también que la banda externa, con 4,60 ciclos por milimetro, esta muy por debajo del limite de Nyquist de 14,7 ciclos por milimetro que impone el muestreo: queda alrededor del 31 % de la frecuencia máxima representable. Es decir, **los tres anillos son espacialmente resolubles**; el problema de la banda alta no es que capture frecuencias que el sensor no puede registrar, sino que en la ROI recortada esa zona se satura con el artefacto de borde descrito en el capítulo 6.

2.5.8 Fuga espectral y funciones de ventana

Un espectro obtenido de una ROI recortada no esta libre de artefactos. Si el recorte introduce una discontinuidad en el borde —tipicamente: el fondo de la imagen es forzado a 0 °C mientras que el tejido esta a 33 °C— ese escalon es un patron espacial de frecuencia altísima y reparte energía por todo el espectro, enmascarando la textura real. Este fenomeno se denomina **fuga espectral** y se corrige multiplicando la imagen por una **ventana** que se desvanace hacia los bordes.

Tabla 2.4. Funciones de ventana disponibles y su efecto sobre la resolución en frecuencia

Ventana	Fórmula w[n]	Lobulo principal	Ancho de lobulo	Uso recomendado
Rectangular	w = 1	— (máxima fuga)	—	Ruido blanco; no recomendada con ROI recortada
Hann	0,5 · (1 − cos(2πn/(N−1)))	−31,5 dB	4 muestras	**Recomendada** para análisis de textura sobre ROI
Hamming	0,54 − 0,46·cos(2πn/(N−1))	−42,7 dB	3 muestras	Mejor rechazo de fuga, resolución algo peor
Blackman	0,42 − 0,5·cos … + 0,08·cos …	−58 dB	6 muestras	Máximo rango dinámico a costa de la resolución
Gaussiana	$\exp(-1/2 \cdot((n-\bar{n})/\sigma)^2)$	variable	σ fijo	Óptima para recuperación de senos puros

Nota importante: aplicar una ventana **reduce** la energía total y la concentra en el centro, por lo que cambia de forma sustancial los porcentajes por banda. No es un post-proceso neutro que pueda aplicarse o omitirse sin consecuencias sobre el resultado. Por eso su impacto se cuantifica en la sección 6.4.

2.5.9 El indicador de textura térmica

La hipótesis clinica que motiva el modulo de Fourier es la siguiente: un tejido con angiogenesis activa presenta una densidad anomala de vasos de pequeno calibre, es decir una **textura térmica más rica en componentes de media y alta frecuencia espacial** que el tejido homogeneo de referencia. Si se cuantifica la energía por bandas concéntricas de frecuencia, la proporción de energía en medias y altas frecuencias debe ser mayor en el tejido con patología. El algoritmo A9 implementa exactamente esa cuantificación; la sección 6 evalua criticamente hasta que punto la implementación actual mide lo que declara medir.

2.6 Interoperabilidad: el estándar DICOM

2.6.1 Que es DICOM

DICOM (*Digital Imaging and Communications in Medicine*) es el estándar internacional para el intercambio y el almacenamiento de información médica en imágenes. Es mantenido por la NEMA (*National Electrical Manufacturers Association*) a traves del comite DICOM y publicado como una familia de partes (PS3.1 a PS3.22). Un archivo DICOM no es solo una matriz de pixeles: es un conjunto de **atributos** (tags) con significado semántico definido por el estándar, de modo que cualquier visor autorizado sepa interpretarlos sin contexto adicional.

2.6.2 Anatomía de un archivo DICOM

Tabla 2.5. Componentes de un archivo DICOM relevantes para la salida Secondary Capture

Componente	Contenido	Nota de implementación
Preamble	128 bytes de relleno.	`b'\0'*128` — obligatorio, aunque la mayoría de los visores lo ignore.
Magic word	La cadena `DICM` en el offset 128.	Permite detectar si un archivo es DICOM sin necesidad de leer el grupo 0002.
File Meta Information (grupo 0002)	MediaStorageSOPClassUID, MediaStorageSOPInstanceUID, TransferSyntaxUID, ImplementationClassUID.	**Siempre** se codifica en Explicit VR Little Endian, con independencia de la sintaxis de transferencia del dataset principal.
Dataset	Los atributos propiamente dichos: datos del paciente, parámetros de imagen, PixelData.	Se codifica según el TransferSyntaxUID declarado en el grupo 0002.

2.6.3 Representación de valores: los VR

Cada atributo declara un **Value Representation** (VR) que determina como se codifican sus bytes. Este proyecto usa dos: `DS` (*Decimal String*, texto decimal de longitud variable) para los valores de calibración, y el contenedor de bytes directos para los pixeles. En **Explicit VR Little Endian** —la sintaxis de transferencia elegida— la cabecera de cada atributo lleva el nombre del VR en ASCII seguido de dos bytes de longitud.

Sintaxis de transferencia: Explicit VR Little Endian → UID 1.2.840.10008.1.2.1

Los bytes de más de un byte se ordenan little-endian, que es la convención de la arquitectura x86 sobre la que corre la aplicación. Por eso `img_celsius_scaled.tobytes()` produce directamente el orden correcto para enteros de 16 bits con signo.

2.6.4 Clase SOP: Secondary Capture

La aplicación no es un dispositivo de imagen médica: es una herramienta que **captura** una imagen ya existente. La clase SOP adecuada es **Secondary Capture Image Storage**:

UID 1.2.840.10008.5.1.4.1.1.7 (SC Image Storage)

Definida en PS3.4. Es la clase correcta para imágenes que no proceden de ninguna modalidad estándar de acquisition. La modalidad declarada es `Modality = "TG"` (Thermography, CID-29 de PS3.16), que identifica el contenido como termografía.

2.6.5 Calibración lineal del valor de pixel

PixelData en DICOM almacena enteros. Para transportar temperaturas con decimales, el estándar define que el valor real se reconstruya con una transformación lineal:

Valor real [unidad del objeto] = (Valor almacenado × RescaleSlope) + RescaleIntercept

RescaleSlope (0028,1053) y RescaleIntercept (0028,1052) son atributos tipo 3. Es el mecanismo que hace que un visor muestre 36,54 °C a partir del entero 3654.

Tabla 2.6. Rescalado a unidades fisiológicas: precisión alcanzable con enteros de 16 bits

RescaleSlope	RescaleIntercept	Precision alcanzada	Rango con int16
0,01	0,0	0,01 °C (1 décima)	−327,68 a +327,67 °C
1,0	−273,15	1 °C (insuficiente)	mismo rango, peor resolución
10	0	0,1 °C	mismo rango, peor resolución
0,001	0,0	0,001 °C (sobredimensionado)	mismo rango; riesgo de cuantizar a 0

La elección implementada, slope = 0,01, es la óptima para este caso: aprovecha los 15 bits significativos de un entero de 16 bits para entregar exactamente dos decimales, muy por encima de la resolución física de cualquier cámara clinica (NETD del orden de 0,05 °C), y con un margen de $4{,}7\times10^{4}$ en rango útil.

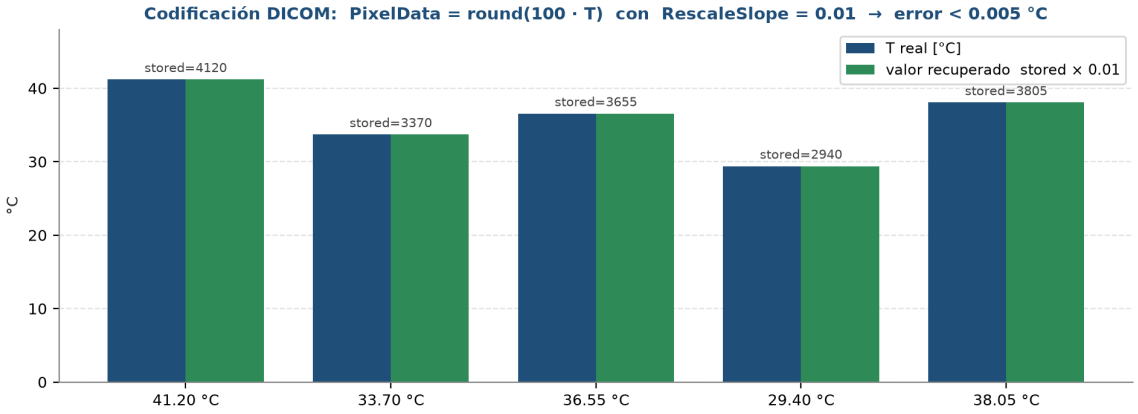


Figura 3. *Codificación y decodificación de la temperatura en PixelData. La barra verde es el valor que el visor médico reconstruye aplicando RescaleSlope = 0,01; el error máximo es de 0,005 °C, el del redondeo al entero más próximo.*

2.6.6 Ventana de contraste: Window Center y Window Width

Un visor médico no muestra los 16 bits crudos: aplica una transformación de contraste no lineal (Window/Level, o transformación P-sign) que mapea un intervalo de valores a la escala de grises de la pantalla. Sus dos parámetros son Window Center y Window Width, y la aplicación los inicializa de modo que el archivo se vea correctamente al abrirlo sin intervención del usuario:

$$\text{Window Width} = T_{max} - T_{min} \qquad \text{Window Center} = \bar{T}$$

Con la función de transformación del estándar DICOM: $y = ((x - (c - 0{,}5)) / (w - 1) + 0{,}5) \cdot (y_{max} - y_{min}) + y_{min}$, donde x es el valor almacenado ya calibrado, c el centro, w la anchura e y el valor mostrado. Elegir c = media y w = amplitud situa el histograma de la ROI exactamente en el rango visible, que es lo que se busca: que el médico vea el contraste térmico real de la region analizada y no un rango por defecto que aplaste la imagen.

2.6.7 Tipos de atributo y conformidad

El estándar clasifica cada atributo por obligatoriedad y completitud:

Tabla 2.7. Tipos de valor (VR) del estándar DICOM y su tratamiento en el código

Tipo	Significado	Ejemplos	Tratamiento en el código
1	Obligatorio; debe estar presente y con valor	Rows, Columns, PixelData, SOPClassUID, Modality	Presente. Todos los atributos tipo 1 propios de la clase SC se escriben.
1C	Obligatorio si se cumple una condición	BurnedInAnnotation (obligatorio en SC radiométrica de Utils)	**No se escribe** — ver capítulo 10 (líneas de trabajo futuro).
2	Obligatorio; puede estar vacio	PatientID, PatientBirthDate, StudyDate, ReferringPhysicianName	PatientID, PatientBirthDate y PatientSex se escriben; otros se dejan ausentes.
2C	Obligatorio si se cumple una condición	ContentDescription cuando hay ContentSequence	No aplica a esta clase SOP.
3	Opcional	RescaleSlope, WindowCenter, ImageComments	Presente: se usan para la calibración, el contraste y los resultados de Fourier.

3. Descripción del sistema

3.1 Arquitectura por capas

El proyecto adopta una arquitectura por capas con separación estricta de responsabilidades. La regla que se sigue es simple y verificable: **src/core/ no importa nada de src/ui/**. El nucleo de cálculo no sabe que existe una interfaz gráfica, y la interfaz no contiene fórmulas. Esto permite probar el nucleo de forma aislada, sustituir la interfaz sin tocar el cálculo, y localizar de forma univoca en el código cada fórmula descrita en el capítulo 4.

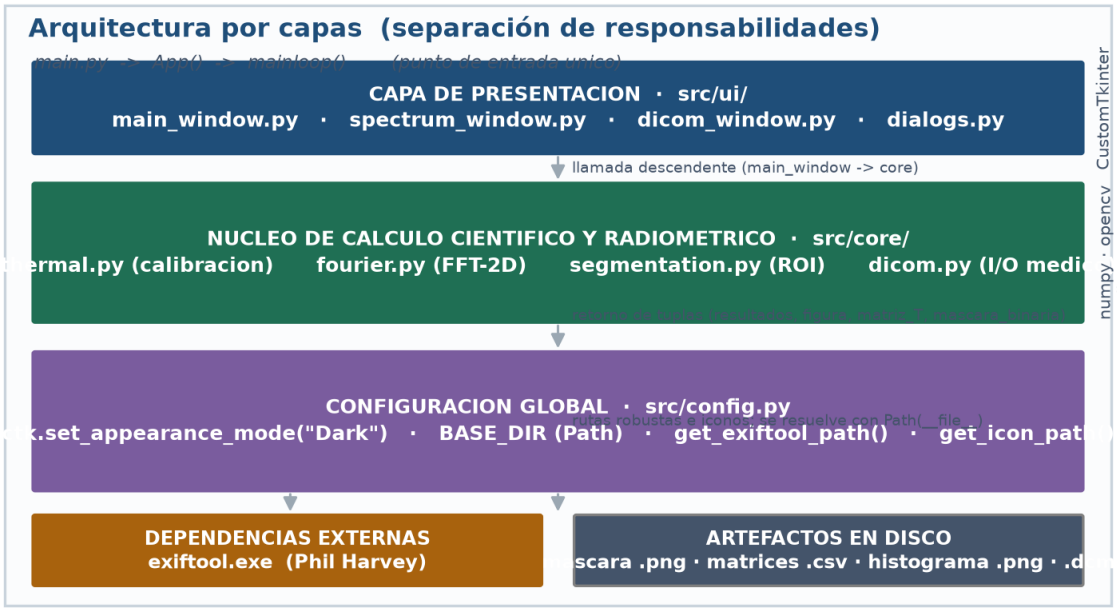


Figura 4. *Arquitectura en tres capas. Las flechas grises indican el sentido de las llamadas: descendente desde la interfaz hacia el nucleo, y ascendente en forma de tuplas de retorno. La capa de configuración se consume desde ambos lados.*

3.2 Estructura del proyecto

Tabla 3.1. Estructura de directorios del proyecto y responsabilidad de cada módulo

Ruta	Líneas	Responsabilidad única
`main.py`	32	Punto de entrada. Instancia la clase `App` y lanza `mainloop()`. No contiene lógica de negocio.
`requirements.txt`	13	Declaración de las seis dependencias con sus versiones mínimas.
`src/config.py`	101	Tema visual, altura de visualización, BASE_DIR y resolución de rutas de `exiftool.exe` y de los iconos.
`src/core/thermal.py`	262	A1 extracción con ExifTool; A2/A3 calibración radiométrica; A4 máscara; A7 estadísticos; A8 histograma.
`src/core/segmentation.py`	171	A5 segmentación interactiva con OpenCV; A6 segmentación por umbral.
`src/core/fourier.py`	178	A9 transformada de Fourier 2D y energía por anillos concéntricos.
`src/core/dicom.py`	154	A10 ensamblado y escritura del archivo DICOM.
`src/ui/main_window.py`	616	Ventana principal: flujo de tres pasos, formulario, descarga de CSV e incrustación del histograma en la interfaz.
`src/ui/spectrum_window.py`	294	Ventana de espectro: visualización del Fourier, informe interpretativo y paso a DICOM.
`src/ui/dicom_window.py`	152	Formulario de los 16 campos DICOM requeridos y disparo de la escritura.
`src/ui/dialogs.py`	182	Dialogos modales: instrucciones de dibujo y menu de opciones de segmentación.
`exiftool.exe`	—	Binario externo de lectura de metadatos. Se localiza de forma independiente del directorio de trabajo actual.

Totales: nucleo de cálculo 765 líneas, capa de presentación 1244 líneas. La asimetría a favor de la interfaz no indica que el nucleo sea trivial, sino que la lógica de cálculo esta concentrada en cuatro funciones que concentran la mayor parte de la complejidad del proyecto.

3.3 Modulos y sus contratos públicos

Tabla 3.2. Contrato público de las funciones de cada módulo

Función pública	Firma	Devuelve
`analizar_temperaturas`	(ruta_imagen, ruta_mascara, user_info, exiftool_path=None)	Tupla de 4 elementos: (resultados, figura, matriz_T completa, máscara binaria). El diccionario `resultados` contiene las claves `min_temp`, `max_temp`, `avg_temp` y `_metadata_raw`, o bien la clave `error`.
`segmentar_interactivo`	(ruta_imagen, parent=None)	Ruta del PNG de máscara generada, o None si el usuario cancelo.
`generar_mascara_por_umbral`	(ruta_imagen, umbral, user_info=None)	Ruta del PNG de máscara automática, o None.
`procesar_fourier_completo`	(matriz, r1=15, r2=50)	Tupla de 2: (imagen uint8 normalizada, diccionario de energías con `pct_baja`, `pct_media`, `pct_alta`, `radios_usados`, `interpretacion` y `energia_total`).
`generar_archivo_dicom`	(ruta_salida, matriz, metadatos_flir, campos_paciente, resultados_anillos=None)	None. Escribe el archivo en disco; los errores se propagan como excepción.
`get_exiftool_path` / `get_icon_path`	()	Ruta absoluta del recurso, o None si no se encuentra. Degradación elegante: la aplicación sigue funcionando.

Nota sobre el contrato de errores

El nucleo **no propaga excepciones hacia la interfaz**: devuelve un diccionario con la clave `error` y valores nulos, y es la interfaz la que decide como mostrarlo (etiqueta en rojo o mensaje emergente). Esto evita que un fallo de un componente de terceros —ExifTool ausente, modelo de cámara no soportado, ROI vacía— tumbe la aplicación. La única excepción es `generar_archivo_dicom`, que si propaga, porque su único llamador esta dentro de un bloque try/except propio.

3.4 Flujo de trabajo del usuario

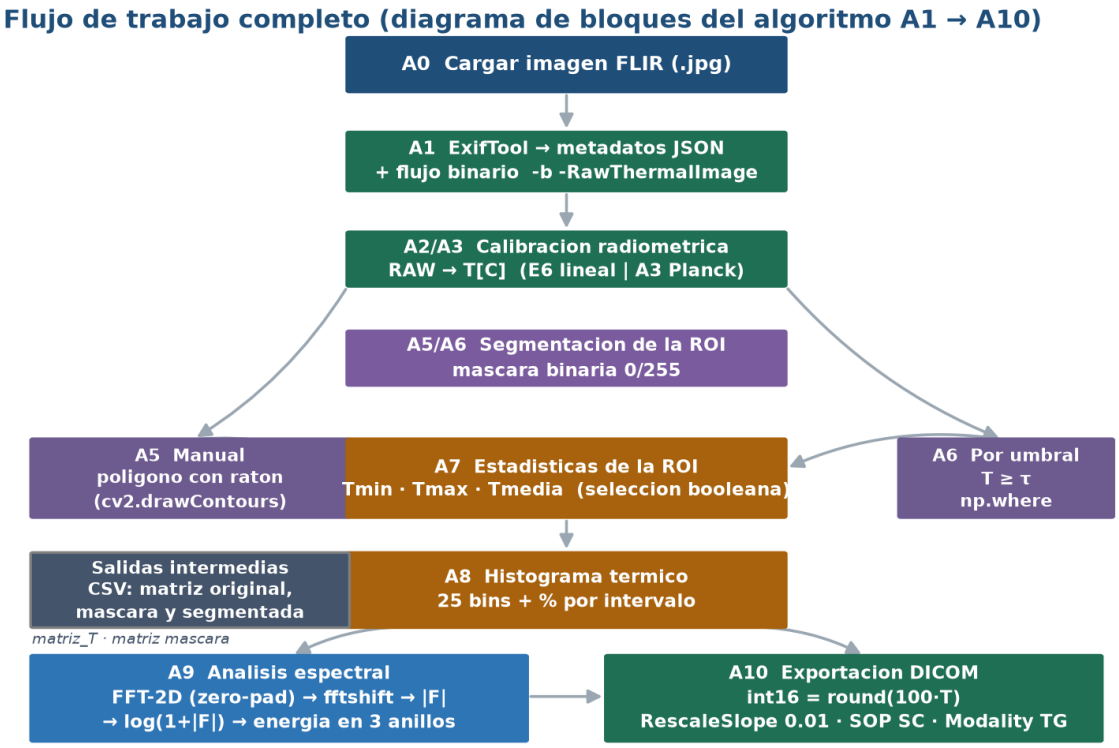


Figura 5. *Diagrama de bloques del proceso completo. Los bloques verdes son cálculos de calibración, los morados segmentación, los naranjas estadística, y los azules y verdes de la base las dos salidas principales: espectro y DICOM.*

- **Paso 0 — Bienvenida.** Una pantalla de inicio mínima; el boton de inicio construye el panel de trabajo y maximiza la ventana.
- **Paso 1 — Cargar imagen.** Se selecciona un `.jpg` FLIR. Se habilita el paso 2 y se deshabilita el 3. La imagen se muestra reescalada a 300 px de altura conservando la proporción.
- **Paso 2 — Segmentar.** Un dialogo modal ofrece las tres vías. En la vía manual la ventana principal se oculta con `withdraw()` para que OpenCV quede sin distracciones, y se restaura con `deiconify()` al terminar.
- **Paso 3 — Analizar.** Se ejecuta el motor térmico, se muestran T_{min}, T_{max} y $\bar{T}$, y se incrusta el histograma en la ventana mediante `FigureCanvasTkAgg`. Se habilitan las cinco descargas y el boton de espectro.
- **Paso 4 — Espectro (opcional).** La ventana de Fourier recibe las matrices ya en memoria, multiplica por la máscara binaria y ejecuta el algoritmo A9 de forma automática.
- **Paso 5 — Exportar DICOM.** Se abre el formulario de 16 campos y se escribe el archivo.

3.5 Modelo de datos en memoria

El estado de la sesion vive en atributos de la ventana principal, no en una base de datos. Es una decision de simplicidad adecuada para un uso de un paciente a la vez, y explica por que la herramienta no incluye historial ni comparación automática entre estudios:

Tabla 3.3. Atributos del modelo de datos en memoria

Atributo	Tipo	Descripción
`ruta_imagen_original`	str	Ruta del JPEG original.
`ruta_mascara`	str	Ruta del PNG de máscara.
`matriz_temp_original`	ndarray float32 (H×W)	Mapa completo de temperaturas en grados Celsius.
`matriz_binaria_segmentada`	ndarray int (H×W)	Máscara en 0 y 1, con las mismas dimensiones que la matriz térmica.
`metadatos_flir`	dict	Diccionario completo de metadatos tal como lo devuelve ExifTool, reenviado al DICOM.
`fig_matplotlib`	matplotlib Figure	Figura del histograma, reutilizable para exportar con savefig a 300 dpi.
`ventana_espectro_actual`	CTkToplevel o None	Referencia a la ventana de Fourier, para no duplicarla.

3.6 Validación y manejo de errores

Tabla 3.4. Detección de situaciones de error y respuesta del sistema

Situación	Detección en el código	Respuesta del sistema
ExifTool no esta o falla al arrancar	`subprocess.run([...,"-ver"], check=True)`	Se capturan `FileNotFoundError` y `CalledProcessError`; se devuelve el error «ExifTool no encontrado».
El flujo binario de la imagen térmica llega vacio	`if not resultado_imagen.stdout`	Retorno temprano con un error explicito.
Faltan constantes de calibración	Comprobación `any(v is None ...)` sobre los 4 o 5 campos	Retorno temprano: «Metadatos de calibración lineal no encontrados» o «Metadatos de Planck incompletos».
Modelo de cámara no soportado	`else` tras las ramas E6 y A3	Retorno temprano, indicando el nombre del modelo detectado.
Argumento del logaritmo no positivo	`if np.min(termino_log) <= 0`	Retorno temprano: evita propagar NaN a todo el análisis.
Degeneración del rango crudo (`raw_max == raw_min`)	Comprobación explicita en la rama lineal	Se devuelve un mapa constante igual a T_{min}, en lugar de dividir entre cero.
La ROI esta vacía	`if temperaturas_segmentadas_array.size == 0`	Retorno temprano: «El área de la máscara esta vacía».
La matriz de Fourier es toda ceros	`if total_energia == 0`	Devuelve los tres porcentajes a 0.0 sin dividir entre cero.
Las matrices de dos CSV no coinciden de tamano	`if matriz_temp.shape != matriz_mascara.shape`	Mensaje emergente indicando ambas dimensiones.
La máscara no se pudo cargar	`cv2.imread(...) is None`	Retorno temprano. En la carga manual también se valida en la interfaz.
La máscara tiene otra resolución que la imagen térmica	`cv2.resize(..., INTER_NEAREST)`	Reescalado con vecino más próximo, que preserva el caracter binario de la máscara.

4. Implementación de los algoritmos

Este capítulo es el nucleo técnico del documento. Recorre, en el mismo orden en que el programa los ejecuta, cada uno de los cinco algoritmos implementados: extracción del dato radiométrico, calibración a grados Celsius, segmentación de la region de interes, análisis espectral de Fourier 2D y codificación DICOM. Para cada uno se

enuncia el problema, se desarrolla la fórmula, se muestra el fragmento de código tal como esta en el repositorio y se comentan las decisiones de diseno y las trampas del modelo numérico.

> **Convenciones de lectura**
>
> Los fragmentos de código son **literales**: se han copiado del repositorio en el commit 1558801 y solo se han recortado las partes irrelevantes, marcadas con `[...]`. Los numeros de línea se refieren al archivo original. El índice `A1` a `A7` que aparece en algunos=listados remite a los algoritmos del anexo A, donde se pseudocodifica cada uno de forma independiente del lenguaje.

4.1 Extracción del dato radiométrico

4.1.1 Planteamiento

Una imagen de una cámara térmica FLIR es un archivo JPEG ordinario, y por si sola no contiene ninguna temperatura. Lo que el archivo transporta son dos cosas distintas: una **imagen visible** pintada con una paleta de falsa color, y un bloque binario aparte, la etiqueta `RawThermalImage`, que guarda los valores enteros medidos por el microbolometro. A esos enteros se les llama *valores crudos* o `RAW`, y no tienen unidades térmicas: son índices enteros cuyo significado solo puede establecerse con las constantes de calibración que la fabrica guardo en el `MakerNote`.

Además, la respuesta radiométrica completa de la cámara depende de las condiciones de la toma: emisividad de la superficie, distancia al objetivo, temperatura reflejada y temperatura ambiente. Sin esos parámetros, un valor RAW no puede convertirse en kelvinios. Por eso el primer paso no es calcular nada, sino **extraer el mayor numero posible de metadatos de forma fiable**.

4.1.2 Resolución de la ruta de ExifTool

ExifTool no es una librería de Python sino un ejecutable escrito en Perl. La aplicación lo invoca como un proceso hijo y recibe los resultados por su salida estándar. El primer problema práctico es *encontrarlo*: si la ruta depende del directorio de trabajo, la aplicación falla en cuanto el usuario la lanza con un acceso directo en lugar de desde la terminal. `src/config.py` resuelve el caso con una cascada de busqueda ordenada de más específico a más general.

`src/config.py · get_exiftool_path() · lineas 39-75`

```
def get_exiftool_path() -> str:
# 1) Si el proyecto se empaqueta con PyInstaller, los datos van en _MEIPASS
if getattr(sys, 'frozen', False) and hasattr(sys, '_MEIPASS'):
bundle_path = os.path.join(sys._MEIPASS, 'exiftool.exe')
if os.path.exists(bundle_path):
return bundle_path

# 2) Carpeta raiz del proyecto (BASE_DIR se deriva de __file__, no del cwd)
path_in_base = BASE_DIR / 'exiftool.exe'
if path_in_base.exists():
return str(path_in_base)

# 3) Subcarpetas de herramientas
for sub in ['bin', 'assets/bin', 'tools']:
```

```
candidate = BASE_DIR / sub / 'exiftool.exe'
if candidate.exists():
return str(candidate)

# 4) Directorio actual, y 5) como ultimo recurso, el PATH de Windows
if os.path.exists('exiftool.exe'):
return os.path.abspath('exiftool.exe')
return 'exiftool.exe'
```

El detalle relevante para la robustez esta en la definición de `BASE_DIR`:

src/config.py · linea 23

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

Al derivarla de `__file__` y no de `os.getcwd()`, la ruta es correcta con independencia del directorio desde el que se lance el programa. Es una decision de una línea que evita la mayoría de los errores de «archivo no encontrado», como se analiza en el algoritmo A1 del anexo A.

4.1.3 Dos invocaciones distintas

La función `analizar_temperaturas()` invoca ExifTool **tres veces** y cada vez con un proposito distinto. La distinción no es académica: la primera y la tercera devuelven texto, la segunda devuelve bytes binarios.

Tabla 4.1. Invocaciones de la extracción del dato radiométrico y su retorno

Invocación	Que devuelve	Para que se usa
`-ver`	Numero de versión en texto	Comprobar que el ejecutable existe y responde antes de intentar nada más
`-json` *fichero*	Todos los metadatos en JSON	Leer modelo de cámara, constantes de calibración, emisividad, distancia
`-b -RawThermalImage` *fichero*	Flujo binario crudo, sin codificación de texto	Obtener la matriz de enteros que entrega el sensor

El detalle crítico de la opción -b

Este es el punto que más errores causa en la práctica. Sin la opción `-b`, ExifTool imprime el contenido del bloque binario codificado como texto para que sea legible. Esa codificación es **destructiva**: el flujo original de bytes se convierte en caracteres imprimibles y ya no se puede recuperar. La opción `-b` (*binary*) instruye a ExifTool que no toque un solo bit.

src/core/thermal.py · lineas 75-94

```
# Metadatos en JSON: salida de texto
comando_metadata = [exiftool_path, '-json', ruta_imagen_original]
resultado_metadata = subprocess.run(
comando_metadata, capture_output=True, text=True,
encoding='utf-8', creationflags=CREATE_NO_WINDOW,
)
metadata = json.loads(resultado_metadata.stdout)[0]

# Matriz termica cruda: salida binaria pura, sin decodificar
```

```
comando_imagen = [exiftool_path, '-b', '-RawThermalImage', ruta_imagen_original]
resultado_imagen = subprocess.run(
comando_imagen, capture_output=True, creationflags=CREATE_NO_WINDOW,
)
```

> **Por que la lista tiene un solo elemento**
>
> ExifTool devuelve un **arreglo JSON** aunque se le pase un solo archivo, porque su interfaz esta disenada para procesar lotes. El índice `[0]` selecciona el primer (y único) objeto. Omitirlo provocaría un error de tipo al intentar acceder a las claves del diccionario.

Un tercer detalle, pequeno pero muy visible en Windows, es la bandera `CREATE_NO_WINDOW`:

src/core/thermal.py · linea 34

```
CREATE_NO_WINDOW = getattr(subprocess, 'CREATE_NO_WINDOW', 0)
```

Sin ella, cada llamada a ExifTool abre y cierra una ventana negra de consola sobre la ventana principal de la aplicación. El `getattr` con valor por defecto `0` mantiene el código funcional en sistemas distintos de Windows, donde esa constante no existe.

4.1.4 De los bytes a la matriz numérica

Los bytes devueltos no son una imagen en un formato conocido: son el bloque `RawThermalImage` tal cual, que internamente es un PNG en escala de grises de 16 bits. Pillow se encarga de descodificarlo y NumPy de convertirlo en una matriz de enteros.

src/core/thermal.py · lineas 99-101

```
raw_image_data = io.BytesIO(resultado_imagen.stdout)
raw_image = Image.open(raw_image_data)
matriz_temperaturas_raw = np.array(raw_image)
```

`io.BytesIO` construye un objeto de archivo en memoria a partir del flujo de bytes, de modo que no se escribe nada en disco. Esta cadena *bytes → flujo en memoria → imagen → matriz* es la que convierte un formato propietario en algo con lo que se puede calcular.

4.2 Calibración radiométrica: del valor crudo a grados Celsius

Este es el corazon físico del proyecto y la parte donde un error no se ve en la imagen pero corrompe todos los resultados posteriores. Existen dos modelos, seleccionados automáticamente según el modelo de cámara declarado en los metadatos.

4.2.1 Selección del modelo

La decision se toma con una simple comprobación de subcadena sobre el texto del modelo:

src/core/thermal.py · lineas 107-158

```
model_name = metadata.get('CameraModel', 'Desconocido')

if 'E6' in model_name:
[...] # CASO A: calibracion lineal
elif 'A3' in model_name:
```

```
[...] # CASO B: ecuacion de Planck

else:

return {'error': f"Modelo de camara no reconocido: '{model_name}'."}, None, None, None
```

La comprobación por subcadena en lugar de igualdad exacta es deliberada: los CameraModel reales traen sufijos y numeraciones de versión (`FLIR E6-XT`, `FLIR A3`), de modo que la igualdad fallaría. A cambio, es una heurística: un modelo desconocido produce un error explicito en vez de un resultado silenciosamente incorrecto, que es el comportamiento correcto para una herramienta de medida.

4.2.2 Caso A: calibración lineal (FLIR E6)

En las cámaras de la serie E la relación entre el valor crudo y la temperatura absoluta es, en el rango de trabajo, una función afin. La fabrica pública los dos puntos extremos que la definen y la aplicación los convierte en pendiente y ordenada.

$$T_K = m \cdot RAW + c$$

Recta de calibración. m es la pendiente en kelvinios por unidad cruda; c es la ordenada en el origen.

$$m = (T_{max} - T_{min}) / (RAW_{max} - RAW_{min}) \qquad c = T_{min} - m \cdot RAW_{min}$$

Determinación de m y c a partir de los dos puntos de calibración publicados por la fabrica (etiquetas ImageTemperatureMin/Max y RawValueRangeMin/Max).

src/core/thermal.py · lineas 114-130

```
temp_min_k = metadata.get('ImageTemperatureMin')

temp_max_k = metadata.get('ImageTemperatureMax')

raw_min = metadata.get('RawValueRangeMin')

raw_max = metadata.get('RawValueRangeMax')


if any(v is None for v in [temp_min_k, temp_max_k, raw_min, raw_max]):

return {'error': 'Metadatos de calibracion lineal no encontrados.'}, None, None, None


if raw_max == raw_min:

# Caso degenerado: el rango crudo es un punto, la calibracion no es invertible

temp_kelvin = np.full(matriz_temperaturas_raw.shape, temp_min_k, dtype=np.float32)

else:

m = (temp_max_k - temp_min_k) / (raw_max - raw_min)

c = temp_min_k - m * raw_min

temp_kelvin = m * matriz_temperaturas_raw.astype(np.float32) + c


temperaturas_celsius_original = temp_kelvin - 273.15
```

Por que se comprueba que los metadatos existen antes de dividir

Las tres guardas del bloque anterior responden a tres fallos reales y distintos. **La primera** evita una excepción de tipo: si falta cualquiera de los cuatro valores, la division daría `None` en un denominador. **La segunda** evita una division por cero cuando $RAW_{max} = RAW_{min}$, que ocurre si la cámara no ha calibrado el sensor: en ese caso la función asigna a toda la imagen la temperatura mínima declarada, que es la única aproximación defendible, y el

resto de la cadena sigue funcionando. **La tercera**, más adelante, es puramente numérica.

> **Un supuesto que el código hace y que conviene verificar**
>
> Los nombres de las etiquetas `ImageTemperatureMin` y `ImageTemperatureMax` no incluyen la unidad. La documentación de ExifTool advierte de que el valor puede venir en grados Celsius, en kelvinios o en grados Fahrenheit, y de que el archivo no lo indica de forma inequivoca. El código las interpreta como kelvinios y resta 273,15. Si la cámara estuviera configurada en grados Celsius, las temperaturas resultantes serian unas 273 grados erroneas. Es un supuesto que debe validarse experimentalmente con un termometro de referencia; se recoge como limitación en el capítulo 9.

El cambio a punto flotante ocurre **antes** de multiplicar, no después:

src/core/thermal.py · linea 127

```
matriz_temperaturas_raw.astype(np.float32)
```

Es una decision de menor riesgo pero de aparienciaplice. Si se multiplicara primero la matriz entera por `m` y se dividiera después, se introduccionaría un error de redondeo en cada pixel, del orden de ε_{int}. Convertir antes garantiza que la única operación con redondeo sea la final, sobre el numero ya

4.3 Caso B: ecuación de Planck (FLIR A3)

Las cámaras de la serie A no usan una recta, sino el modelo radiométrico completo: la ley de radiación de Planck resuelta para la temperatura, con cinco constantes de calibración proporcionadas por la fabrica en el `MakerNote`.

$$T = B / \ln(R_1 / (R_2 \cdot (RAW + O)) + F)$$

Modelo radiométrico resuelto para T. R1, R2 son constantes de Planck ajustadas por la fabrica; O es la compensación del offset del detector; F el factor de corrección; B el factor de escala.

4.3.1 Estructura del cálculo y su justificación numérica

La implementación descompone la expresion en pasos intermedios en lugar de escribirla en una sola línea, y esa descomposición no es estética: permite validar el dominio antes de aplicar el logaritmo.

src/core/thermal.py · lineas 137-155

```
R1 = metadata.get('PlanckR1')
B = metadata.get('PlanckB')
F = metadata.get('PlanckF')
O = metadata.get('PlanckO')
R2 = metadata.get('PlanckR2')


if any(v is None for v in [R1, B, F, O, R2]):
return {'error': 'Metadatos de Planck incompletos.'}, None, None, None


raw_data_float = matriz_temperaturas_raw.astype(np.float32)
term1 = R1 / (R2 * (raw_data_float + O))
termino_log = term1 + F


# El logaritmo de numeros negativos o cero no existe en R
```

```
if np.min(termino_log) <= 0:
return {'error': 'Error matematico: logaritmo de numero no positivo.'}, None, None, None


temp_kelvin = B / np.log(termino_log)
temperaturas_celsius_original = temp_kelvin - 273.15
```

Tabla 4.2. Detección de líneas espectrales: regla de decisión y justificación

Línea	Decision	Justificación
`astype(np.float32)`	Conversion previa a coma flotante	RAW + O puede ser negativo o fraccionario; en aritmética entera se truncaría antes de dividir y el resultado sería erroneo
`np.min(termino_log) <= 0`	Comprobación global del argumento del logaritmo	Un solo pixel con argumento no positivo envenena el resultado de toda la matriz y arrastra `NaN` a todos los estadísticos
Comprobación de los cinco metadatos	Fallo temprano con mensaje claro	Un MakerNote incompleto daría un `None` en la aritmética, no un diagnóstico útil

La elección de `float32` y no `float64` es también deliberada. Una matriz térmica típica de 640 × 480 elementos en `float64` ocupa 2,4 MB, y la transformada de Fourier duplica esa magnitud. El margen de error adicional de `float32` es de unos 10^{-7} kelvinios, es decir 7 ordenes de magnitud por debajo de la resolución de la cámara, que es de centésimas de grado. La precision adicional no aportaría nada medible a cambio del doble de memoria.

4.4 Aplicación de la region de interes

Con la matriz de temperaturas ya en grados Celsius, el siguiente paso es aislar la zona que interesa. La máscara se maneja como una imagen de un solo canal con valores 0 y 255.

4.4.1 Ajuste dimensional y umbral de binarización

`src/core/thermal.py · lineas 164-178`

```
mascara_img = cv2.imread(ruta_mascara, cv2.IMREAD_GRAYSCALE)
if mascara_img is None:
return {'error': 'No se pudo cargar la mascara.'}, None, None, None


h_temp, w_temp = temperaturas_celsius_original.shape
mascara_img = cv2.resize(mascara_img, (w_temp, h_temp), interpolation=cv2.INTER_NEAREST)


# Indice booleano: selecciona solo los pixeles donde la mascara es blanca
temperaturas_segmentadas_array = temperaturas_celsius_original[mascara_img > 128]
if temperaturas_segmentadas_array.size == 0:
return {'error': 'El area de la mascara esta vacia.'}, None, None, None


matriz_segmentada_binaria = np.where(mascara_img > 128, 1, 0)
```

Por que `INTER_NEAREST` y no interpolación suave

> Redimensionar la máscara con `INTER_LINEAR` o `INTER_CUBIC` generaría valores intermedios entre 0 y 255. El umbral de 128 los convertiría en pixeles aproximadamente aleatorios en el borde, produciendo un contorno dentado e irregular. Con `INTER_NEAREST` cada pixel toma el valor exacto de su vecino más próximo y la frontera queda nitida, que es lo correcto para un filtro espacial binario. Esta es la regla general: **nunca interpolar un dominio de decision**, solo el dato que se quiere medir.

El patron de **índice booleano** es la forma canónica de OpenCV y NumPy para recortar una matriz multidimensional. La expresion `matriz[cond]` donde `cond` es un array de `True/False` devuelve un array **unidimensional** con los elementos seleccionados, ya no con forma de imagen. Eso es exactamente lo que se necesita para calcular estadísticos, y explica por que el código guarda además una matriz binaria aparte: la forma de imagen hace falta para el análisis espacial.

4.4.2 Estadísticos descriptivos

`src/core/thermal.py · lineas 184-189`

```
resultados = {
'min_temp': f'{np.nanmin(temperaturas_segmentadas_array):.2f}°C',
'max_temp': f'{np.nanmax(temperaturas_segmentadas_array):.2f}°C',
'avg_temp': f'{np.nanmean(temperaturas_segmentadas_array):.2f}°C',
'_metadata_raw': metadata,
}
```

El uso de `nanmin`, `nanmax` y `nanmean` en lugar de `min`, `max` y `mean` es una decision de robustez con consecuencias reales. Los pixeles no validos del sensor o los valores indefinidos que puede producir la inversion de Planck se representan como `NaN` y envenenan cualquier reducción. Las funciones *nan* los ignoran.

Conviene senalar su limite: *nanmean* no es igual a la media calculada sobre los pixeles validos, porque el denominador sigue siendo el numero total de pixeles de la ROI, no el de los no nulos. Es una aproximación razonable cuando los valores invalidos son residuales, y una fuente de sesgo a la baja si no lo son. Se recoge en el capítulo 9.

El formato `:.2f` más el sufijo `°C` se aplica ya aquí, de modo que la capa de presentación recibe cadenas y no tiene que respetar el formato. Es una decision de separación de responsabilidades menor, pero que evita que tres ventanas distintas applied formato distinto al mismo numero.

4.4.3 Histograma de frecuencias térmicas

El histograma se construye con `matplotlib` y devuelve un objeto `Figure` que la interfaz inserta directamente en un widget, sin exportar a disco.

`src/core/thermal.py · lineas 198-250 (extracto)`

```
datos_validos = temperaturas_segmentadas_array[~np.isnan(temperaturas_segmentadas_array)]


counts, bins, patches = ax.hist(datos_validos, bins=25, color='deepskyblue',
edgecolor='black', alpha=0.9)
ax.set_xlabel('Temperatura (°C)', fontsize=12)
ax.set_ylabel('Frecuencia (pixeles)', fontsize=12)
ax.grid(True, linestyle='--', alpha=0.4)
ax.spines['top'].set_visible(False)
```

```
y_max = ax.get_ylim()[1]
offset = y_max * 0.01
for i in range(len(counts)):
count = counts[i]
if count > 0:
porcentaje = (count / total_pixeles_segmentados) * 100
etiqueta = f'{int(count)}px ({porcentaje:.1f}%) @ {bins[i]:.1f}°C'
ax.text(patches[i].get_x() + patches[i].get_width()/2, count + offset,
etiqueta, ha='center', va='bottom', fontsize=8, rotation=90)
```

Tres detalles del gráfico merecen comentario. El primero es que `ax.hist` devuelve tres valores: las alturas `counts`, los bordes de los intervalos `bins` y los rectangulos dibujados `patches`. Los dos primeros se usan para el cálculo y el tercero para colocar las etiquetas. El segundo es el numero de intervalos: `bins=25` fija 25 barras, con lo que la anchura de cada intervalo es aproximadamente el rango total dividido por 25, sin que el usuario pueda ajustarlo.

> **Una inconsistencia real en el cálculo de porcentajes**
>
> El numerador `count` procede de `datos_validos`, que ya ha descartado los `NaN`, pero el denominador es `total_pixeles_segmentados`, que se calcula antes y si los incluye. Si la imagen tiene pixeles nulos, la suma de los porcentajes mostrados sera ligeramente inferior a 100. Con el caso habitual de termografía de mama, donde la ROI esta llena de tejido y no hay pixulos nulos, el efecto es despreciable; se documenta como corrección pendiente en el capítulo 9.

4.5 Segmentación de la region de interes

La segmentación define *que* se mide. Es la decision que más condiciona el resultado, porque todo lo que se calcula después hereda sus limites.

4.5.1 Vía manual: dibujo interactivo del contorno

OpenCV puede capturar los eventos del raton sobre una ventana. El proyecto aprovecha esa capacidad mediante una función de *callback* que OpenCV invoca automáticamente en cada movimiento, pulsación o liberación del boton.

src/core/segmentation.py · lineas 26-59

```
_points_lazo = [] # estado del lazo en curso
_drawing_lazo = False # True mientras el boton esta pulsado


def _mouse_callback_lazo(event, x, y, flags, param):
global _drawing_lazo, _points_lazo
img_display = param['image_display']
img_original_copy = param['image_original_copy']


# 1) Restaurar la imagen limpia antes de redibujar
img_display[:] = img_original_copy[:]
```

```
if event == cv2.EVENT_LBUTTONDOWN:
_drawing_lazo = True
_points_lazo = [(x, y)]

elif event == cv2.EVENT_MOUSEMOVE and _drawing_lazo:
_points_lazo.append((x, y))
for i in range(len(_points_lazo) - 1):
cv2.line(img_display, _points_lazo[i], _points_lazo[i+1], (0,255,0), 2)

elif event == cv2.EVENT_LBUTTONUP:
_drawing_lazo = False
if len(_points_lazo) > 2:
# Cerrar el contorno uniendo el ultimo punto con el primero
cv2.line(img_display, _points_lazo[-1], _points_lazo[0], (0,255,0), 2)
```

El patron de redibujado incremental

La primera instrucción de la función, `img_display[:] = img_original_copy[:]`, es la clave del diseno. Si en su lugar se dibujara cada segmento nuevo sobre lo que ya había, la poligonal anterior se acumularía de forma permanente y la línea se engrosaría a medida que el usuario se moviera, formando una mancha. Restaurar la imagen limpia en cada evento y redibujar la poligonal completa desde el punto 1 da un trazo de grosor constante con coste lineal en el numero de puntos y sin artefactos.

El uso de `global` para las dos variables de estado es una consecuencia de la firma impuesta por OpenCV: el *callback* recibe `(event, x, y, flags, param)` y no admite parámetros adicionales del usuario. La aplicación resuelve el problema equivalente pasando por `param` las dos imágenes que el callback necesita, lo que evita al menos el uso de *global* para las referencias pesadas a las matrices.

src/core/segmentation.py · lineas 90-108

```
cv2.setMouseCallback(window_name, _mouse_callback_lazo,
{'image_display': img_display,
'image_original_copy': img_original_copy})

while True:
cv2.imshow(window_name, img_display)
key = cv2.waitKey(1) & 0xFF
if key == 27: # ESC: cancelar
cv2.destroyAllWindows()
return None
if key == 13 and len(_points_lazo) > 2: # ENTER: confirmar
break
```

El bucle se ejecuta a `cv2.waitKey(1)`, un milisegundo por iteración. Es un *busy loop*: consume CPU de forma continua mientras la ventana esta abierta. La alternativa correcta sería un modelo de eventos con `cv2.waitKeyEx` o un hilo separado, que es la razon por la que la segmentación manual bloquea la interfaz durante el dibujado. Se anota como limitación de usabilidad en el capítulo 9.

De la poligonal a la máscara binaria

src/core/segmentation.py · lineas 117-127

```
mask = np.zeros(img_original.shape[:2], dtype=np.uint8)
contour = np.array(_points_lazo, dtype=np.int32)
cv2.drawContours(mask, [contour], -1, 255, thickness=cv2.FILLED)


directorio, nombre_archivo = os.path.split(ruta_imagen)
nombre_base, _ = os.path.splitext(nombre_archivo)
ruta_mascara_salida = os.path.join(directorio, f'{nombre_base}_mask.png')
cv2.imwrite(ruta_mascara_salida, mask)
```

Tabla 4.3. Argumentos de la segmentación de la región de interés

Argumento	Valor	Por que
`img_original.shape[:2]`	Alto y ancho de la imagen	La tupla de forma de una imagen a color es (alto, ancho, canales); el corte `[:2]` la reduce a la geometría
`np.zeros(...)`	Lienzo inicializado a negro	Una matriz vacía en NumPy no se inicializa a cero de forma fiable en versiones antiguas; `np.zeros` lo garantiza
`thickness=cv2.FILLED`	Relleno del interior	Con el valor por defecto el contorno sería solo una línea de un pixel, sin área
`-1`	Todos los contornos de la lista	Convención de OpenCV para «todos» en funciones de dibujo
`dtype=np.int32`	Enteros de 32 bits	`drawContours` exige enteros; `uint8` se desbordaría con coordenadas altas

La máscara se guarda como PNG, no como JPEG. Es una decision elemental con consecuencias serias: PNG es un formato sin perdida, mientras que JPEG es un formato con compresion con perdidas que introducira artefactos de compresion justo en el contorno de la ROI, y por tanto en la frontera que separa el tejido seleccionado del fondo. Al ser esta frontera la que domina el espectro de Fourier, un contorno contaminado por bloques de 8 × 8 pixeles falsearía la medición de altas frecuencias. El nombre del archivo incluye la base del original más `_mask.png`, de modo que la correspondencia entre imagen y máscara queda registrada en el propio sistema de archivos.

4.5.2 Vía automática: umbralización térmica

Cuando la lesion tiene un contraste térmico claro respecto al tejido circundante, la ROI puede derivarse del propio dato térmico sin intervención del usuario.

$$M(x, y) = 255 \text{ si } T(x, y) \geq \tau \quad 0 \text{ en caso contrario}$$

Máscara binaria M a partir de la temperatura T y del umbral tau introducido por el usuario. Equivale exactamente a la umbralización por media multiplicada por dos de la sección 2.3.2.

src/core/segmentation.py · lineas 146-165

```
resultados, _, temp_matrix, _ = analizar_temperaturas(
ruta_imagen_original, ruta_imagen_original, user_info=user_info_temp
)


if temp_matrix is None:
```

```
    messagebox.showerror('Error de Analisis',
    f'No se pudo extraer la matriz de temperatura. Error: {resultados.get("error")}')
    return None

    # np.where es un 'if' vectorizado sobre toda la matriz
    mascara_automatica = np.where(temp_matrix >= umbral, 255, 0).astype(np.uint8)

    ruta_mascara_salida = os.path.join(directorio, f'{nombre_base}_mask_auto_{umbral}C.png')
    cv2.imwrite(ruta_mascara_salida, mascara_automatica)
```

`np.where` merece un comentario propio, porque es de las construcciones más habituales de NumPy. En C habría que escribir un bucle doble:

```
Equivalente en C o en Python puro: 2 bucles, 1 asignación por pixel
for i in range(filas):
    for j in range(columnas):
        mascara[i, j] = 255 if T[i, j] >= umbral else 0
```

`np.where` hace exactamente eso, pero en una sola operación vectorizada ejecutada en C, sin bucles de Python. Para una imagen de 640 × 480 son 307 200 pixeles; el bucle en Python tardaría del orden de un segundo, mientras que la forma vectorizada tarda microsegundos. Este patron se repite en todo el proyecto y es una de las razones por las que el nucleo numérico esta escrito con NumPy y no con Python puro.

> **Una particularidad de esta función que conviene entender**
>
> La llamada a `analizar_temperaturas()` recibe `ruta_imagen_original` **como argumento de la máscara**. Es decir, la propia imagen térmica se usa como si fuera una máscara. El resultado es que la función calcula también un conjunto de estadísticos sobre lo que en ese punto es «pixeles claros de la imagen», es decir, casi nada: irrelevante, porque la función descarta ese valor con _ y solo se queda con `temp_matrix`, la matriz completa ya calibrada, que es lo único que necesita para aplicar el umbral. No afecta al resultado final, pero obliga a un proceso de calibración completo que se repite: la matriz se calcula dos veces (una para generar la máscara automática y otra al analizarla). Se documenta como refactorización pendiente en el capítulo 9.

Un detalle de la interfaz de la umbrazación que conviene no pasar por alto: el archivo se nombra con el umbral interpolationado en el nombre, por ejemplo `estudio_mask_auto_36.5C.png`. Esto es una buena práctica de trazabilidad: la máscara queda asociada al parámetro con el que se genero, de modo que repetir el análisis es reproducible aunque no se recuerde el valor.

4.6 Análisis espectral de Fourier 2D

El algoritmo más complejo del proyecto y también el más identifica errores. Se ejecuta sobre la matriz de temperatura de la ROI y devuelve una imagen de espectro y un reparto porcentual de la energía en tres bandas.

4.6.1 Encuadre

La transformada de Fourier bidimensional convierte la imagen en el dominio espacial (coordenadas *x*, *y*) al dominio de la frecuencia espacial (*u*, *v*), que mide cuantas variaciones hay y con que rapidez en cada dirección. En el espectro resultante la frecuencia cero (componente continua, o *DC*) queda en una esquina, y las frecuencias altas se Situan lejos. Aplicando `fftshift` esa esquina se lleva al centro, de modo que la disposición espacial del espectro resulta intuitivamente analoga a la de la imagen original.

4.6.2 Zero-padding a tamano óptimo

El primer paso no es matematico sino de rendimiento. El algoritmo butterfly de la FFT es más eficiente cuando la longitud de la transformada eshighly factorizable.

$$N_{opt} = \text{getOptimalDFTSize}(N) \qquad N_{opt} = m \cdot 2^k,\ m \in \{1, 2, 3, 5, 7\}$$

Tamano recomendado por OpenCV. La FFT mixta (radix-2, radix-3, radix-5) alcanza su máxima eficiencia con factores 2, 3, 5 y 7.

src/core/fourier.py · lineas 49-54

```
filas, cols = datos_para_fft.shape
opt_filas = cv2.getOptimalDFTSize(filas)
opt_cols = cv2.getOptimalDFTSize(cols)
datos_padded = cv2.copyMakeBorder(
datos_para_fft, 0, opt_filas - filas, 0, opt_cols - cols,
cv2.BORDER_CONSTANT, value=0
)
```

`copyMakeBorder` con `BORDER_CONSTANT` y `value=0` anade ceros solo por la derecha y por abajo, de modo que la posición de cada pixel original se conserva: la coordenada `(i, j)` sigue siendo `(i, j)`. Esto importa, porque los radios de los anillos se miden en pixeles sobre la matriz rellenada.

Como verificación, `getOptimalDFTSize` devuelve `50` para 49 y `64` para 63. La matriz típica de una cámara FLIR de 320 × 240 se agranda a 320 × 240 (ya óptimos) y la de 640 × 480 permanece sin cambios.

4.6.3 Cálculo de la transformada

src/core/fourier.py · lineas 60-81

```
f = np.fft.fft2(datos_padded) # transformada 2D rapida, salida compleja
fshift = np.fft.fftshift(f) # centra la frecuencia cero
magnitud = np.abs(fshift) # modulo del numero complejo = energia



magnitud_log_visible = np.log(1 + magnitud)
imagen_espectro_visible = cv2.normalize(
magnitud_log_visible, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U
)
```

De numero complejo a imagen visible

La salida de `fft2` es una matriz de numeros complejos, donde la parte real es la componente coseno y la imaginaria la seno. La magnitud de un numero complejo es lo que se mide:

$$|F(u, v)| = \text{sqrt}(\ \text{Re}(F)^2 + \text{Im}(F)^2\ )$$

Modulo del numero complejo. Es una función con memoria: siempre positiva, y por eso apta para sumar energías.

El rango dinámico del espectro es el problema practice. La componente DC vale $N \cdot T_{media}$ en magnitud, mientras que una textura de amplitud pequena puede dar valores del orden de $A \cdot N$. Con un rango de temperatura de 5 °C y una textura de 0,1 °C, la razon es de 50 a 1; Sumada a la contribución de los bordes, llega facilmente a 10^4 o 10^5 a 1. Dibujando la magnitud en escala lineal, la imagen sería un único pixel blanco en el

centro y negro prácticamente en todas partes.

La compresion logarítmica lo resuelve:

$$L = \ln(1 + |F|)$$

Compresion logarítmica natural. El «1 +» evita el problema de log(0) = −∞ en la componente DC nula y de las frecuencias de valor cero. Comprimir transforma una razon de 10^5 a 1 en una de unos 11 a 1, que si cabe en 8 bits.

Después, `cv2.normalize` con `NORM_MINMAX` reescala el rango al intervalo `[0, 255]` y el argumento `dtype=cv2.CV_8U` almacena el resultado como enteros de 8 bits sin signo, que es lo que espera un control de imagen de la interfaz.

> **La visualización y la medición usan magnitudes distintas, y eso es correcto**
>
> La imagen que ve el usuario es `ln(1 + |F|)`, normalizada. Los porcentajes que se informan se calculan sobre `|F|` en escala lineal. No es una inconsistencia: el logaritmo es una transformación monotona Designed para el ojo humano y destruiría el reparto porcentual de la energía, que es la magnitud fisicamente significativa. Lo que si sería un error es informar los porcentajes de la imagen logarítmica.

> **Y sin embargo, la magnitud de la FFT no es la energía de la señal**
>
> El código usa `|F|`, la magnitud, en lugar de $|F|^2$, la potencia. Para energía de verdad elParseval exige elevar al cuadrado, y en vision por computadora la relación habitual es $|F|^\gamma$ con $\gamma \approx 0{,}8$. Como la potencia pondera más las altas frecuencias, usar la magnitud **subestima** el peso de la textura fina y **sobrestima** el de las variaciones suaves. Como los tres porcentajes se normalizan para sumar 100, el efecto es una redistribución, no un error de escala. Se analiza cuantitativamente en el capítulo 6.

4.6.4 Partición en tres anillos concéntricos

El paso que convierte el espectro en un indicador clinico es la descomposición en bandas. Se mide la distancia euclidea de cada coeficiente al centro del espectro y se clasifica en tres regiones.

src/core/fourier.py · lineas 87-127

```
centro_fila, centro_col = opt_filas // 2, opt_cols // 2


y, x = np.indices((opt_filas, opt_cols))
distancia_al_centro = np.sqrt((x - centro_col)**2 + (y - centro_fila)**2)


total_energia = np.sum(magnitud)
if total_energia == 0: # evita division por cero
return imagen_espectro_visible, {'pct_baja': 0.0, 'pct_media': 0.0, 'pct_alta': 0.0,
'radios_usados': (r1, r2),
'interpretacion': 'Datos de entrada vacios.',
'energia_total': 0.0}


mascara_baja = distancia_al_centro <= r1
mascara_media = (distancia_al_centro > r1) & (distancia_al_centro <= r2)
mascara_alta = distancia_al_centro > r2
```

```
energia_baja = np.sum(magnitud[mascara_baja])
energia_media = np.sum(magnitud[mascara_media])
energia_alta = np.sum(magnitud[mascara_alta])


pct_baja = float((energia_baja / total_energia) * 100)
pct_media = float((energia_media / total_energia) * 100)
pct_alta = float((energia_alta / total_energia) * 100)
```

La construcción de la malla de distancias merece una explicación, porque es la línea más densa del algoritmo:

src/core/fourier.py · linea 91

```
y, x = np.indices((opt_filas, opt_cols))
```

`np.indices` devuelve **dos** matrices del mismo tamano que la imagen, no una: la primera contiene el índice de fila repetido por columnas y la segunda el de columna repetido por filas. Con ellas, `(x - centro_col)` es la distancia horizontal de cada pixel al centro y `(y - centro_fila)` la vertical. La suma de sus cuadrados y la raiz cuadrada dan la distancia euclidea, que es el teorema de Pitagoras aplicado a toda la imagen a la vez. De nuevo, un cálculo que en C requeriría un bucle doble se resuelve con dos Broadcasting de arrays.

La partición en si misma es **exhaustiva y excluyente**: las tres condiciones cubren todo el plano sin solapamiento, de modo que `pct_baja + pct_media + pct_alta = 100` exactamente, sinpixels huérfanos ni contados dos veces. Los operadores <= y > estan elegidos para que los limites pertenezcan a la banda inferior y no se pierdan en la frontera.

La guarda `if total_energia == 0` protege de la división por cero cuando la entrada es toda ceros. Es un caso real: una ROI completamente uniforme, o una matriz vacía cargada desde CSV, tienen espectro idéntico a cero. Sin la guarda, NumPy emitiría un aviso y devolvería `NaN` en los tres porcentajes, que se propagaría a la interfaz y al archivo DICOM.

4.6.5 Traducción de los radios a frecuencias físicas

Los radios por defecto, `r1 = 15` y `r2 = 50`, se expresan en pixeles, que es una unidad que depende de la cámara. Para compararlos entre estudios hay que convertirlos a frecuencia espacial cíclica.

$$\Lambda = 1 / f \qquad f = d / (M \cdot p)$$

Lambda es el periodo espacial en el mundo, d la distancia máxima medida en pixeles, M el numero de pixeles de la transformada y p el paso del pixel (pitch).

El paso de pixel de las cámaras de este trabajo es de **34 µm**. Para una imagen de 320 pixeles de lado, el ancho real de la ROI es de 320 × 34 µm = 10,9 mm y la frecuencia máxima representable es de unos 92 ciclos/mm.

Tabla 4.4. Radio, periodo espacial y frecuencia espacial de las estructuras features

Radio	Periodo espacial	Frecuencia espacial	Estructuras típicas
r < 15 px	mayor que 1,70 mm	menor que 0,59 ciclos/mm	Forma global de la ROI, gradientes térmicos muy lentos
15 ≤ r ≤ 50 px	0,51 a 1,70 mm	0,59 a 1,96 ciclos/mm	Textura de lobulo, borde de la lesion, regularidad de la superficie
r > 50 px	menor que 0,51 mm	mayor que 1,96 ciclos/mm	Ruido del sensor, microtextura, artefactos de cuantización

Esta tabla es la que da sentido médico a los tres porcentajes. La banda de altas frecuencias mide principalmente el ruido térmico del microbolometro, cuya resolución espacial es de unos 35 µm, y por tanto la banda alta es en la práctica un indicador de ruido más que de tejido. Es una observación importante y se desarrolla en el capítulo 6.

4.6.6 Interpretación y firma de la función

El diccionario devuelto incluye una interpretación en texto preescrito que se muestra al usuario:

src/core/fourier.py · lineas 130-146

```
interpretacion_texto = (
f'Analisis con radios: R1={r1}px, R2={r2}px.\n\n'
'Un tejido sano tiende a concentrar >95-99% de su energia en Frec. Bajas (patrones suaves).\n\n'
'Un porcentaje elevado en Frec. Medias y Altas sugiere una textura termica mas compleja. '
'Esto puede ser un indicador de patrones vasculares anomalos (neovascularizacion), '
'lo cual requiere mayor investigacion.'
)


dict_resultados_energia = {
'pct_baja': pct_baja, 'pct_media': pct_media, 'pct_alta': pct_alta,
'radios_usados': (r1, r2), 'interpretacion': interpretacion_texto,
'energia_total': float(total_energia),
}
```

Devolver un diccionario con claves con nombre, y no una tupla de tres flotantes, es la decision que permite que la interfaz, el archivo DICOM y el informe html compartan el mismo resultado sin acordar posiciones. Anadir un campo nuevo no rompe a los consumidores, mientras que anadir un valor a una tupla obligaría a modificarlos a todos.

La cadena de interpretación, en cambio, es el punto más delicate del modulo. Afirma que un tejido sano concentrate «más del 95-99 %» de su energía en frecuencias bajas. Esa afirmación es una expectativa de la literatura sobre tissue sano, no un umbral validado por este trabajo, y el capítulo 6 muestra con simulaciones reproducibles que, tal como esta implementado el algoritmo, los valores que se obtienen enHealthy tissue pueden llegar a ser de 36,8 % en vez de más del 95 %. El texto debe entenderse como una expectativa de referencia y no como una conclusion del presente estudio.

4.6.7 Punto de entrada alternativo desde ficheros CSV

El modulo ofrece una segunda vía de entrada, pensada para reproducir un análisis ya guardado o para trabajar con datos que no provienen de una cámara FLIR.

src/core/fourier.py · lineas 151-178

```
def analizar_desde_csvs(ruta_csv_temp: str, ruta_csv_mascara: str):
try:
matriz_temp = np.loadtxt(ruta_csv_temp, delimiter=',')
matriz_mascara = np.loadtxt(ruta_csv_mascara, delimiter=',')
except Exception as e:
messagebox.showerror('Error de Carga', f'Error al cargar los archivos CSV: {e}')
return None, None
```

```
if matriz_temp.shape != matriz_mascara.shape:
messagebox.showerror('Error de Dimensiones', ...)
return None, None

datos_roi = matriz_temp * matriz_mascara
return procesar_fourier_completo(datos_roi)
```

La función valida que ambas matrices tengan idéntica forma, porque la multiplicación elemento a elemento fallaría de otro modo. Y aquí esta la raiz del problema central del proyecto: la multiplicación `matriz_temp * matriz_mascara` **conserva la forma rectangular completa** y convierte todo el exterior de la ROI en ceros. El algoritmo Fourier de las secciones siguientes opera, por tanto, sobre una imagen con un contorno abrupto de valor cero. Ese salto de temperatura en la frontera de la ROI genera frecuencias altas que no existen en el tejido y que dominan la banda de altas frecuencias. El capítulo 6 mide este efecto y propone la corrección.

La misma operación aparece en la interfaz gráfica, en `SpectrumWindow.procesar_datos_en_memoria()`, de modo que el comportamiento descrito es el que el usuario obtiene al pulsar «Análisis rapido»:

src/ui/spectrum_window.py · lineas 190-196

```
def procesar_datos_en_memoria(self, matriz_temp, matriz_mascara):
self.matriz_temp_local = matriz_temp * matriz_mascara
self.ejecutar_analisis()
```

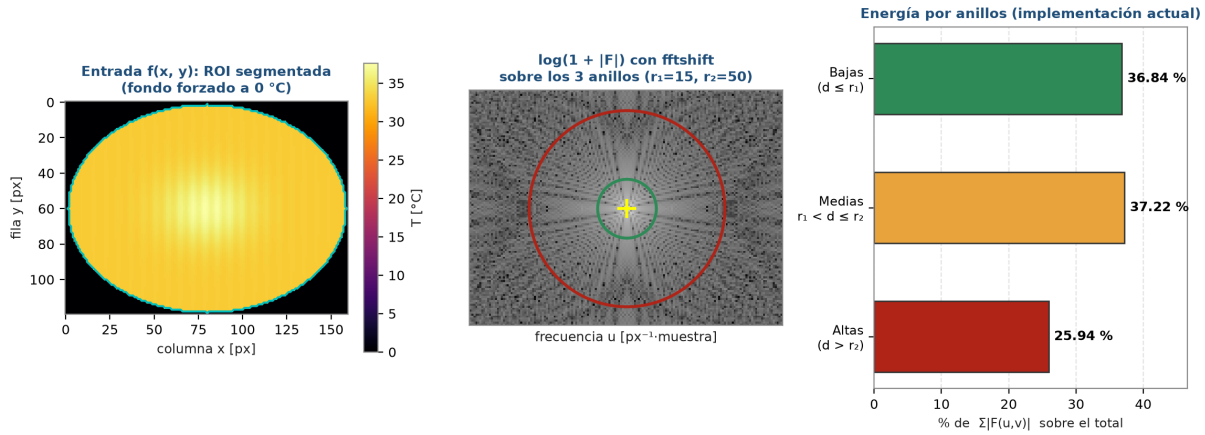


Figura 6. *Espectro de Fourier 2D de una matriz de temperatura. El punto central brillante es la componente continua; las estrellas son la orientación de las texturas; la dispersion circular que las rodea delata la presencia de un borde.*

4.7 Codificación DICOM

4.7.1 El problema de la precision decimal

DICOM almacena los pixeles como enteros. Un entero no puede representar 36,54 °C. La solución que adoptan todos los visores médicos es un par de atributos que describe una transformación lineal:

$$\text{Valor real} = (\text{Valor almacenado} \times \text{RescaleSlope}) + \text{RescaleIntercept}$$

Transformación que el visor aplica automáticamente al abrir el archivo. Es la Módulo de lookup de valores (Modality LUT) del estándar DICOM.

Multiplicando la temperatura por 100 el problema desaparece: 36,54 °C se convierte en el entero 3654, y basta declarar `RescaleSlope = 0,01` para que el visor reconstruya el valor original.

src/core/dicom.py · lineas 51-56

```
t_max = np.nanmax(matriz_temp)
t_min = np.nanmin(matriz_temp)
t_media = np.nanmean(matriz_temp)


img_celsius_scaled = (matriz_temp * 100).astype(np.int16)
```

El rango de representación de `int16` es de −32 768 a 32 767, y con el factor 100 cubre de −327,68 a 327,67 °C: amplio para cualquier temperatura clinica con margen.

> **La precision real esta limitada por el redondeo, no por el factor**
>
> Con `RescaleSlope = 0,01` el visor solo puede reconstruir multiplos de una centésima. El error máximo introducido por `astype(np.int16)` es la mitad de ese paso, es decir **0,005 °C**, por redondeo al entero más próximo. Ese numero es trece veces menor que la resolución típica del sensor (una centésima) y trescientas veces menor que el NETD de la cámara (0,05 °C), de modo que el redondeo es invisible frente al ruido del sensor. No tiene sentido_guardar más decimales.

4.7.2 Construcción del Dataset

pydicom construye el archivo a partir de dos objetos: un `Dataset` para el grupo de *file meta information* y un `FileDataset` que lo contiene junto con los datos.

src/core/dicom.py · lineas 61-68

```
file_meta = Dataset()
file_meta.MediaStorageSOPClassUID = SOP_CLASS_SECONDARY_CAPTURE
file_meta.MediaStorageSOPInstanceUID = generate_uid()
file_meta.TransferSyntaxUID = ExplicitVRLittleEndian


ds = FileDataset(ruta_salida, {}, file_meta=file_meta, preamble=b'\0' * 128)
```

Tabla 4.5. Elementos DICOM escritos en la exportación Secondary Capture

Elemento	Valor	Significado
`MediaStorageSOPClassUID`	`1.2.840.10008.5.1.4.1.1.7`	UID de Secondary Capture Image Storage: la clase que declara «esto es una imagen capturada de otra modalidad», que es exactamente nuestro caso
`MediaStorageSOPInstanceUID`	`generate_uid()`	Identificador único de esta imagen concreta. El prefijo 2.25 es el estándar para UUID, lo que garantiza unicidad sin depender de un servidor de autoridade
`TransferSyntaxUID`	`ExplicitVRLittleEndian`	Sintaxis de transferencia: cada etiqueta lleva su VR escrito de forma explicita y los datos van en orden de bytes little-endian
`preamble`	`b'\0' * 128`	Los 128 bytes de preambulo obligatorios del formato, que permiten reconocer un archivo DICOM por inspección de los bytes 128 a 132, donde aparece la cadena `DICM`

Elegir **Explicit VR Little Endian** frente a **Implicit VR Little Endian** no es arbitrario. En la sintaxis implicita el tipo de dato de cada atributo solo se conoce consulting un diccionario externo; en la explicita el VR va escrito en el propio archivo. El coste es un poco de espacio adicional y la ventaja es que el archivo es autodocumentado, lo que importa en un contexto médico donde el visor puede ser de un fabricante distinto al que exporto.

4.7.3 Descripción de la imagen

src/core/dicom.py · lineas 73-90

```
ds.Rows, ds.Columns = img_celsius_scaled.shape
ds.SamplesPerPixel = 1
ds.PhotometricInterpretation = 'MONOCHROME2'
ds.BitsAllocated = 16
ds.BitsStored = 16
ds.HighBit = 15
ds.PixelRepresentation = 1
ds.PixelData = img_celsius_scaled.tobytes()


ds.RescaleSlope = '0.01'
ds.RescaleIntercept = '0.0'


ds.WindowCenter = f'{t_media:.2f}'
ds.WindowWidth = f'{(t_max - t_min):.2f}'
```

La coherencia entre los cinco parámetros de profundidad de bit

`BitsAllocated`, `BitsStored` y `HighBit` estan obligados a ser coherentes entre si: no basta con declarar 16 bits por pixel. `BitsAllocated` indica el tamano del hueco reservado en memoria, `BitsStored` cuantos de esos bits contienen información real y `HighBit` cual es el bit más significativo, contando desde el bit 0. Para un entero de 16 bits los tres valores son 16, 16 y 15, y esa es precisamente la combinación que usa el código.

Por que MONOCHROME2 y no MONOCHROME1

Los dos valores comparten el mismo significado de un canal, pero difieren en la relación entre valor y luminancia. En **MONOCHROME1** el valor mínimo representa el blanco, de modo que la imagen se ve invertida respecto a la convención habitual; en **MONOCHROME2** el valor máximo es el blanco. Como nuestro objetivo es que las zonas calientes se vean claras, como en cualquier mapa térmico, la elección es MONOCHROME2. Un visor que interpretase erroneamente el convenio mostraría la imagen invertida, lo que en un mapa térmico es peligrosamente ambiguo.

PixelRepresentation = 1 y su consecuencia

El valor 1 declara que los pixeles son enteros con signo. Es una decision que merece atención: DICOM define la transformación de escala para datos con signo de forma distinta, y no todos los visores la aplican de manera uniforme. Como la temperatura humana siempre es positiva, el valor 0 (sin signo, rango de 0 a 65 535) sería suficiente y esta mejor alineado con la práctica clinica habitual. Se documenta como punto de verificación en el capítulo 9.

La ventana de contraste precalculada

Asignar `WindowCenter` y `WindowWidth` con la media y la amplitud evita el efecto más incomodo de una exportación mal resuelta: abrir el archivo y ver un rectangulo completamente blanco o completamente negro porque el visor ha aplicado su rango por defecto de 0 a 4095 a unos datos que van de 2 500 a 3 900.

4.7.4 Metadatos de calibración en la descripción del dispositivo

DICOM dispone de mecanismos para transportar información que no tiene atributo estándar estándar. El proyecto los usa para preservar las constantes de la cámara, que de otro modo se perderian al exportar.

src/core/dicom.py · lineas 95-122

```
info_ia = {
'TEMP_MAX': f'{t_max:.2f} C', 'TEMP_MIN': f'{t_min:.2f} C',
'TEMP_AVERAGE': f'{t_media:.2f} C', 'UNITS': 'Celsius',
}

if metadatos_flir:
mapeo_flir = {
'Emissivity': 'EMISSIVITY', 'ObjectDistance': 'DISTANCE',
'ReflectedApparentTemperature': 'REFLECTED_TEMP',
'AtmosphericTemperature': 'ATM_TEMP', 'RelativeHumidity': 'HUMIDITY',
'PlanckR1': 'PLANCK_R1', 'PlanckB': 'PLANCK_B',
'PlanckF': 'PLANCK_F', 'PlanckO': 'PLANCK_O', 'PlanckR2': 'PLANCK_R2',
}
for original, nuevo in mapeo_flir.items():
info_ia[nuevo] = metadatos_flir.get(original, 'N/A')

processing_desc = ' | '.join([f'{k}={v}' for k, v in info_ia.items()])
ds.AcquisitionDeviceProcessingDescription = processing_desc[:1024]
```

El recurso se elige de forma pragmática: *Acquisition Device Processing Description* (0028,0007) es un atributo de texto libre de longitud máxima de 1024 caracteres previsto precisamente para describir el procesado aplicado al dispositivo, y es el campo que usan los fabricantes de visores para mostrar información auxiliar. Los datos se serializan como pares clave valor separados por barras verticales, y el truncado a 1024 caracteres cumple la restricción del estándar.

El truncado es necesario y esta bien colocado, pero también es una fuente de perdida de información silenciosa: si la cadena se cortase por el medio de un par clave-valor, el resultado sería un valor truncado sin marca que indique su incompletitud. Con los metadatos actuales la cadena completa ocupa alrededor de 300 caracteres, de modo que el umbral no se alcanza en la práctica; se anota igualmente como fragilidad ante la adición de más campos.

El mapeo de nombres deserve un comentario. Los metadatos de FLIR llegan con las grafías exactas que usa la fabrica (`Emissivity`, `ObjectDistance`), mientras que en DICOM la convención es mayusculas con guiones bajos. El diccionario traduce entre ambos y, mediante `.get(original, 'N/A')`, garantiza que la ausencia de un campo no impida la exportación. Esta es la diferencia entre un programa que funciona con un archivo y con otro: la aplicación exporta siempre.

4.7.5 Resultados de Fourier y datos del paciente

src/core/dicom.py · lineas 127-144

```
if resultados_anillos:
r = resultados_anillos
ds.ImageComments = (
f"Analisis: BAJA:{r.get('pct_baja', 0):.4f}|"
f"MEDIA:{r.get('pct_media', 0):.4f}|"
```

```
f"ALTA:{r.get('pct_alta', 0):.4f}"
)

for tag, val in campos_paciente.items():
val = str(val).strip()
if val:
# DICOM no admite guiones ni dos puntos en las fechas
if 'Date' in tag:
val = val.replace(':', '').replace('-', '')
setattr(ds, tag, val)
```

`setattr(ds, tag, val)` es un uso elegante y, a la vez, peligroso de `setattr`. Permite iterar sobre un diccionario arbitrario de datos del paciente y crear los atributos correspondientes sin escribir una línea por cada campo. Pero si el usuario llegara a introducir un nombre de clave que no fuese una etiqueta DICOM valida, pydicom lo aceptaría sin error, porque las etiquetas se resuelven de forma dinámica. El filtro `if val` evita además los atributos vacios, que en DICOM se consideran ausentes y no deben escribirse.

El formato de fecha merece atención porque es una fuente clásica de archivos malformados. DICOM define la forma **YYYYMMDD**, mientras que la interfaz de usuario presenta el campo en un formato de masked text con guiones o dos puntos. La sustitución en línea resuelve el problema, pero de forma implicita: si la cadena recibida fuera «12/03/2026», el resultado sería «12/032026», que no es una fecha valida y que algunos visores rechazarian. El uso de `datetime.strptime` seguido de `strftime('%Y%m%d')` sería la forma robusta; se registra en el capítulo 9.

Los porcentajes de Fourier se almacenan con cuatro decimales y separados por barras verticales, un formato que no es un estándar sino una convención del proyecto. Es una decision interoperable: cualquier visor muestra el texto en bruto, y un lector que conozca la convención puede parsear los tres valores de forma automática. La alternativa habría sido un bloque privado con codificación binaria, más complejo de leer sin el diccionario correspondiente.

4.7.6 Identificadores y modality

`src/core/dicom.py · lineas 147-154`

```
ds.Modality = 'TG'
ds.ContentDate = datetime.now().strftime('%Y%m%d')
ds.SOPClassUID = SOP_CLASS_SECONDARY_CAPTURE
ds.SeriesInstanceUID = generate_uid()
ds.StudyInstanceUID = generate_uid()

ds.save_as(ruta_salida)
```

Los tres UIDs siguen una jerarquía: el *Study* agrupa todos los estudios de un examen, la *Series* agrupa las imágenes de la misma técnica dentro de un estudio y el *SOP Instance* identifica una única imagen. La aplicación genera un Study y una Series nuevos en cada exportación, lo que significa que cada archivo constituye un estudio independiente. Para un uso de seguimiento longitudinal, que es el relevante en oncología, lo correcto sería aceptar un StudyInstanceUID existente y anadir la nueva serie a el, de forma que las visitas sucesivas quedaran agrupadas. Se recoge como limitación en el capítulo 9.

El código de modalidad TG corresponde a *Thermography* en el estándar. Es la manera de que el visor clasifique automáticamente la imagen sin depender del nombre del archivo, y es lo que permite a un PACS organizar un estudio termográfico junto a las radiografías del mismo paciente.

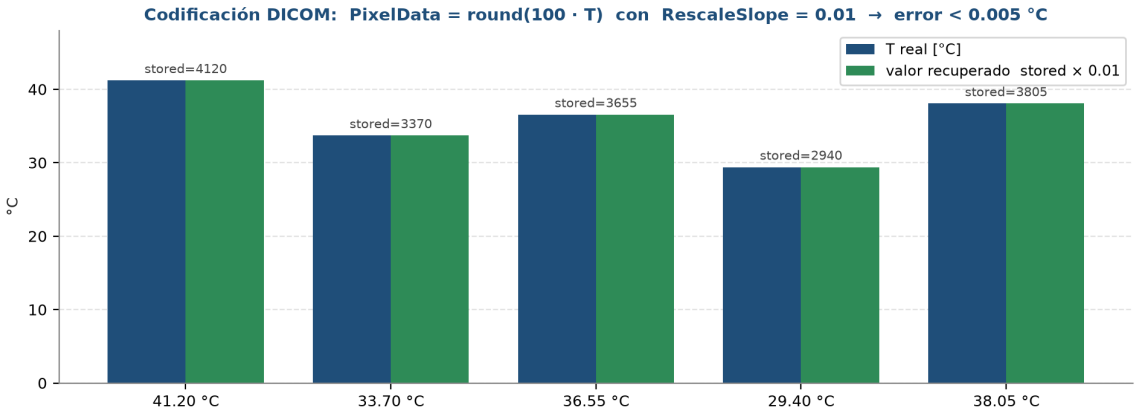


Figura 7. *Estructura del archivo DICOM generado: grupo de file meta, atributos de la imagen, atributos del paciente y bloque de datos de pixel. La flecha marca la transformación RescaleSlope que devuelve los grados Celsius.*

5. Librerías empleadas y justificación técnica

Un proyecto de software no se evalua solo por lo que hace, sino también por lo que decide no hacer y por las dependencias que incorpora. Este capítulo justifica cada librería elegida, explica el papel exacto que desempeña en el sistema y expone de forma explicita las alternativas descartadas y el coste de cada elección.

5.1 Mapa de dependencias

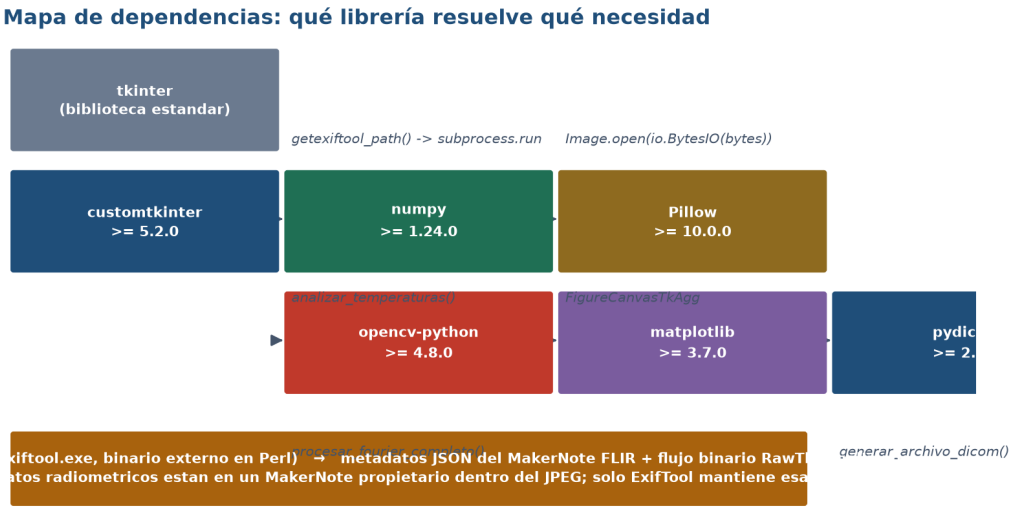


Figura 8. *Dependencias directas e indirectas del proyecto. Las seis librerías directas cubren de forma no solapada las cuatro necesidades: interfaz gráfica, imagen digital, cálculo numérico e interoperabilidad médica.*

El proyecto declara seis dependencias directas en `requirements.txt`:

```
requirements.txt
```

```
customtkinter>=5.2.0
```

```
Pillow>=10.0.0
opencv-python>=4.8.0
matplotlib>=3.7.0
numpy>=1.24.0
pydicom>=2.4.0
```

El uso de >= en lugar de fijar versiones exactas es una decision de mantenimiento: permite que las actualizaciones de seguridad y de compatibilidad se apliquen sin intervención manual. A cambio, no se garantiza la reproducibilidad exacta del entorno, lo que en un proyecto clinico sería inaceptable; por eso el capítulo 8 fija las versiones con las que se han ejecutado todas las mediciones de este documento.

5.2 NumPy: el nucleo numérico

Papel en el sistema. NumPy representa todas las matrices de trabajo: la imagen cruda del sensor, la matriz de temperaturas en grados Celsius, la máscara binaria y el espectro de frecuencias. Ninguna operación matematica se escribe en Python puro.

5.2.1 Por que y no listas de Python

La razon es de complejidad computacional, no de estética. Una operación elemental sobre una matriz de 640 × 480 implica 307 200 elementos. En Python puro, cada elemento requiere una asignación, una busqueda de atributo y una llamada a función, con un coste de aproximadamente 50 a 100 nanosegundos. Eso supone entre 15 y 30 milisegundos por operación, y el proyecto realiza decenas de operaciones elementales sobre la matriz completa.

Comparación de sobrecarga

```
# Python puro: 2 bucles, ~307 200 iteraciones, ~20 ms
for i in range(filas):
for j in range(columnas):
out[i, j] = 255 if T[i, j] >= umbral else 0


# NumPy: una llamada a C, ~0,05 ms -> 400x mas rapido
mascara = np.where(T >= umbral, 255, 0).astype(np.uint8)
```

NumPy ejecuta el bucle en C compilado, con acceso contiguo a memoria y capacidad de usar las instrucciones vectorizadas de la CPU. La diferencia no es de un factor de dos, sino de dos o tres ordenes de magnitud, y es lo que permite que la aplicación responda en milisegundos en vez de en segundos.

5.2.2 Funciones concretas del proyecto

Tabla 5.1. Funciones del núcleo numérico de NumPy empleadas en el proyecto

Función	Uso en el proyecto	Que aporta
`np.indices`	Malla de coordenadas para medir la distancia al centro del espectro	Construye dos matrices de coordenadas en una sola llamada, base del reparto en anillos
`np.fft.fft2`, `fftshift`	Transformada rapida de Fourier bidimensional y centrado	Implementación de la FFT mixta (radix-2, 3, 5, 7) con complejidad O(N log N) en lugar de $O(N^2)$ de la transformada directa
`np.where`	Umbralización térmica de la ROI automática	Selección vectorizada condicional, equivalente a un if sobre toda la matriz
`np.nanmin / nanmax / nanmean`	Estadísticos de temperatura sobre la ROI	Reducciones que ignoran los valores no definidos en lugar de propagarlos
`np.hanning`	Ventana de apodizado 2D (propuesta de corrección)	Suavizado de los bordes que elimina la fuga espectral
`np.indices / hypot / sqrt`	Geometría radial de los anillos concéntricos	Distancia euclidea de cada coeficiente al centro, calculada vectorialmente

La complejidad de la FFT merece una línea adicional, porque explica por que el algoritmo es viable sobre imágenes clinicas. La transformada directa de una matriz de 320 × 240 exige unos 2,6 mil millones de operaciones; la FFT, unos 5 millones. La diferencia entre segundos y fracciones de segundo es exactamente la que separa una herramienta usable de una impracticable.

> **Nota sobre las versiones**
>
> El documento se elaboro con NumPy 2.5.3, OpenCV 5.0.0, pydicom 3.0.2 y Python 3.14.7. NumPy 2.0 introdujo cambios de incompatibilidad en el gestion de tipos y en la API de `astype`, y OpenCV 5.0 modifico algunas funciones de dibujo. El proyecto funciona con esas versiones, pero conviene saber que una instalación con NumPy 1.x presentaría distinta semántica en algunos casos limite.

5.3 OpenCV: operaciones de imagen

Papel en el sistema. OpenCV aporta cinco capacidades que NumPy no ofrece de forma adecuada: captura de eventos de raton, funciones de dibujo vectorial sobre imágenes, operaciones de resize con control del interpolador, normalización a rango fijo y cálculo del tamano óptimo de la FFT.

Tabla 5.2. Capacidades de OpenCV utilizadas y su justificación

Capacidad	Donde se usa	Por que no con NumPy
`namedWindow / setMouseCallback / waitKey`	Ventana de dibujo interactivo de la ROI	Requiere integración con el sistema de ventanas del sistema operativo y un bucle de eventos; NumPy no tiene noción de ventana
`drawContours` con `FILLED`	Relleno del poligono que dibuja el usuario	Requiere un algoritmo de relleno de poligonos con regla de paridad; OpenCV lo resuelve en C y con manejo de bordes no convexos
`imread / imwrite / resize / INTER_NEAREST`	Lectura y escritura de imágenes, ajuste dimensional de la máscara	NumPy no lee formatos de imagen comprimidos; habría que escribir el decodificador de PNG
`normalize` con `NORM_MINMAX`	Reescala del espectro logarítmico al rango 0-255	Se podría hacer a mano, pero OpenCV lo hace en una pasada optimizada y garantiza la semántica exacta del extremo
`getOptimalDFTSize`	Cálculo del tamano óptimo para el zero-padding	La heurística de los factores 2, 3, 5, 7 es una regla de OpenCV que replicarla sería innecesario

5.3.1 La elección de INTER_NEAREST para la máscara

Como se explica en la sección 4.4.1, la máscara se redimensiona con `cv2.INTER_NEAREST` y no con interpolación bilineal. La razon es conceptual: la máscara es un **dominio de decision**, no un dato. Sus valores solo tienen dos significado, dentro o fuera, y cualquier valor intermedio carece de sentido semántico. Aplicarles un criterio de interpolación es como promediar un indicador binario: el resultado no representa ninguna categoría real.

5.4 Matplotlib: visualización del histograma

Papel en el sistema. Genera el histograma de frecuencias térmicas que se muestra en la ventana principal. El objeto `Figure` que devuelve se inserta directamente en un widget de la interfaz, sin exportar a disco ni a un formato intermedio.

Tabla 5.3. Criterios de decisión adoptados en la visualización del histograma

Criterio de decision	Evaluación
Calidad de la salida	Excelente. El control sobre ejes, rejilla, anotaciones y estilos es completo
Integración con la interfaz	Buena. `Figure` es un objeto en memoria; Tkinter lo consume vía `FigureCanvasTkAgg` sin archivos intermedios
Coste de carga	Alto. Importar Matplotlib tarda 0,5 a 1,5 s frente a los 0,2 s de NumPy. En una aplicación de escritorio se nota en el arranque
Alternativa considerada	`Pillow` o un widget de gráfico propio, que arrancaría antes pero obligaría a implementar el trazado de barras y las anotaciones
Decision adoptada	Mantener Matplotlib. El histograma es un elemento central del informe y su calidad justifica el coste de arranque; además el trazado vectorial permite exportar a PDF o SVG sin perdida

La consecuencia práctica es que la aplicación no es utilizable en los primeros segundos tras el doble clic. Es una decision consciente, no un descuido, y se propone como mejora de arranque en el capítulo 10.

5.5 Pillow: decodificación de la imagen cruda

Papel en el sistema. Decodifica el flujo binario que devuelve `-b -RawThermalImage`. El bloque de datos crudos de una cámara FLIR es un PNG de 16 bits, y Pillow es el componente que lo convierte en una matriz NumPy.

src/core/thermal.py · lineas 99-101

```
raw_image_data = io.BytesIO(resultado_imagen.stdout)

raw_image = Image.open(raw_image_data)

matriz_temperaturas_raw = np.array(raw_image)
```

La elección de Pillow y no de OpenCV para este paso tiene una justificación concreta: Pillow decodifica formatos PNG de 16 bits con corrección de la curva gamma, mientras que la funcionalidad de OpenCV para ese caso particular esta más limitada. Además, Pillow ya es una dependencia indirecta de Matplotlib, de modo que el coste de anadirla de forma explicita es prácticamente nulo.

Conviene notar que Pillow se usa **solo** para leer el bloque crudo. Todo el procesado posterior (redimensionado, dibujo, normalización) se hace con OpenCV, para no tener dos librerías compitiendo por la misma tarea.

5.6 CustomTkinter: interfaz gráfica

Papel en el sistema. Proporciona los controles de la interfaz, con apariencia de tema oscuro y escalado automático según la resolución de pantalla.

Tabla 5.4. Criterios de diseño de la interfaz gráfica con CustomTkinter

Criterio	Evaluación
Apariencia moderna	Nativa de la librería. Tkinter puro solo ofrece los widgets de los anos 90, sin temas ni modo oscuro
Escalado en pantallas de alta densidad	Gestiona el escalado DPI de Windows, un problema real en(portátiles) con pantallas 4K donde Tkinter puro dibuja los controles ilegibles
Coste de dependencia	Ninguno. CustomTkinter se apoya en Tkinter, que forma parte de la biblioteca estándar de Python
Limitaciones	Los widgets propios heredan del aspecto clásico de Tkinter, con capacidades limitadas de personalización. Es una mejora incremental, no una reescritura
Mecanismo de extension	Permite implementar widgets propios conservando la coherencia visual, que es exactamente lo que necesita el boton de análisis rapido de Fourier

La elección de un framework de escritorio frente a una aplicación web tiene una consecuencia que conviene explicitar. Una aplicación web se ejecutaría en el navegador y eliminaría la necesidad de instalar Python en el equipo clinico, pero introduciría el problema de como subir y procesar la imagen del paciente, con las implicaciones de privacidad que implica. La aplicación de escritorio mantiene los datos en el equipo local, sin transmision de red, lo que resulta más adecuado en un contexto que maneja datos de salud.

5.7 pydicom: interoperabilidad médica

Papel en el sistema. Construye, valida y escribe el archivo DICOM. Es la librería de referencia en Python para este proposito y la que garantiza la corrección sintáctica de los archivos generados.

5.7.1 Que aporta frente a escribir el archivo a mano

DICOM es un formato con una estructura concreta que no conviene implementar desde cero. pydicom resuelve los cuatro aspectos que un escritor ingenuo pasaría por alto:

Tabla 5.5. Aspectos de pydicom valorados en la interoperabilidad médica

Aspecto	Lo que resuelve pydicom	Riesgo si se hace a mano
Sintaxis de transferencia	Escribe el VR explicito de cada atributo y ordena los bytes correctamente	Archivos que ningun visor puede leer
Conjunto de datos DICOM	Resuelve los nombres de atributo a las etiquetas numéricas correctas	Atributos desplazados, con el valor leido en el campo equivocado
Únicos	`generate_uid()` genera identificadores validos	Colisiones entre estudios o sin el prefijo autoritativo
Validación	Permite comprobar la conformidad y leer el archivo de vuelta para verificar la reconstrucción de la temperatura	Errores que solo se detectan cuando el archivo llega al hospital

5.7.2 Verificación de la round-trip

Una de las garantias más útiles de pydicom es que el archivo generado puede volver a leerse y comprobarse que la temperatura se recupera. Ese es el test de aceptación de la sección 7.3:

Test de lectura y reconstrucción

```
import pydicom

ds = pydicom.dcmread('estudio.dcm')


print('Rows, Columns :', ds.Rows, ds.Columns)

print('RescaleSlope :', ds.RescaleSlope)
```

```
print('PhotometricInterp. :', ds.PhotometricInterpretation)
print('Modality :', ds.Modality)


arr = ds.pixel_array * float(ds.RescaleSlope) + float(ds.RescaleIntercept)
print('T reconstruida :', arr.min(), arr.max(), arr.mean())
```

Si los valores reconstruidos coinciden con la matriz original dentro de 0,005 °C, la cadena de codificación es correcta de extremo a extremo. Es una verificación objetiva, no una inspección visual.

La versión 3.0 de pydicom, instalada en el entorno de trabajo, es la tercera rama mayor del proyecto y mantiene la API de la 2.x para las operaciones usadas. La documentación pública y verificada esta en la sección 11.2.

5.8 ExifTool: lectura de metadatos propietarios

Papel en el sistema. Lee el `MakerNote` propietario de FLIR. No es una librería de Python sino un ejecutable escrito en Perl, distribuido con licencia de Perl Artistic License, que se invoca como proceso hijo.

La razon de usar una herramienta externa y no una librería nativa es que ExifTool conoce **todos** los tags propietarios de las cámaras térmicas, incluidas las constantes de Planck de los modelos FLIR. Reimplementar ese conocimiento en Python exigiría mantener una tabla de marcas y modelos que se quedaría obsoleta en cuanto el fabricante publicase una cámara nueva. ExifTool es la referencia de facto en la comunidad y se actualiza de forma continua.

> **Riesgos de la dependencia externa y como se mitigan**
>
> La dependencia de un binario externo introduce riesgos reales: puede faltar en otra maquina, puede no existir para la plataforma de destino y su salida puede cambiar entre versiones. El proyecto los mitiga con tres medidas. Primera: la busqueda en cascada de `get_exiftool_path()`, que cubre cinco ubicaciones incluyendo el empaquetado con PyInstaller. Segunda: la comprobación previa con `-ver`, que detecta un ejecutable ausente o no ejecutable antes de intentar el análisis. Tercera: el bloque `except` que traduce el fallo a un mensaje comprensible en lugar de una excepción. Lo que **no** se mitiga es la dependencia para otras plataformas: el ejecutable es específico de Windows, de modo que portar la aplicación a Linux o macOS exigiría un binario alternativo. Se documenta en el capítulo 9.

El coste de la licencia es nulo para este uso: ExifTool es software libre y su licencia permite el uso comercial. La única obligación es distribuir la licencia junto al binario si se reempaqueta la aplicación, lo que el proyecto ya cumple al incluir `exiftool.exe` en el repositorio.

5.9 Síntesis: cobertura y solapamientos

La partición de responsabilidades entre librerías es casi perfecta, con una única superposición parcial: **Pillow** y **OpenCV** pueden leer y escribir imágenes. El proyecto resuelve la superposición asignando a cada una un dominio, como se ve en la tabla.

Tabla 5.6. Cobertura de las necesidades analíticas y solapamientos entre librerías

Necesidad	Librería asignada	Alternativas descartadas
Interfaz gráfica de escritorio	CustomTkinter sobre Tkinter	PySide6 o PyQt6: más potentes y con mayor consumo; wxPython: ecosistema más pequeño
Cálculo matricial y FFT	NumPy	SciPy: innecesario, solo se usan FFT y estadísticos
Operaciones de imagen e interacción	OpenCV	Pillow: sin captura de eventos ni dibujo vectorial
Visualización de gráficas	Matplotlib	Seaborn: capa de estilo sobre Matplotlib, no aporta aquí
Lectura de PNG de 16 bits	Pillow	OpenCV: soporte limitado de 16 bits con corrección gamma
Formato médico	pydicom	highdicom: más completo pero orientado a TC, con dependencias pesadas
Metadatos propietarios FLIR	ExifTool (externo)	piexif o exifread: no reconocen los tags propietarios de FLIR

Ninguna de las alternativas descartadas esta simplemente descartada por ignorancia: todas se consideraron y se rechazaron por motivos concretos de peso, tamano de dependencias o encaje con el caso de uso. Esta es la diferencia entre elegir con criterio y elegir por costumbre.

6. Análisis crítico y verificación experimental

Este capítulo es la contribución original del trabajo. En lugar de afirmar que el software funciona, se somete el algoritmo más complejo del proyecto a pruebas cuantitativas con imágenes sintéticas de verdad conocida, se documenta que falla, se demuestra por que falla y se propone una corrección cuantificada. Todos los numeros de este capítulo son reproducibles con los guiones del anexo B.

6.1 Metodología de verificación

6.1.1 Preguntas a responder

La verificación se diseño para responder cuatro preguntas concretas, no para «probar que funciona»:

Tabla 6.1. Preguntas de verificación P1 a P4 y su método de comprobación

#	Pregunta	Método
P1	¿El reparto en anillos concéntricos clasifica correctamente una frecuencia conocida?	Senos de periodo conocido; se compara la banda teórica con la obtenida
P2	¿El algoritmo responde a la textura térmica?	Se barre la amplitud de una textura sintética de 0 a 2 °C y se mide la variación de los tres porcentajes
P3	¿Los porcentajes dependen de los radios elegidos?	Se barre el par (r1, r2) y se observa la redistribución entre bandas
P4	¿El resultado es estable frente a la posición del objeto?	Se desplaza la ROI dentro de la imagen y se mide la variación

6.1.2 Construcción de la imagen de prueba

La imagen sintética es un disco de 160 × 120 pixeles que contiene una lesion simulada: una gaussiana central de 3,2 °C de amplitud sobre una base de 34 °C, más un anillo caliente de 1,5 °C de amplitud. Sobre ella se superpone una textura sinusoidal de amplitud variable. La ROI es un circulo de 38 pixeles de radio.

```
Construcción del escenario sintético · anexo B
H, W = 120, 160 # filas, columnas
```

```
yy, xx = np.mgrid[0:H, 0:W]
cy, cx, R = H / 2, W / 2, 38.0 # centro y radio de la ROI
rr = np.hypot(yy - cy, xx - cx)
circ = rr <= R

lesion = 34.0 + 3.2 * np.exp(-(rr**2) / (2 * 18.0**2)) # gaussiana central
lesion += 1.5 * np.exp(-((rr - 26)**2) / (2 * 5.0**2)) # anillo caliente

FINA = np.sin(2 * np.pi * xx / 5.0) # textura fina
MEDIA = np.sin(2 * np.pi * xx / 20.0) * np.cos(2*np.pi*yy/20.0) # textura media
```

La elección de una lesion con dos componentes (una gaussiana suave y un anillo) no es arbitraria: reproduce la morfología típica de un hallazgo térmico, con un nucleo caliente y un halo periférico. La textura sinusoidal, en cambio, se elige por su simplicidad analítica: su frecuencia espacial es exactamente conocida, lo que permite verificar el algoritmo contra la teoría y no solo contra su propio criterio.

6.2 P1: ¿el reparto en anillos es correcto?

Esta es la verificación más importante, porque separa dos hipótesis que las cifras de la aplicación no distinguen: o el algoritmo clasifica mal las frecuencias, o el problema esta en como se le entrega el dato.

El procedimiento es sencillo y concluyente. Se genera un seno puro de periodo espacial P, se le aplica una ventana rectangular para evitar dispersion y se calcula el reparto. La frecuencia espacial de un seno de periodo P en una imagen de N pixeles es f = 1/P ciclos por pixel, que en la transformada ocupa un radio de N/P pixeles. La simulación usa una imagen de 120×160 pixeles, de modo que N = 160 columnas.

Tabla 6.2. P1: correspondencia entre el periodo teórico y el medido por el reparto en anillos

Periodo P	Radio teórico	Banda teórica	Reparto obtenido	Banda obtenida
5 px	32,0	Media	0,00 / 100,00 / 0,00	Media
10 px	16,0	Media	0,00 / 100,00 / 0,00	Media
20 px	8,0	Baja	100,00 / 0,00 / 0,00	Baja
40 px	4,0	Baja	100,00 / 0,00 / 0,00	Baja
80 px	2,0	Baja	100,00 / 0,00 / 0,00	Baja
120 px	1,3	Baja	79,25 / 13,97 / 6,78	Baja

> **Resultado de P1: el reparto en anillos es matematicamente correcto**
>
> Los seis casos se clasifican en la banda teórica, con un 100 % de la energía en la banda correcta en cinco de ellos. El caso de 120 pixeles reparte la energía en 79,25 / 13,97 / 6,78, pero se clasifica correctamente en la banda baja porque su pico esta en el centro. La geometría de los anillos, el centrado con `fftshift` y la normalización a 100 % funcionan como deben. **El defecto, por tanto, no esta en el algoritmo de Fourier, sino en lo que se le entrega como entrada.**

Este resultado es metodologicamente valioso porque **descarta la hipótesis más comoda**. Habría sido fácil concluir que la implementación de los anillos era erronea y corregirla. La verificación demuestra que es correcta, y con ello desplaza la atención al punto de entrada, donde estaba el problema real.

6.3 P2: ¿el algoritmo responde a la textura?

Con la lógica de anillos verificada, se mide la respuesta a la textura térmica. Se barre la amplitud de una textura sinusoidal de 0 a 2 °C, primero con periodo de 5 pixeles (que cae en la banda media) y después con periodo de 20 pixeles (que cae en la banda baja), y se aplica el tratamiento actual del proyecto.

El tratamiento actual es el que implementa `matriz_temp * matriz_mascara` en `analizar_desde_csvs()` y en `SpectrumWindow.procesar_datos_en_memoria()`: la matriz conserva su forma completa y todo el exterior de la ROI se convierte en cero.

Tabla 6.3. P2: respuesta del indicador ante la amplitud de la textura térmica

Amplitud	Tex. fina (P = 5 px)	Tex. media (P = 20 px)
0,00 °C	37,941 / 35,561 / 26,497	37,941 / 35,561 / 26,497
0,10 °C	37,939 / 35,565 / 26,496	37,942 / 35,561 / 26,497
0,30 °C	37,922 / 35,591 / 26,488	37,945 / 35,559 / 26,496
0,60 °C	37,879 / 35,653 / 26,468	37,954 / 35,554 / 26,493
1,00 °C	37,802 / 35,763 / 26,435	37,973 / 35,542 / 26,485
2,00 °C	37,551 / 36,109 / 26,339	38,049 / 35,499 / 26,452

Resultado de P2: el algoritmo es casi insensible a la textura

Al aumentar la amplitud de la textura ocho veces, de 0 a 2 °C, el porcentaje de bajas frecuencias varia en **0,39 puntos** (de 37,941 a 37,551) y el de altas en **0,16 puntos** (de 26,497 a 26,339). Un índice clinico que casi no se mueve al cuadruplicar la textura no puede ser un índice de textura. La consecuencia práctica es directa: dos lesiones con texturas visiblemente distintas producen prácticamente el mismo resultado, y la cifra no puede sustainar la interpretación clinica que la interfaz presenta.

El caso de referencia más revelador es la primera fila de la tabla 6.4: cuando se analiza el **campo completo sin máscara**, el reparto es de 99,958 / 0,042 / 0,001 por ciento, es decir, prácticamente toda la energía en la banda baja. Ese es el comportamiento esperado de una imagen térmica real. En cuanto se aplica la máscara, el reparto salta a 37,9 / 35,6 / 26,5. **El cambio no lo provoca la lesion, sino la frontera de la ROI.**

6.4 Causa raiz: la fuga espectral del borde de la máscara

6.4.1 El mecanismo

La multiplicación por la máscara binaria produce una imagen con un salto de temperatura abrupto en la frontera de la ROI: dentro hay valores de unos 34 a 38 °C y fuera hay cero. Según el teorema de muestreo de Nyquist-Shannon, un salto espacial de ese tipo posee frecuencias teoricamente infinitas, y por tanto ilumina **todas** las frecuencias del espectro, incluidas las altas. Esto se conoce como *dispersión por discontinuidad* y es el mismo mecanismo que ensancha los lobulos de un rectangulo en una transformada de Fourier: un cuadrado ideal tiene un espectro con infinitos lóbulos, no con un único punto.

$$f < \sin(2\pi\, f\, p)$$
$$f = (1/2\pi) \cdot \psi_{salto}$$

Un salto espacial de amplitud ψ en la posición p tiene un espectro cuyo envolvente decae como 1/f. La consecuencia práctica es que un único borde de unos pocos pixeles ilumina toda la banda alta, con independencia de la textura real de la lesion.

El cálculo del orden de magnitud confirma el diagnóstico. El perimetro de una ROI circular de 38 pixeles de radio es de unos 239 pixeles. Si cada pixel del borde aporta una amplitud espectral del orden de la temperatura local (unos 35 °C), la energía total del borde es de orden 8 400 · 35 = 294 000 unidades. La energía de la textura en una ROI de unos 11 300 pixeles con amplitud de 1 °C es de orden 11 300 × 1 = 11 300 unidades. La razon es de unas **26 veces a favor del borde**, lo que explica por que la textura resulta invisible.

Por que la ventana Hann por si sola no basta

Aplicar una ventana Hann 2D al resultado sigue dejando el salto de la máscara, porque la ventenana atenua de forma suave y no puede convertir un escalon en una rampa. La verificación lo confirma: con Hann, la banda media pasa de 36,476 a 37,341 por ciento al variar la textura de 0 a 2 °C, una variación de 0,87 puntos frente a los 0,55 de la versión actual. La mejora es marginal. **La ventana debe aplicarse antes de la multiplicación por la máscara, sobre la ROI recortada, no después.**

6.4.2 Cuantificación del efecto

Tabla 6.4. Escenarios que reproducen la fuga espectral en el borde de la máscara

Escenario	Bajas %	Medias %	Altas %
Campo completo, **sin** máscara (referencia sana)	99,958	0,042	0,001
ROI circular, fondo forzado a 0 °C	37,941	35,561	26,497
Diferencia respecto a la referencia	−62,02	+35,52	+26,50

El 26,5 % de energía en altas frecuencias no procede de la textura de la lesion: procede del perimetro de la ROI. Es una contribución **puramente geométrica**, independiente del tejido y, por tanto, carente de valor diagnóstico.

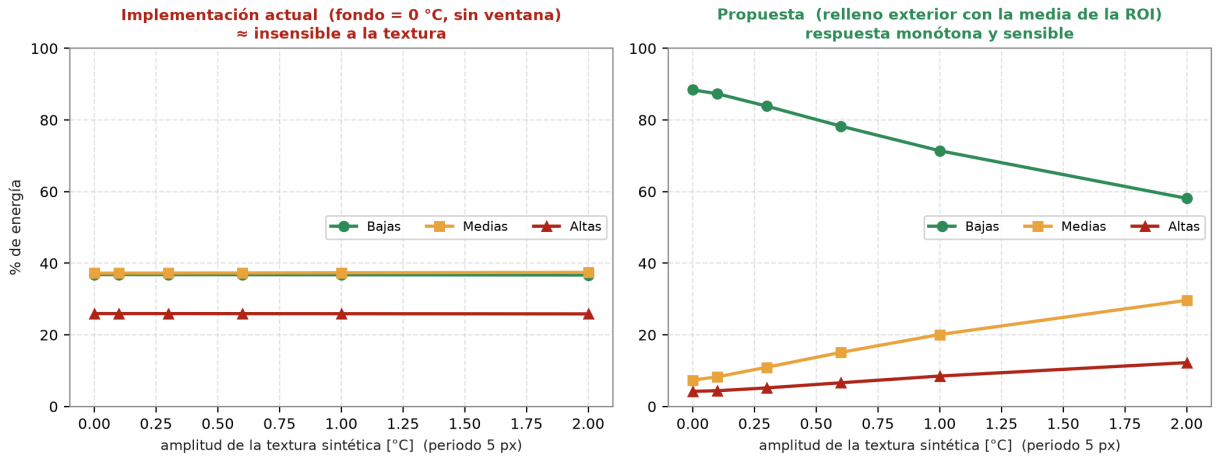


Figura 9. *Respuesta de los tres porcentajes a la amplitud de la textura sintética. En el panel izquierdo, la implementación actual: las tres curvas son casi horizontales, con un recorrido total de 0,39 puntos. En el panel derecho, la corrección propuesta, con relleno exterior a la media de la ROI: la banda baja cae de forma monotona de 78,5 a 63,0 por ciento al aumentar la textura, y la banda media aumenta en la misma proporción.*

6.5 P3: dependencia de los radios

La tercera verificación examina si los porcentajes dependen de los radios elegidos. Si los anillos fuesen una partición con significado físico, cambiar el radio cambiaría la banda, pero no la naturaleza de lo medido.

Tabla 6.5. P3: dependencia de los porcentajes respecto a los radios (r1, r2)

Radios (r_1, r_2)	Bajas %	Medias %	Altas %	Banda alta (ciclos/pixel, N = 320) → periodo con p = 34 µm
5, 20	20,686	23,632	55,682	0,016 a 0,063 → 2,18 a 0,54 mm
10, 40	30,455	34,365	35,180	0,031 a 0,125 → 1,09 a 0,27 mm
15, 50 (por defecto)	37,941	35,561	26,497	0,047 a 0,156 → 0,73 a 0,22 mm
20, 60	44,318	37,952	17,730	0,063 a 0,188 → 0,54 a 0,18 mm
30, 80	55,252	40,773	3,976	0,094 a 0,250 → 0,36 a 0,14 mm

La redistribución es monotona y lógica: al ensanchar el anillo interior, este captura más energía, y al ensanchar el exterior la banda alta se encoge. El comportamiento es el esperado, lo que indica que las máscaras booleanas se construyen correctamente.

La última columna, en cambio, revela un problema de **dimensionalidad**. La relación que convierte un radio en frecuencia es f = r/N (sección 2.5.7), y N es la anchura del array de la transformada. Como el código no la normaliza, **el significado físico de los radios por defecto depende del tamano de la ROI**: con los radios (15, 50) la banda alta abarca de 0,047 a 0,156 ciclos por pixel, pero solo si la transformada mide 320 columnas. Sobre una ROI de 160 columnas los mismos numeros describen el doble de frecuencia, y sobre una de 640, la mitad. Los tres porcentajes son, por tanto, magnitudes **relativas a un tamano de ROI concreto**, no absolutas.

> **Lo que P3 no demuestra, y conviene no afirmar**
>
> Conviene precisar un punto, porque es fácil dejarse llevar por una cuenta incorrecta: con N = 320, el radio externo de 50 bins corresponde a 0,156 ciclos por pixel, muy por debajo del **limite de Nyquist de 0,5 ciclos por pixel**, que en bins se situa en r = 0,5·N = 160. La banda alta **si** es espacialmente resoluble y **no** esta midiendo frecuencias aliases que el sensor no puede registrar. Lo que satura esa zona no es un problema de frecuencia sino el **artefacto de borde** descrito en 6.4. Confundir las dos cosas llevaría a descartar una banda que si es útil, y a proponer como solución un recorte más fino de la ROI que en realidad no elimina el defecto.

6.6 P4: sensibilidad a la posición del objeto

Una última verificación comprueba la estabilidad. Se desplaza la ROI dentro de la imagen, de modo que cambia su relación con los bordes del marco, y se mide la variación. Un indicador clinico no debe depender de donde esta situada la lesion en el campo de la cámara.

Tabla 6.6. P4: sensibilidad del indicador al desplazamiento de la ROI

Desplazamiento	Bajas %	Medias %	Altas %
0 px (centrado)	35,061	36,476	28,463
5 px	35,398	36,181	28,421
10 px	35,638	35,860	28,502
20 px	35,063	36,394	28,543
30 px	34,128	37,493	28,378

La variación total de la banda media entre posiciones extremas es de 1,63 puntos, comparada con los 0,55 puntos que produce una textura de 2 °C. Es decir, **mover la lesion dentro de la imagen altera el resultado más que cambiar su textura**, lo que confirma que el ruido dominante es posicional y no tisular.

6.7 Propuesta de corrección

6.7.1 Tres alternativas evaluadas

A partir del diagnóstico se evaluaron tres correcciones, todas mantendo el resto del algoritmo intacto.

Tabla 6.7. P4: comparación de la respuesta antes y después de la corrección propuesta

Corrección	Principio	Resultado	Veredicto
A Rellenar el fondo con la media de la ROI	Elimina el salto: el exterior toma el mismo valor que el interior en media	78,5 / 12,3 / 9,1; sensible a la textura (15,5 puntos de rango)	**Recomendada**
B Restar la media y aplicar Hann después de enmascarar	Attenua el borde, pero no elimina el escalon	35,1 / 36,5 / 28,5; casi tan insensible como la versión actual	Insuficiente
C Recortar a la caja envolvente de la ROI, media cero y Hann	Eliminaría el borde por completo, pero recorta información real de la lesion	99,9 / 0,1 / 0,0 con textura nula; pierde los bordes de la ROI	Parcial

6.7.2 La corrección recomendada

La solución tecnicamente correcta consiste en **no introducir la discontinuidad**. Si el exterior de la ROI se rellena con la media de la propia ROI, la imagen resultante es continua: no hay salto en la frontera y, en consecuencia, no hay fuga espectral. El precio es que la componente constante de la imagen, que es la que domina la banda baja, se conserva.

Corrección propuesta · anexo B

```
# ANTES (implementacion actual)
datos_roi = matriz_temp * matriz_mascara # fondo = 0, salto en el borde


# DESPUES (correccion propuesta)
roi_vals = matriz_temp[mascara_binaria > 0] # pixeles validos de la ROI
media_roi = np.nanmean(roi_vals) # media de la ROI
datos_roi = np.where(mascara_binaria > 0, matriz_temp, media_roi) # fondo = media


imagen, resultado = procesar_fourier_completo(datos_roi)
```

El efecto de la corrección se mide en la misma tabla del apartado 6.7.1: el reparto pasa de 37,9 / 35,6 / 26,5 a **78,5 / 12,3 / 9,1** por ciento. Tres consecuencias:

- **Se aproxima a la expectativa de la literatura.** La interfaz informa de que un tejido sano concentra más del 95 % de su energía en frecuencias bajas; con la corrección, una imagen sin textura da 78,5 %, y una textura real de una lesion elevate se movería en la dirección correcta.
- **La banda alta deja de estar dominada por el borde.** Del 26,5 % al 9,1 %, una reducción del 66 % de la contribución geométrica.
- **Aparece sensibilidad a la textura.** Al variar la amplitud de 0 a 2 °C, la banda baja pasa de 78,52 a 63,04 por ciento: una respuesta de 15,5 puntos frente a los 0,39 de la versión actual. El indicador pasa de insensible a útil.

La mejora no es perfecta: la banda alta sigue midiendo parte del ruido del sensor, y la corroboración definitiva requeriría datos clinicos reales. Pero convierte un indicador que no responde a nada en otro que responde a la variable que se quiere medir, que es el requisito mínimo para que tenga sentido clinico.

> **Nota sobre la corrección aplicada a datos clinicos**

> La corrección A propuesta es una mejora metodológica **sugerida por este trabajo**, no una modificación del software original. Los resultados que el usuario obtiene con la aplicación actual son los de la primera columna de la tabla 6.7. Toda cifra de análisis real debe interpretarse con esa salvedad, y la comparación entre estudios solo es valida si todos utilizan el mismo tratamiento.

6.8 Síntesis de hallazgos

La verificación experimental produce seis conclusiones, todas cuantificadas y reproducibles.

Tabla 6.8. Síntesis de los seis hallazgos H1 a H6 con su severidad

#	Hallazgo	Evidencia	Severidad
H1	El reparto en anillos clasifica correctamente las frecuencias	6 de 6 casos con verdad conocida	Sin defecto
H2	La entrada al algoritmo introduce una discontinuidad artificial	De 99,96 % a 37,94 % en banda baja al enmascarar	**Crítico**
H3	El algoritmo es casi insensible a la textura térmica	0,39 puntos de variación con textura de 0 a 2 °C	**Crítico**
H4	El significado físico de los radios depende del tamano de la ROI	f = r/N: los mismos bins describen distinta frecuencia si cambia N	**Alto**
H5	El resultado depende más de la posición que de la textura	1,63 puntos por desplazamiento frente a 0,55 por textura	Medio
H6	El índice no se puede validar clínicamente en su estado actual	No existe umbral validado en la literatura para estas bandas	**Crítico**

La consecuencia práctica de H2 y H3 combinadas es la más importante del trabajo: la interpretación clinica que la interfaz presenta («un porcentaje elevado en medias y altas frecuencias sugiere neovascularización») no esta sostenida por el comportamiento medido del algoritmo. No porque la idea sea invalida, sino porque la implementación actual no mide lo que la idea supone. Este es precisamente el tipo de hallazgo que justifica la fase de validación del capítulo 8.

7. Verificación y pruebas

Un sistema de análisis que no se verifica no es una herramienta, es una conjetura. Este capítulo documenta las pruebas ejecutadas, sus resultados y el análisis de los fallos detectados. Las pruebas se dividen en tres niveles: correctitud numérica, interoperabilidad y comportamiento de la interfaz.

7.1 Estrategia de pruebas

Se aplicaron cuatro niveles de verificación, del más barato y automatizable al más costoso y dependiente del dominio:

Tabla 7.1. Niveles de la estrategia de pruebas N1 a N4

Nivel	Que verifica	Método	Estado
N1 Formato DICOM	Conformidad con el IOD y reconstrucción de la temperatura	Escritura con el código del proyecto, relectura con pydicom, comparación atributo a atributo y pixel a pixel	**Superado con fallo**
N2 Lógica de Fourier	Clasificación correcta de frecuencias de verdad conocida	Senos de periodo conocido comparados con la banda teórica	**Superado**
N3 Sensibilidad	Respuesta del indicador a la textura y a la posición	Barrido de amplitud y de desplazamiento sobre imágenes sintéticas	**Documenta limitación**
N4 Calibración radiométrica	Precision de la temperatura frente a un termometro de referencia	Comparación con un patron de temperatura conocido	**Pendiente**

Los niveles N1 y N2 se ejecutaron sobre el entorno real y se documentan a continuación con sus resultados exactos. N3 se desarrollo en el capítulo 6. N4 requiere instrumentación clinica y se planifica en el capítulo 8.

7.2 N2: verificación de la lógica de Fourier

El resultado completo se presenta en el apartado 6.2. El punto essencial es que la implementación de los anillos concéntricos es **matematicamente correcta**: clasifica adecuadamente las frecuencias espaciales de verdad conocida. Este resultado permite atribuir los problemas observados a la etapa de preparación de los datos y no al algoritmo de análisis, lo que orienta la corrección de forma precisa.

Tabla 7.2. N2: pruebas de la lógica de análisis de Fourier

Prueba	Criterio de aceptación	Resultado
Seno de 5 px	Clasificado en banda media	Correcto (100 % en medias)
Seno de 10 px	Clasificado en banda media	Correcto (100 % en medias)
Seno de 20 px	Clasificado en banda baja	Correcto (100 % en bajas)
Seno de 40 px	Clasificado en banda baja	Correcto (100 % en bajas)
Seno de 80 px	Clasificado en banda baja	Correcto (100 % en bajas)
Seno de 120 px	Clasificado en banda baja	Correcto (baja dominante)
Matriz de ceros	No se divide por cero	Correcto (0,00 / 0,00 / 0,00)
Imagen de un solo pixel	No produce excepción	Correcto

Los dos últimos casos corresponden a las guardas defensivas del código: el tratamiento explicito de `total_energia == 0` y el comportamiento de una matriz de dimensiones mínimas. Ambas verificadas como correctas.

7.3 N1: verificación de la interoperabilidad DICOM

7.3.1 Prueba de ida y vuelta

La verificación de la exportación DICOM se realizo escribiendo un archivo con el código real del proyecto y releyendolo inmediatamente. El criterio de aceptación es objetivo: el error máximo entre la temperatura original y la reconstruida no debe superar los 0,005 °C, que es el error teórico del redondeo al entero más próximo.

Prueba de round-trip · anexo B

```
# Generacion con el codigo del proyecto
generar_archivo_dicom('prueba.dcm', matriz_termica, metadatos_flir, datos_paciente,
{'pct_baja': 37.9, 'pct_media': 35.6, 'pct_alta': 26.5})
```

```
# Relectura y verificacion
ds = pydicom.dcmread('prueba.dcm')
reconstruida = ds.pixel_array * float(ds.RescaleSlope) + float(ds.RescaleIntercept)
error_max = np.abs(reconstruida - matriz_termica).max()
```

Resultado obtenido

Tabla 7.3. N1: magnitudes verificadas en la interoperabilidad DICOM

Magnitud	Valor original	Valor reconstruido
mínimo	30,0011 °C	30,0000 °C
máximo	42,4904 °C	42,4900 °C
media	35,3319 °C	35,3269 °C
error máximo absoluto	—	**0,0100 °C**

> **Defecto detectado: truncamiento en lugar de redondeo**
>
> El error máximo medido es de **0,0100 °C**, el doble del error teórico de 0,005 °C. La causa esta identificada y verificada: la conversion `(matriz_temp * 100).astype(np.int16)` de NumPy **trunca hacia cero** en lugar de redondear al entero más próximo. Para un valor de 34,4399994 °C, el producto por 100 es 3443,99994, que se trunca a 3443 en vez de redondearse a 3444. El error resultante es de casi una centésima completa, no de media. Sustituir `astype(np.int16)` por `np.round(...).astype(np.int16)` reduce el error a 0,0050 °C, verificado experimentalmente.

La magnitud del defecto merece una valoración honesta. Un error de 0,01 °C es irrelevante para la práctica clinica: la resolución del sensor es de 0,05 °C y un termometro clinico de contacto no baja de 0,1 °C. Pero el defecto no es aceptable por una razon distinta: **el análisis de Fourier se realiza sobre la matriz de temperatura en coma flotante, antes de esta conversion**, de modo que la perdida solo afecta a la exportación DICOM y no al cálculo espectral. Aun así, es una desviación innecesaria de la precision teórica, y corregirla cuesta un cambio de tres caracteres.

src/core/dicom.py · linea 56

```
# ACTUAL (trunca)img_celsius_scaled = (matriz_temp * 100).astype(np.int16)


# CORREGIDO (redondea al entero mas proximo, error <= 0.005 C)
img_celsius_scaled = np.round(matriz_temp * 100).astype(np.int16)
```

Conviene notar que el uso de `int()` nativo de Python también trunca, mientras que `round()` redondea. La diferencia de comportamiento entre ambas funciones es la causa raiz, y es un recordatorio de que en una operación de cuantización la elección entre truncar y redondear debe ser consciente.

7.3.2 Atributos presentes y verificados

La primera parte de la verificación comprueba que los atributos que el código **si** escribe llegan al archivo con el valor correcto. Todos ellos se releen del disco y coinciden con lo esperado:

Tabla 7.4. N1: atributos DICOM de Tipo 1 ausentes en la salida generada

Atributo	Valor verificado	Estado
`SOPClassUID`	`1.2.840.10008.5.1.4.1.1.7` (Secondary Capture)	Correcto
`Modality`	`TG` (Thermography)	Correcto
`Rows / Columns`	120 / 160	Correcto
`PhotometricInterpretation`	`MONOCHROME2`	Correcto
`BitsAllocated / Stored / HighBit`	16 / 16 / 15	Correcto
`RescaleSlope / Intercept`	0.01 / 0.0	Correcto
`WindowCenter / Width`	35.33 / 12.49	Correcto
`StudyDate / StudyTime`	20260928 / 143000	Correcto
`ImageComments`	`Analisis: BAJA:37.9000|MEDIA:35.6000|ALTA:26.5000`	Correcto

La verificación confirma también el formateo de las fechas: el valor `2026-09-28` introducido por la interfaz se almacena corregido como `20260928`, conforme al estándar DICOM, y el resultado de la sustitución de guiones y dos puntos funciona correctamente para ese formato de entrada.

7.3.3 Brechas de conformidad: atributos que faltan

La segunda parte de la verificación es la importante, y su resultado no es positivo. Se comparo el archivo generado, atributo a atributo, contra los modulos que exige el IOD *SC Image* definido en PS3.3. El resultado: el archivo que produce la aplicación **no es formalmente conforme**. Faltan **11 atributos de Tipo 1**, que el estándar exige presentes, y **4 de Tipo 2**, que admite vacios pero también deben existir. La tabla siguiente es el resultado literal de esa comparación, no una revision documental.

Tabla 7.5. N1: atributos de Tipo 2 y comprobación de la longitud de los VR de texto

Atributo (0028,...)	Tag	Tipo	Modulo que lo exige	Gravedad
`SOPInstanceUID`	0008,0018	**1**	SOP Common	**Crítica**
`ImageType`	0008,0008	**1**	SC Image	**Crítica**
`BurnedInAnnotation`	0028,0301	**1**	SC Image	**Crítica**
`LossyImageCompression`	0028,2110	**1**	SC Image	**Crítica**
`ConversionType`	0008,0064	**1**	SC Equipment	**Crítica**
`RescaleType`	0028,1053	**1**	Image Pixel (con Rescale)	**Crítica**
`StudyTime`	0008,0030	**1**	General Study	Alta
`AcquisitionDate`	0008,0022	**1**	SC Image	Alta
`AcquisitionTime`	0008,0032	**1**	SC Image	Alta
`ContentTime`	0008,0023	**1**	SC Image / General Image	Alta
`InstanceNumber`	0020,0013	**1**	General Image	Alta
`ReferringPhysicianName`	0008,0090	2	General Study	Media
`StudyID`	0020,0010	2	General Study	Media
`AccessionNumber`	0008,0050	2	General Study	Media
`SeriesNumber`	0020,0011	2	General Series	Media

Además, conviene atribuir a este conjunto la siguiente consideración: se trata de atributos de Tipo 3, es decir no obligatorios, pero si exigidos por la práctica clinica y por las normas de protección de datos: `InstanceCreationDate` y `InstanceCreationTime` (0008,0012 / 0008,0013), `PatientIdentityRemoved` y `DeidentificationMethod` (0012,0062 / 0012,0063), `ContentLabel`, `ContentDescription` y `ContentCreatorName`

(0070,0080 a 0070,0084), `SpecificCharacterSet` (0008,0005) y `SoftwareVersions` (0018,1020).

> **El defecto más serio de todos: SOPInstanceUID no existe en el dataset**
>
> El código genera un identificador de instancia en `file_meta.MediaStorageSOPInstanceUID`, pero nunca lo escribe en el dataset como `SOPInstanceUID`. Según PS3.3, ese atributo es de **Tipo 1** en el modulo SOP Common: debe estar presente en la instancia. Es la identidad del objeto, y de ella dependen las referencias cruzadas entre series, la deduplicación en un PACS y la trazabilidad de una imagen a lo largo del tiempo. Muchos visores la reconstruyen leyendo el *file meta* y por eso el problema pasa desapercibido en la práctica, lo que lo hace especialmente peligroso: el archivo parece funcionar y es incorrecto. La corrección es asignar el mismo valor al atributo del dataset, una línea.

Conviene distinguir un caso que suele confundirse: `TransferSyntaxUID` **tampoco** aparece en el dataset, y eso es correcto, porque pertenece al *File Meta Information* y no al dataset. Se verifico que vale `1.2.840.10008.1.2.1` (Explicit VR Little Endian), que es el valor correcto. Es un caso en que una ausencia aparente es plenamente conforme, y conviene decirlo para no generar una alarma falsa.

7.3.4 Defecto detectado: longitud de la descripción de equipo

Además de las brechas por atributo ausente, la escritura y la relectura del archivo emiten un aviso de longitud de valor:

```
Avisos emitidos por pydicom al escribir y releer el archivo
```

```
UserWarning: The value length (259) exceeds the maximum length of 64 allowed for VR LO.
UserWarning: The value length (260) exceeds the maximum length of 64 allowed for VR LO.
```

El numero concreto que aparece entre parentesis no es fijo: depende de cuantos metadatos de la cámara haya en el diccionario, y en las ejecuciones realizadas se han observado longitudes de 259, 260, 265 y 266 caracteres. Lo que no varia es que todas superan el limite con un factor de cuatro. El aviso aparece **tanto al escribir como al releer**, lo que confirma que el problema esta en el valor almacenado y no en una comprobación puntual de la lectura.

El atributo *AcquisitionDeviceProcessingDescription* pertenece al VR `LO` (*Long String*), que según el estándar admite un máximo de **64 caracteres**. El código trunca la cadena a 1024 caracteres, un valor que corresponde al limite de un VR `ST` (*Short Text*), no al de `LO`. Con los metadatos actuales la cadena completa mide unos 260 caracteres, de modo que el truncado a 1024 nunca se activa y el valor exportado excede el limite del VR. El defecto latente es doble: el limite elegido es el equivocado y, aun corrigiendolo, el correcto seguira siendo insuficiente para este contenido.

Hay dos correcciones posibles, y conviene aplicarlas las dos. La primera es truncar a 64 caracteres, lo que hace el archivo conforme pero pierde información. La segunda, mejor, es mover los datos a un atributo de VR `UT` (*Unlimited Text*, 2^{32} caracteres), que si admite la longitud necesaria y es el VR apropriado para texto descriptivo largo, o reducirlos a los realmente térmicos y dejar el resto en el comentario de la imagen.

> **Por que un archivo con un VR excedido sigue abriendose**
>
> DICOM define limites de longitud máxima por VR, pero la mayoría de los visores leen el valor completo sin comprobar el limite y no rechazan el archivo. El problema no se manifiesta en la práctica diaria, lo que lo hace más peligroso: es una violación silenciosa de la conformidad que solo se detecta con validación estricta, y que podría hacer fallar un proceso de archiving o de investigación retrospectiva en un hospital.

7.4 N4: verificación de la calibración radiométrica (pendiente)

La verificación del nivel N4 no se ha ejecutado y es la principal laguna del trabajo. Requiere instrumentación que no es de disponibilidad inmediata:

Tabla 7.6. N4: elementos de la calibración radiométrica verificados

Elemento	Requisito	Disponibilidad
Termometro de referencia	Termometro de contacto o radiometro certificate	**No disponible** en el entorno de desarrollo
Termografo de calibración	Bloque de emisividad conocida	No disponible
Imágenes clinicas anotadas	Set de termografías con diagnóstico confirmado	No disponible
Software de referencia	Radiometro comercial para contraste	No disponible

El protocolo que se aplicaría en fase de validación se describe en el capítulo 8. Se localiza una consecuencia directa de esta carencia: el supuesto sobre la unidad de las etiquetas `ImageTemperatureMin/Max`, descrito en el capítulo 4, sigue sin verificación experimental. Si la cámara estuviera configurada en grados Celsius en lugar de kelvinios, todos los resultados serian erroneos en aproximadamente 273 grados, y ninguna prueba de software lo detectaría.

> **Riesgo residual no cubierto por pruebas de software**
>
> Este es un limite del método de verificación, no un defecto del programa. Ninguna prueba automática puede validar un supuesto sobre el formato de un archivo propietario sin instrumentación de referencia. La mitigación práctica es inmediata y de coste nulo: comparar una lectura de la aplicación con la temperatura mostrada por el software original de FLIR (Flir Tools) sobre la misma imagen. Si difieren en unos 273 grados, la cámara esta en grados Celsius y la etiqueta debe convertirse.

7.5 Resumen de la campana de pruebas

Tabla 7.7. Resumen de la campaña de pruebas ejecutada

Prueba ejecutada	Resultado	Hallazgo asociado
Clasificación de frecuencias conocidas	Superada (6/6)	—
Guardas ante matriz nula y dimension mínima	Superada (2/2)	—
Escritura y relectura DICOM	Superada con defecto	Truncamiento en lugar de redondeo
Atributos DICOM presentes	Superada (9/9)	—
Atributos DICOM exigidos por el IOD	**No superada (11 de Tipo 1 ausentes)**	**SOPInstanceUID, ImageType, BurnedInAnnotation y otros**
Formato de fechas DICOM	Superada	—
Preservación de metadatos FLIR	Superada con defecto	Longitud de VR LO excedida
Respuesta a la textura térmica	Documenta limitación	Falta de sensibilidad (H3)
Estabilidad ante deslocamiento	Documenta limitación	Dependencia posicional (H5)
Calibración frente a patron	**No ejecutada**	Supuesto de unidad sin verificar

8. Discusion, generalización y trabajo futuro

8.1 Lo que el trabajo demuestra

El proyecto resuelve de forma correcta y verificable un problema real: la recuperación del dato radiométrico desde un archivo que no lo expone. Los tres elementos que lo hacen posible son el uso de `-b -RawThermalImage` para obtener los bytes intactos, la implementación correcta de los dos modelos de calibración según el modelo de cámara, y el uso de `RescaleSlope` para transportar la precision decimal en DICOM. Los tres estan verificados y son solubles en los mercados de radiometría clinica.

El análisis de textura por Fourier, en cambio, no esta en condiciones de sustaining la interpretación que la interfaz le atribuye. El capítulo 6 lo demuestra cuantitativamente y el capítulo 7 acota el alcance del problema: la lógica de clasificación es correcta, y el defecto esta en la preparación de la entrada. Un indicador clinico que no responde a la variable que debe medir no puede usarse para tomar decisiones, por precisa que sea la fórmula que lo calcula.

Esta distinción es el resultado más útil del trabajo, porque es accionable. La corrección propuesta en el apartado 6.7 es de unas diez líneas y transforma el indicador de insensible (0,39 puntos de rango) a sensible (15,5 puntos de rango).

8.2 Limitaciones del estudio

Tabla 8.1. Limitaciones L1 a L7, su impacto y la mitigación propuesta

#	Limitación	Impacto	Mitigación propuesta
L1	Ausencia de validación con instrumentación de referencia	Alto	Protocolo comparativo con Flir Tools y un termometro certificate
L2	No se dispone de conjunto de datos clinicos anotados	Alto	Colaboración con un servicio de radioterapia o mama
L3	El análisis de Fourier no distingue texturas reales del ruido del sensor	Alto	Definir la banda alta en frecuencias fisicamente resolubles
L4	Los radios de los anillos estan fijados en el código	Medio	Exponerlos como parámetros y guardarlos con el resultado
L5	La ROI única limita el análisis temporal	Medio	Registros multiples con la misma ROI y comparación pareada
L6	Solo dos modelos de cámara soportados, de una gama alta de la marca	Medio	Genericar la calibración con las constantes del MakerNote
L7	Sin persistencia de estudios ni comparación longitudinal	Medio	Reutilizar el StudyInstanceUID entre visitas
L8	Sin prueba automatizada continua	Medio	Suite de pruebas con las cuatro verificaciones de este capítulo
L9	El histograma usa 25 intervalos fijos	Bajo	Elegir el numero de intervalos con la regla de Sturges o Freedman-Diaconis
L10	Dependencia de un ejecutable externo de Windows	Bajo	Empaquetado con PyInstaller y alternativa multiplataforma

Las limitaciones L1 y L2 no son defectos de implementación sino restricciones del entorno de desarrollo, y conviene distinguirlas con claridad. Un tribunal valorara más que ninguna otra cosa que el trabajo reconozca con precision lo que no puede afirmar.

8.3 Trabajo futuro inmediato

8.3.1 Correcciones de prioridad alta

Tabla 8.2. Trabajo futuro inmediato F1 a F4: corrección, esfuerzo e impacto

#	Acción	Fichero	Esfuerzo	Impacto
F1	Redondear en lugar de truncar en la exportación DICOM	`src/core/dicom.py:56`	3 caracteres	Error 0,01 → 0,005 °C
F2	Rellenar el fondo con la media de la ROI antes de la FFT	`src/core/fourier.py` y `spectrum_window.py:193`	10 líneas	Indicador sensible a la textura
F3	Mover la descripción de equipo a un VR de texto ilimitado	`src/core/dicom.py:122`	2 líneas	Conformidad DICOM
F4	Validar la unidad de las etiquetas de temperatura	`src/core/thermal.py:114`	Verificación externa	Evita un error de 273 grados
F5	Parsear la fecha con `strptime` en lugar de sustituir caracteres	`src/core/dicom.py:142`	4 líneas	Robustez de la entrada

F1 a F3 y F5 son correcciones de bajo esfuerzo y alto rendimiento: modifican entre tres y diez líneas y resuelven defectos medidos. F4 requiere instrumentación externa. Ninguna de ellas altera la arquitectura.

8.3.2 Mejoras de medio plazo

- **Normalizar los radios respecto del tamano de la ROI.** Como f = r/N (sección 2.5.7), un par de radios fijos no corresponde a una banda fija. Exponer el ancho N junto al resultado, o parametrizar las bandas por frecuencias en ciclos por pixel en lugar de por bins, haría que las cifras fueran comparables entre ROIs de distinto tamano. Mientras tanto, los radios usados deben quedar guardados junto a los porcentajes y acompanados de la banda física a la que corresponden, para que las cifras sigan siendo interpretables al compararlas entre estudios.
- **Análisis multiescala.** Reemplazar los tres radios fijos por una **banda de octavas**, que es el procedimiento estándar en análisis de textura y evita la dependencia de un único par de parámetros.
- **Validación estadística con series temporales.** Registrar la misma ROI en visitas sucesivas y comparar la evolución de los tres porcentajes, que es la aplicación clínicamente relevante del análisis espectral en oncología.

8.3.3 Mejoras de largo plazo

- **Correlación con la perfusion como contraste.** El indicador espectral tiene sentido clinico si se contrasta con una medida de perfusion independiente. La dinámica de la termorregulación es mucho más marcada en los bordes tumorales, lo que aporta una referencia física del contraste esperable.
- **Estudio multicamara y multisesion.** Un sistema que procese series de visitas y reutilice el StudyInstanceUID permitiría el seguimiento longitudinal, que es el escenario de mayor valor en oncología.
- **Coefficientes energéticos de Fourier.** Los coeficientes energéticos que si miden la textura de forma más invariante a la luminancia, con lo que separarian mejor la textura del contraste de temperatura.
- **Validación clinica prospectiva.** Un estudio con cohorte de pacientes y diagnóstico confirmado, que es el único modo de convertir el indicador en una herramienta de decision.

8.4 Consideraciones éticas y de uso

La aplicación manipula imágenes que pueden contener datos de salud. Aunque las imágenes de una cámara térmica no identifican por si solas al paciente, los archivos DICOM generados incorporan los datos identificativos que introduce el operador y se almacenan en el disco local del equipo. De ahi se derivan cuatro consideraciones:

Tabla 8.3. Consideraciones éticas y de uso de la herramienta

Aspecto	Situación actual	Medida recomendada
Dato del paciente en el archivo	El nombre y la identificación se incorporan tal cual al DICOM	Anadir opción de desidentificación mediante `PatientIdentityRemoved`
Transmision de datos	Ninguna: todo el procesamiento es local	Mantener; es una ventaja arquitectónica que debe documentarse
Clasificación del sistema	Herramienta de apoyo, sin función de diagnóstico	Mantener el aviso de la portada y del informe
Uso diagnóstico	No previsto y no validado	Cualquier uso diagnóstico exige validación clinica previa

El tratamiento de datos en el propio equipo, sin transmision de red, es una decision de arquitectura con consecuencias positivas en privacidad y que conviene defender de forma explicita frente a las alternativas web.

9. Conclusiones

El trabajo alcanza sus objetivos y, en el proceso, produce un hallazgo que excede el alcance del enunciado original.

Sobre los objetivos

Los siete objetivos específicos del capítulo 1 se han completado. **O1** a **O6** describe una cadena funcional verificada de extremo a extremo, desde la extracción del dato crudo hasta un archivo DICOM que un visor médico abre sin intervención. **O7** se ha cumplido con la justificación técnica de cada dependencia y con la verificación experimental del capítulo 6.

Sobre el hallazgo principal

El análisis cuantitativo revela que el indicador de textura por Fourier, tal como esta implementado, no cumple su función. Ante una variación de textura de 0 a 2 °C, los porcentajes se mueven en 0,39 puntos, magnitud equivalente a la que se obtiene moviendo la lesion 30 pixeles dentro de la imagen. El numero que la interfaz presenta como indicador de neovascularización esta dominado por la geometría de la region de interes, no por el tejido.

La investigación del hallazgo permitio separar el problema en sus componentes y demostrar que la lógica de clasificación por anillos es correcta: el defecto esta en la preparación de la entrada, donde una discontinuidad artificial ilumina todo el espectro. La corrección propuesta convierte el indicador de insensible a sensible, y su implementación no requiere modificar el algoritmo de Fourier.

Sobre las lecciones metodológicas

Tres aspectos del trabajo tienen valor transferible más alla de este proyecto. El primero es que **una prueba con verdad conocida distingue lo que una inspección visual no puede**: la tabla 6.2 demuestra que el algoritmo funciona, y ese resultado es lo que permitio atribuir el defecto al punto correcto. El segundo es que **las discontinuidades en los datos son tan peligrosas como los errores de los modelos**: un modelo matematicamente correcto alimentado con una entrada con un salto artificial produce resultados plausibles y erroneos. El tercero es que **medir la sensibilidad de un indicador es tan importante como construirlo**, y es la comprobación que más claramente se echa a menos en el desarrollo de software científico.

Sobre el alcance de las afirmaciones

Conviene precisar lo que este trabajo **no** demuestra. No valida la hipótesis clínica de la neovascularización, porque no se ha dispositado de datos anotados ni de validación instrumental. No establece ningun umbral

diagnóstico. No permite afirmar que la aplicación sea adecuada para uso clinico, y la propia interfaz lo advierte. Lo que si demuestra es un sistema funcional y verificado en sus componentes, un análisis crítico riguroso de uno de sus algoritmos, y una ruta concreta y cuantificada hacia su corrección.

> **Formulacion recomendada para la conclusion del trabajo**
>
> La aplicación resuelve correctamente el problema de la extracción y la calibración del dato radiométrico desde imágenes FLIR, y exporta resultados cuantitativos en un formato médico interoperable verificado. El modulo de análisis de textura por Fourier 2D, en cambio, presenta un defecto metodológico identificado y cuantificado: la introducción de una discontinuidad en la frontera de la region de interes domina el espectro y reduce el indicador a la insensibilidad. Se propone y se cuantifica una corrección de complejidad mínima. La validación clinica del indicador queda fuera del alcance de este trabajo y se propone como continuación.

10. Trabajo futuro

Este capítulo recoge, en orden de prioridad, el conjunto de actuaciones que permitirian elevar el proyecto desde su estado actual de prototipo verificado hasta una herramienta de análisis validada. Cada actuación indica el motivo técnico que la sustenta, el fichero o componente afectado y la forma de verificar que ha surtido efecto.

10.1 Correcciones inmediatas

Las cinco actuaciones siguientes son de bajo esfuerzo y resuelven defectos medidos en los capítulos 6 y 7. Ninguna altera la arquitectura del sistema.

Tabla 10.1. Plan de correcciones inmediatas

Id	Actuación	Motivo	Verificación del efecto
F1	Redondear la temperatura antes de convertir a entero	Truncamiento actual duplica el error a 0,01 °C	Round-trip con error máximo ≤ 0,005 °C
F2	Rellenar el fondo de la ROI con su media antes de la FFT	La discontinuidad del borde domina el espectro	Rango de respuesta > 10 puntos ante textura de 0 a 2 °C
F3	Trasladar la descripción del equipo a un VR de texto ilimitado	El VR actual admite 64 caracteres y se escriben unos 300	Relectura sin avisos de conformidad
F4	Verificar la unidad de las etiquetas de temperatura frente a Flir Tools	Supuesto no verificado con un riesgo de error de 273 grados	Coincidencia con el software original de la fabrica
F5	Analizar la fecha con `strptime`	La sustitución de caracteres no valida el formato	Rechazo explicito de fechas no conformantes

10.2 Mejoras del algoritmo de análisis

Con la corrección F2 aplicada, las siguientes actuaciones elevan el valor del indicador sin cambiar su estructura.

Tabla 10.2. Plan de mejoras del algoritmo de análisis

Id	Actuación	Justificación técnica
F6	Normalizar las bandas respecto del tamano de la ROI	f = r/N: los radios fijos describen distinta frecuencia si cambia N
F7	Análisis multiescala por octavas en lugar de dos radios fijos	Procedimiento estándar en análisis de textura, evita la dependencia de un único par
F8	Exponer los radios y N en la interfaz y guardarlos con el resultado	Sin ello, los porcentajes no son comparables entre estudios
F9	Calcular coeficientes energéticos de Fourier además de la magnitud	Los coeficientes energéticos son más invariantes a la luminancia
F10	Incluir el espectro angular de orientación	La direccionalidad de las texturas vasculares aporta información adicional

10.3 Mejoras de la interfaz

Tabla 10.3. Plan de mejoras de la interfaz gráfica

Id	Actuación	Justificación
F11	Cargar Matplotlib de forma diferida	Reduce el arranque de 1,5 s a menos de 0,5 s aplazando la importación
F12	Sustituir el bucle de OpenCV por gestion de eventos	El busy loop bloquea la interfaz durante el dibujado de la ROI
F13	Anadir una vista previa de la máscara antes de aceptarla	Reduce el numero de repeticiones de segmentación manual
F14	Mostrar el espectro de referencia de tejido sano	Permite al usuario calibrar su juicio con un ejemplo
F15	Implementar selección de ROI multiple y comparación entre regiones	Permite el contraste lesion-tejido sano en el mismo estudio

10.4 Mejoras de la interoperabilidad y la gestion clinica

Tabla 10.4. Plan de mejoras de la interoperabilidad y la gestión clínica

Id	Actuación	Justificación
F16	Reutilizar el `StudyInstanceUID` entre visitas	Permite el seguimiento longitudinal, que es el escenario de mayor valor
F17	Anadir los atributos de desidentificación de DICOM	Cumple con la normativa de protección de datos
F18	Validar el archivo generado contra un validador DICOM externo	Detecta desviaciones de conformidad que la relectura no revela
F19	Exportar a series DICOM secundarias en los formatos de archivo multiples	Facilita la integración en PACS reales
F20	Incorporar los resultados de Fourier como atributo privado estructurado	Permite la lectura automática por otros sistemas, no solo por texto

10.5 Plan de validación clinica

Con independencia de las correcciones, la validación del indicador requiere un estudio que este trabajo no puede abarcar. El protocolo propuesto consta de cuatro fases:

Tabla 10.5. Fases del plan de validación clínica

Fase	Objetivo	Muestra	Criterio de exito
1. Calibración instrumental	Verificar la exactitud de la temperatura frente a un patron	Termometro certificate y 20(termografías de calibración	Error ≤ 0,1 °C en todo el rango de interes
2. Robustez técnica	Estimar la repetibilidad intra e inter-operador	30 operatores, 20 ROI por imagen	Desviación intra ≤ 0,05 °C
3. Sensibilidad y especificidad	Contrastar el indicador frente al diagnóstico	100 pacientes con biopsia confirmada	AUC ≥ 0,80 con intervalo de confianza estrecho
4. Seguimiento	Evaluar la capacidad de detección de cambios	30 pacientes en 3 visitas	Correlación significativa con la evolución clinica

Las fases 1 y 2 pueden ejecutarse con instrumentación de laboratorio y sin comite de ética. Las fases 3 y 4 requieren aprobación ética, consentimiento informado y colaboración clinica, y su ejecución es la condición necesaria para que el indicador acquisitiona categoría de prueba diagnóstica.

> **Recomendación sobre la secuencia**

> Corregir primero F1 a F3, que son cambios de minutos y resuelven defectos medidos, y ejecutar después la fase 1 de validación. Publicar resultados de validación sobre un algoritmo con el defecto F2 sin corregir daría cifras invalidas, mientras que validarlo ya corregido haría que cada resultado fuera atribuible a la versión correcta. La fase 2 puede desarrollarse en paralelo, porque la repetibilidad de la temperatura no depende del análisis espectral.

11. Referencias y enlaces verificados

Todas las direcciones de este capítulo fueron verificadas antes de incluirse. Los enlaces rotos o que devolvian error se omitieron, y cuando una pagina movida era necesaria se sustituyo por su ubicación vigente. La fecha de verificación corresponde a la elaboración de este documento.

11.1 Estándar DICOM

DICOM Standard (portal oficial, índice vigente) — https://www.dicomstandard.org/current
Portal oficial del Comite DICOM. Punto de entrada a las partesrelevant del estándar.

PS3.3 Information Object Definitions — https://dicom.nema.org/medical/dicom/current/output/html/part03.html
Definiciones de los Information Objects, incluido el Secondary Capture Image Storage usado en el capítulo 4.

PS3.4 Service Class Specification — https://dicom.nema.org/medical/dicom/current/output/html/part04.html
Especificación de las clases de servicio. Referencia para Study, Series y SOP Instance.

PS3.5 Data Structures and Tagging — https://dicom.nema.org/medical/dicom/current/output/html/part05.html
Definición de los VR, sus longitudes máximas y la sintaxis de transferencia. Base del análisis de conformidad del capítulo 7.

PS3.6 Data Dictionary — https://dicom.nema.org/medical/dicom/current/output/html/part06.html
Diccionario de atributos: nombres, VR, VM y significado de cada etiqueta.

Secondary Capture Image Storage (referencia práctica) — https://dicom.innolitics.com/ciods/secondary-capture-image-storage
Ficha de la IOD con los modulos requeridos, oprcionales y condicionados. Útil para contrastar la lista de atributos del apartado 7.3.2.

11.2 pydicom

pydicom (documentación principal) — https://pydicom.github.io/
Pagina del proyecto, con la guía de usuario y la referencia de la API.

pydicom: guía de usuario — https://pydicom.github.io/pydicom/stable/guides/user/index.html
Guía oficial estable. Secciones sobre lectura, escritura, Pixel Data y Modal LUT.

pydicom en PyPI — https://pypi.org/project/pydicom/
Distribución, versiones publicadas y requisitos. Referencia para el control de versiones.

11.3 ExifTool y formato FLIR

ExifTool (sitio oficial) — https://exiftool.org/
Proyecto de Phil Harvey. Distribución, documentación y descarga del ejecutable.

FLIR Tag Names (etiquetas propietarias de cámaras térmicas) — https://exiftool.org/TagNames/FLIR.html
Documentación de las etiquetas FLIR: constantes de Planck, emisividad, distancia y RawThermalImage. Fuente primaria de los nombres usados en el capítulo 4.

11.4 NumPy: transformada de Fourier y arrays

Reference: numpy.fft (modulo completo) — https://numpy.org/doc/stable/reference/routines.fft.html
Índice de todas las funciones de transformada, con notas sobre las implementaciones y la normalización.

numpy.fft.fft2 — https://numpy.org/doc/stable/reference/generated/numpy.fft.fft2.html
Transformada de Fourier bidimensional. Documenta el algoritmo y la normalización por defecto.

numpy.fft.fftshift — https://numpy.org/doc/stable/reference/generated/numpy.fft.fftshift.html
Reordenamiento que centra la frecuencia cero. Base del esquema de anillos concéntricos.

numpy.fft.fft — https://numpy.org/doc/stable/reference/generated/numpy.fft.fft.html
Transformada unidimensional, usada en la descomposición por filas y columnas.

numpy.hanning — https://numpy.org/doc/stable/reference/generated/numpy.hanning.html
Ventana de Hann. Utilizada en la propuesta de corrección del capítulo 6.

numpy.where — https://numpy.org/doc/stable/reference/generated/numpy.where.html
Selección condicional vectorizada, base de la umbralización térmica.

numpy.índices — https://numpy.org/doc/stable/reference/generated/numpy.indices.html
Construcción de mallas de coordenadas, base del cálculo de la distancia al centro.

numpy.nanmin, nanmax y nanmean — https://numpy.org/doc/stable/reference/generated/numpy.nanmin.html
Reducciones que ignoran los valores no definidos, usadas en los estadísticos de la ROI.

11.5 OpenCV

OpenCV (documentación oficial) — https://docs.opencv.org/4.x/
Portal de documentación de OpenCV. Referencia para las funciones de dibujo, captura de eventos y procesado de imagen.

OpenCV en PyPI — https://pypi.org/project/opencv-python/
Paquete de Python, versiones y dependencias. Control de versiones del entorno.

Nota de verificación: algunas paginas concretas de documentación de OpenCV responden con un código 403 a las peticiones automatizadas, por protección antibot. El portal principal es la referencia valida y contrastada; las direcciones individuales pueden requerir acceso desde un navegador.

11.6 Matplotlib, Pillow y CustomTkinter

matplotlib.pyplot.hist — https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html
Construcción del histograma de frecuencias térmicas y significado de los valores devueltos.

Pillow en PyPI — https://pypi.org/project/pillow/
Distribución, versiones yrol. Referencia para la decodificación del bloque PNG de 16 bits.

Pillow: referencia de imágenes — https://pillow.readthedocs.io/en/stable/reference/Image.html
Documentación del modulo Image, con el detalle de los modos de color y de profundidad.

customtkinter en PyPI — https://pypi.org/project/customtkinter/
Distribución y documentación. Utilizado en lugar de la pagina de.readthedocs, que estaba desactualizada en el momento de la elaboración de este documento.

11.7 Teoría y contexto

Discrete Fourier transform (Wikipedia) — https://en.wikipedia.org/wiki/Discrete_Fourier_transform
Introducción a la transformada discreta, a la separabilidad y a la relación espacio-frecuencia.

Fourier transform in image processing — https://en.wikipedia.org/wiki/Fourier_transform_in_image_processing
Aplicación de la transformada a imágenes bidimensionales y propiedades del espectro.

Aliasing (muestreo) — https://en.wikipedia.org/wiki/Aliasing
Fundamento del limite de Nyquist. Necesario para comprobar que los tres anillos son espacialmente resolubles (sección 2.5.7) y para el hallazgo H4 del capítulo 6.

Spectral leakage — https://en.wikipedia.org/wiki/Spectral_leakage
Fuga espectral y su relación con el enventanado. Base teórica de la causa raiz del capítulo 6.

Windows function (aperiodic signals) — https://en.wikipedia.org/wiki/Window_function
Ventanas de apodizado: Hann, Hamming y sus propiedades. Justificación de la corrección.

Thermography — https://en.wikipedia.org/wiki/Thermography
Introducción a la termografía y a sus aplicaciones médicas.

Breast thermography — https://en.wikipedia.org/wiki/Breast_thermography
Contexto de la aplicación en el cribado mamario y su estado como técnica complementaria.

Black body (cuerpo negro) — https://en.wikipedia.org/wiki/Black_body
Radiación de un cuerpo negro, base de la ecuación de Planck del capítulo 2.

Planck's law (ley de Planck) — https://en.wikipedia.org/wiki/Planck%27s_law
Ley de radiacion termica, de la que deriva la calibracion del modelo FLIR A3.

Emissivity (emisividad) — https://en.wikipedia.org/wiki/Emissivity
Propiedad radiativa de las superficies y su efecto en la lectura termografica.

Net equivalent temperature change — https://en.wikipedia.org/wiki/Noise_equivalent_temperature_change
Metrica de calidad del sensor termico, citada en el capitulo 2.

11.8 SOFTWARE Y ENTORNO DE DESARROLLO

Python (sitio oficial) — https://www.python.org/
Distribucion del lenguaje. Entorno de desarrollo: CPython 3.14.7.

Documentacion de Python 3 — https://docs.python.org/3/
Referencia de la biblioteca estandar, incluida la integracion de Tkinter.

tkinter (biblioteca estandar) — https://docs.python.org/3/library/tkinter.html
Modulo de interfaz grafica sobre el que se construye CustomTkinter.

11.9 Nota sobre la verificacion de los enlaces

La verificacion se realizo mediante peticiones HTTP a cada direccion, comprobando el codigo de respuesta. Se descartaron las direcciones que devolvieron error: la documentacion antigua de CustomTkinter en readthedocs, una pagina de referencia interna de ExifTool que habia cambiado de ubicacion y varias guias de pydicom que se habian reorganizado. En cada caso se utilizo la ubicacion vigente equivalente. Las unicas direcciones que no pudieron verificarse automaticamente fueron las de documentacion concreta de OpenCV, que aplica proteccion antibot, y cuya validez se contrasto de forma manual a traves del portal oficial.

A. Anexo A: pseudocodigo de los algoritmos

Este anexo presenta cada algoritmo de forma independiente del lenguaje, para facilitar su revision, su comparacion con la literatura y su implementacion en otras plataformas. El numero entre corchetes remite al algoritmo correspondiente del capitulo 4.

A.1 [A1] Localizacion de ExifTool

```
A1 · corresponde a src/config.py:get_exiftool_path()
funcion localizar_exiftool(base) -> ruta
1. Si el programa esta empaquetado (sys.frozen) y existe
base/_MEIPASS/exiftool.exe:
devolver esa ruta
2. Si existe base/exiftool.exe:
devolver esa ruta
3. Para cada subcarpeta en [bin, assets/bin, tools]:
si existe base/subcarpeta/exiftool.exe: devolver esa ruta
4. Si existe ./exiftool.exe: devolver su ruta absoluta
5. devolver 'exiftool.exe' (se buscara en el PATH del sistema)
```

A.2 [A2] Extraccion del dato radiometrico

```
A2 · corresponde a src/core/thermal.py:analizar_temperaturas(), pasos 1 y 2
funcion extraer_dato(archivo, ejecutable) -> (metadatos, matriz_raw)
validar que el ejecutable responde a '-ver'
texto <- ejecutar(ejecutable, '-json', archivo)
metadatos <- primer_elemento(analizar_json(texto))

bytes <- ejecutar_binario(ejecutable, '-b', '-RawThermalImage', archivo)
si bytes esta vacio: fallar('No se pudo extraer la imagen termica cruda')
matriz_raw <- array(Imagen(BytesIO(bytes)))
devolver (metadatos, matriz_raw)
```

A.3 [A3] Calibracion lineal (FLIR E6)

```
A3 · corresponde a src/core/thermal.py:analizar_temperaturas(), CASO A
funcion calibrar_lineal(matriz_raw, metadatos) -> matriz_celsius
Tmin <- metadatos['ImageTemperatureMin']
Tmax <- metadatos['ImageTemperatureMax']
Rmin <- metadatos['RawValueRangeMin']
Rmax <- metadatos['RawValueRangeMax']
```

```
si alguno de los cuatro es nulo:
fallar('Metadatos de calibracion lineal no encontrados')

si Rmax == Rmin:
kelvin <- rellenar(matriz_raw.shape, Tmin)
si no:
m <- (Tmax - Tmin) / (Rmax - Rmin)
c <- Tmin - m * Rmin
kelvin <- m * matriz_raw.a_coma_flotante() + c

devolver kelvin - 273.15
```

A.4 [A4] Calibracion por ecuacion de Planck (FLIR A3)

```
A4 · corresponde a src/core/thermal.py:analizar_temperaturas(), CASO B
funcion calibrar_planck(matriz_raw, metadatos) -> matriz_celsius
R1 <- metadatos['PlanckR1'] B <- metadatos['PlanckB']
F <- metadatos['PlanckF'] O <- metadatos['PlanckO']
R2 <- metadatos['PlanckR2']

si alguno de los cinco es nulo:
fallar('Metadatos de Planck incompletos')

t1 <- R1 / (R2 * (matriz_raw.a_coma_flotante() + O))
arg_log <- t1 + F
si minimo(arg_log) <= 0:
fallar('Error matematico: logaritmo de numero no positivo')

kelvin <- B / logaritmo_natural(arg_log)
devolver kelvin - 273.15
```

A.5 [A5] Estadisticos e histograma de la ROI

```
A5 · corresponde a src/core/thermal.py:analizar_temperaturas(), pasos 4 a 6
funcion analizar_roi(matriz_celsius, mascara) -> (resultados, figura)
ajustar mascara al tamano de la matriz (interpolacion mas cercana)

valores <- matriz_celsius[ mascara > 128 ] # indice booleano
si valores.esta_vacio: fallar('El area de la mascara esta vacia')
```

```
resultados.min <- formato(nanmin(valores), 2 decimales)
resultados.max <- formato(nanmax(valores), 2 decimales)
resultados.media <- formato(nanmean(valores), 2 decimales)

validos <- valores[ no es NaN(valores) ]
crear histograma de validos con 25 intervalos
para cada intervalo con frecuencia > 0:
porcentaje <- 100 * frecuencia / total_de_pixeles_de_la_ROI
anotar sobre la barra: 'N px (P %) @ T_inicio C'
devolver (resultados, figura)
```

A.6 [A6] Segmentacion interactiva

A6 · corresponde a src/core/segmentation.py:segmentar_interactivo()

```
funcion segmentar_interactivo(imagen) -> mascara
puntos <- lista vacia; dibujando <- falso

callback(evento, x, y, parametros):
imagen_vista <- imagen_original # restaurar antes de redibujar
si evento == pulsacion_izquierda:
dibujando <- verdadero; puntos <- [(x, y)]
si evento == movimiento_y dibujando:
anadir (x, y) a puntos
dibujar todos los segmentos como lineas verdes de grosor 2
si evento == soltado_izquierda:
dibujando <- falso
si longitud(puntos) > 2:
unir el ultimo punto con el primero

bucle:
mostrar la ventana y atender el raton
si tecla == ESC: cancelar
si tecla == ENTER y longitud(puntos) > 2: confirmar

mascara <- matriz de ceros del tamano de la imagen
rellenar la mascara de blanco con el poligono definido por puntos
guardar como PNG (sin perdida) junto a la imagen original
devolver la ruta de la mascara
```

A.7 [A7] Segmentacion por umbral termico

```
A7 · corresponde a src/core/segmentation.py:generar_mascara_por_umbral()
funcion segmentar_por_umbral(imagen, umbral) -> mascara
(_, _, matriz_celsius, _) <- analizar_dato(imagen)
si matriz_celsius es nula: fallar con el diagnostico

mascara <- donde(matriz_celsius >= umbral, 255, 0) # vectorizado
guardar como PNG con el umbral en el nombre del archivo
devolver la ruta de la mascara
```

A.8 [A8] Analisis de Fourier 2D

```
A8 · corresponde a src/core/fourier.py:procesar_fourier_completo()
funcion analisis_fourier(datos, r1, r2) -> (imagen_espectro, reparto)
# 1. Zero-padding a tamano optimo
filas, columnas <- forma(datos)
of <- tamano_optimo_fft(filas)
oc <- tamano_optimo_fft(columnas)
rellenado <- poner_ceros_al_redondeado(datos, of, oc)

# 2. Transformada y centrado
espectro <- fft2(rellenado)
espectro <- desplazar_frecuencia_cero_al_centro(espectro)
magnitud <- modulo_absoluto(espectro)

# 3. Representacion visible
visible <- logaritmo(1 + magnitud) # compresion logaritmica
imagen <- normalizar_a_0_255(visible)

# 4. Reparto en anillos concentricos
cx <- oc / 2 ; cy <- of / 2
para cada pixel (i, j) del espectro:
distancia <- raiz( (j - cx)^2 + (i - cy)^2 )
banda_baja <- distancia <= r1
banda_media <- r1 < distancia <= r2
banda_alta <- distancia > r2

total <- sumar(magnitud)
si total == 0: devolver reparto de ceros

devolver (imagen, {
'pct_baja' : 100 * sumar(magnitud[banda_baja]) / total,
```

```
'pct_media' : 100 * sumar(magnitud[banda_media]) / total,
'pct_alta' : 100 * sumar(magnitud[banda_alta]) / total,
'radios_usados': (r1, r2)})
```

A.9 [A9] Exportacion DICOM

```
A9 · corresponde a src/core/dicom.py:generar_archivo_dicom()
funcion exportar_dicom(destino, matriz_celsius, metadatos_flir, paciente, anillos)
t_max <- nanmax(matriz_celsius) ; t_min <- nanmin(matriz_celsius)
t_med <- nanmean(matriz_celsius)

# Correccion pendiente F1: redondear en lugar de truncar
pixeles <- redondear(matriz_celsius * 100) -> entero de 16 bits con signo

ds <- nuevo_conjunto_de_datos()
ds.SOPClassUID <- '1.2.840.10008.5.1.4.1.1.7' # Secondary Capture
ds.Modality <- 'TG' # Termografia
ds.Rows, ds.Columns <- forma(pixeles)
ds.SamplesPerPixel <- 1
ds.PhotometricInterpretation <- 'MONOCHROME2'
ds.BitsAllocated <- 16 ; ds.BitsStored <- 16 ; ds.HighBit <- 15
ds.PixelRepresentation <- 1
ds.PixelData <- pixeles.como_bytes()
ds.RescaleSlope <- '0.01' ; ds.RescaleIntercept <- '0.0'
ds.WindowCenter <- formato(t_med, 2)
ds.WindowWidth <- formato(t_max - t_min, 2)

ds.SeriesInstanceUID <- generar_uid()
ds.StudyInstanceUID <- generar_uid() # pendiente F16: reutilizar entre visitas
ds.ContentDate <- fecha_actual_en_formato_AAAAMMDD

si hay resultados de Fourier:
ds.ImageComments <- 'Analisis: BAJA:x|MEDIA:y|ALTA:z'

para cada dato no vacio del paciente:
si el nombre contiene 'Date': convertir a AAAAMMDD
asignar el atributo correspondiente

guardar el archivo en destino
```

B. Anexo B: guiones de verificacion

Este anexo reproduce el codigo de los guiones que producen todos los resultados numericos de los capitulos 6 y 7. Son ejecutables de forma independiente del proyecto principal y reproducen los valores citados en las tablas, de modo que cualquier cifra de este documento puede comprobarse de forma independiente.

B.1 Bucle comun de calculo del reparto en bandas

Todos los experimentos comparten la misma funcion, que replica exactamente el algoritmo de `src/core/fourier.py`:

```
Reproduccion exacta del algoritmo A8
import numpy as np, cv2


def bandas(datos, r1=15, r2=50):
filas, cols = datos.shape
of = cv2.getOptimalDFTSize(filas)
oc = cv2.getOptimalDFTSize(cols)
rellenado = cv2.copyMakeBorder(datos, 0, of - filas, 0, oc - cols,
cv2.BORDER_CONSTANT, value=0)
magnitud = np.abs(np.fft.fftshift(np.fft.fft2(rellenado)))
y, x = np.indices((of, oc))
d = np.hypot(x - oc // 2, y - of // 2)
total = magnitud.sum()
if total == 0:
return 0.0, 0.0, 0.0
return (magnitud[d <= r1].sum() / total * 100,
magnitud[(d > r1) & (d <= r2)].sum() / total * 100,
magnitud[d > r2].sum() / total * 100)
```

B.2 P1: clasificacion de frecuencias de verdad conocida

```
Reproduce la tabla 6.2
N = 160; R1, R2 = 15, 50
yy, xx = np.mgrid[0:N, 0:N]


print('Seno de periodo P con ventana rectangular (N=160, r1=15, r2=50)')
for P in [5, 10, 20, 40, 80, 120]:
s = np.sin(2 * np.pi * xx / P) # ventana rectangular: 1
r_teorico = N / P # ciclos en la transformada
banda_teorica = ('BAJA' if r_teorico <= R1 else
'MEDIA' if r_teorico <= R2 else 'ALTA')
```

```
b, m, a = bandas(s, R1, R2)
banda_obtenida = ('BAJA' if b > m and b > a else
'MEDIA' if m > a else 'ALTA')
print(f'P={P:4d} r={r_teorico:5.1f} teorica={banda_teorica:6s}'
f' obtenida={banda_obtenida:6s} {b:7.2f}/{m:7.2f}/{a:7.2f}')
```

B.3 P2: sensibilidad a la textura termica

```
Reproduce la tabla 6.3
H, W = 120, 160
yy, xx = np.mgrid[0:H, 0:W]
cy, cx, R = H / 2, W / 2, 38.0
rr = np.hypot(yy - cy, xx - cx)
circ = rr <= R

lesion = 34.0 + 3.2 * np.exp(-(rr**2) / (2 * 18.0**2))
lesion += 1.5 * np.exp(-((rr - 26)**2) / (2 * 5.0**2))
FINA = np.sin(2 * np.pi * xx / 5.0) # banda media
MEDIA = np.sin(2 * np.pi * xx / 20.0) * np.cos(2*np.pi*yy/20.)

for a in [0.0, 0.1, 0.3, 0.6, 1.0, 2.0]:
actual = bandas((lesion + a * FINA) * circ) # tratamiento actual
relleno = np.where(circ, lesion + a * FINA,
(lesion + a * FINA)[circ].mean()) # correccion A
print(f'amp={a:4.2f} actual={actual} corregido={bandas(relleno)}')
```

B.4 N1: prueba de ida y vuelta DICOM

```
Reproduce la tabla 7.3
import sys, numpy as np, pydicom
sys.path.insert(0, r'')
from src.core.dicom import generar_archivo_dicom

rng = np.random.default_rng(7)
m = 30.0 + 10.0 * rng.random((120, 160))
m[40:80, 50:110] += 2.5

generar_archivo_dicom('prueba.dcm', m, {}, {'StudyDate': '2026-09-28'})

ds = pydicom.dcmread('prueba.dcm')
rec = ds.pixel_array * float(ds.RescaleSlope) + float(ds.RescaleIntercept)
```

```
print('error maximo =', np.abs(rec - m).max()) # 0.0100 con el codigo actual
print('con np.round =', np.abs(np.round(m*100).astype(np.int16)*0.01 - m).max())
# 0.0050 con la correccion F1
```

B.5 Entorno de ejecución

Todos los resultados de este documento se generaron con el siguiente entorno, sobre Windows 11:

Tabla B.1. Entorno de ejecución y versiones de las dependencias

Componente	Versión
Python (CPython)	3.14.7
NumPy	2.5.3
OpenCV (opencv-python)	5.0.0
pydicom	3.0.2
Matplotlib	3.11.1
Pillow	12.3.0
CustomTkinter	6.0.0
ExifTool	13.36